In-Circuit
Programming Manual

PUBLICATION ISSUED BY:

Via Kennedy, 24
10019 STRAMBINO (TO)
Tel.: +39 0125 63 68 11
Fax: +39 0125 63 68 99
e-mail: seica@seica.com
Web site: http://www.seica.com

Copyright © 2004
All rights reserved

Microsoft Windows is a registered trademark of Microsoft Corporation

Document code: MA-LAFPICEN-02

Applicable to: S20WIN Software, from rel. 3.11

Date of publication: 2004 - 09 - 23

Seica S.p.A. reserves the right to change the technical specifications without notice.

1. GENERAL INFORMATION 1-1

2. HARDWARE ARCHITECTURE 2-1

2.1 PC 2-2
2.2 INTERFACE CARD (TIMIO) 2-2
2.3 ATE 2-2
2.3.1 Instruments board (LAM) 2-2
2.3.2 Channel board (S64) 2-4
2.3.3 Channel board (P32) 2-5
2.3.4 Board (SCAFP) 2-5
2.3.5 User power supply and User power supply control card (SERVIZI) 2-6

3. TESTING METHODS 3-1

3.1 "STANDARD" IN-CIRCUIT MEASUREMENTS 3-1
3.1.1 Resistor measurement 3-1
3.1.2 Capacitor measurement 3-3
3.1.3 Inductor measurement 3-4
3.1.4 Diode and Zener diode measurement 3-5
3.1.5 Transistor measurement 3-7
3.1.6 Mosfet measurement 3-9
3.1.7 Relay measurement 3-10
3.1.8 Transformer measurement 3-12
3.1.9 Digital component measurement 3-12
3.2 THE ANALOG GUARDING TECHNIQUE 3-14
3.3 THE DIGITAL GUARDING TECHNIQUE 3-15
3.4 SEQUENCES OF MEASUREMENTS 3-16
3.5 TYPES OF MEASUREMENT 3-16
3.5.1 2 lines, 2 points measurement 3-17
3.5.2 2-lines, 2 points measurement with guarding 3-17
3.5.3 3-lines measurement 3-18
3.5.4 3-lines, 2 points measurement with guarding 3-18
3.5.5 Double read measurement 3-19
3.5.6 4 points, double read measurement 3-19
3.6 IN-CIRCUIT AC MEASUREMENTS 3-20
3.6.1 Capacitor AC measurement 3-20
3.6.2 Inductor AC measurement 3-21
3.6.3 Capacitor and inductor in parallel 3-22

4. IN-CIRCUIT PROGRAMMING LANGUAGE 4-1

4.1 INSTRUCTIONS 4-2
if-else-endif 4-3
switch-case-endswitch 4-4
for-endfor 4-5
goto-label 4-6
label 4-7
$include 4-8
$section 4-9
GLOBAL 4-10
LOCAL 4-11
4.2 COMMANDS 4-12
ALM 4-15
BEE 4-16
BER 4-17
BOK 4-18
CFE 4-19
CKN 4-20
CLE 4-21
D2P 4-22
DEB 4-23

DR2 4-24
DRI 4-25
DRM 4-26
DRV 4-27
EB1 4-28
EB2 4-29
ELM 4-30
ESF 4-31
EXG 4-32
EXS 4-33
FAR 4-34
FIM 4-35
FLP 4-36
FLU 4-37
FLY 4-38
FRC 4-39
GND 4-40
GUA 4-41
HIG 4-42
HIM 4-43
HLD 4-44
ICH 4-45
INV 4-46
JER 4-47
JOK 4-48
JUM 4-49
LAB 4-50
LI1, LI2, LI3, LI4 4-52
LN1 4-53
LN2 4-54
LN3 4-55
LN4 4-56
LNK 4-57
LOW 4-58
MAX 4-59
MES 4-60
MMD 4-61
MOV 4-63
MRC 4-64
NCK 4-66
NER 4-67
NRC 4-68
ONE 4-69
OPC 4-70
POL 4-71
POP 4-72
PRN 4-73
PSH 4-74
PUL 4-75
PUM 4-76
PW1, PW2, PW3, PW4, PW5, PW6 4-77
RET 4-78
SCA 4-79
SCR 4-80
SER 4-81
SOK 4-82
STP 4-83
SUB 4-84
SYN 4-85
TIM 4-86

UPG 4-87
VCC 4-88
VIM 4-89
VIW 4-90
WAI 4-91
WRT 4-93
ZER 4-94
4.3 MACROS 4-95
4.3.1 System macro in the File.srg: 4-96
@AURES 4-97
@AUTIC 4-98
@CASE 4-99
@CLUSTER 4-101
@CON 4-103
@CONT 4-105
@CTO 4-106
@FIX 4-107
@ISO 4-108
@ISOHV 4-109
@ISOV 4-110
@OPICN/P 4-111
@PAR 4-113
@PINAD 4-115
@RESHV 4-116
@RESV 4-117
@SHORT 4-118
@SHOW 4-119
@VIS 4-120
@Z 4-121
@ZENHV 4-123
4.3.2 System macro in the library file: 4-124
@C 4-125
@IEEE 4-126
@L 4-127
@PAT 4-128
4.3.3 System macros written in both files .srg File and .lib File: 4-130
@APPEND 4-131
@CALC 4-132
@COMPARE 4-133
@CONV 4-134
@COPY 4-135
@DEF 4-136
@INPUT 4-137
@OUTPUT 4-138
@SUBCOMP 4-139
4.3.4 User Macro: 4-140
User macro # 4-141
User macro & 4-142
User macro % 4-143
4.4 NUMERIC SYSTEM VARIABLES 4-144
4.5 SYSTEM ALPHANUMERIC VARIABLES 4-144

5. MACRO DEVELOPMENT 5-1

6. TEST PROGRAM DEVELOPMENT 6-1

6.1 TEST PROGRAM DEVELOPMENT WITHOUT CAD DATA 6-2
6.1.1 FLYING PROBE SYSTEMS 6-2
6.1.1.1 Board configuration 6-2
6.1.1.2 Autolearn the coordinates 6-2
6.1.1.3 Modification of the file.par (that contains the component parameters) 6-3

6.1.1.4 Generation of the database (file.SHO) for the short test 6-3
6.1.1.5 Creation of the source file (File.srg) 6-3
6.1.1.6 Optimization of the test program 6-3
6.1.1.7 Compilation of a test program 6-3
6.1.1.8 Execution of a test program 6-4
6.1.1.9 Debug of a test program 6-4
6.1.2 BED of NAIL SYSTEMS 6-4
6.1.2.1 Configuration of the board 6-4
6.1.2.2 Creation of the source file (File.srg) 6-4
6.1.2.3 Optimization of the test program 6-4
6.1.2.4 Compilation of a test program 6-4
6.1.2.5 Execution of a test program 6-4
6.1.2.6 Debug of a test program 6-5
6.2 TEST PROGRAM DEVELOPMENT WITH CAD DATA 6-5
6.2.1 FLYING PROBE SYSTEMS 6-5
6.2.1.1 Creation of the board 6-5
6.2.1.2 Configuration of the board 6-5
6.2.1.3 Import data from CAD environment 6-5
6.2.1.4 Generation of the database (file.SHO) for the short test 6-6
6.2.1.5 Generation of the source file (File.srg) 6-6
6.2.1.6 Optimization of the test program 6-6
6.2.1.7 Compilation of a test program 6-6
6.2.1.8 Board orientation check 6-6
6.2.1.9 Translation of the coordinates 6-6
6.2.1.10 Execution of a test program 6-7
6.2.1.11 Debug of a test program 6-7
6.2.2 BED OF NAILS SYSTEMS 6-7
6.2.2.1 Creation of the board 6-7
6.2.2.2 Configuration of the board 6-7
6.2.2.3 Import data from CAD environment 6-7
6.2.2.4 Generation of the source file (File.srg) 6-7
6.2.2.5 Optimization of the test program 6-7
6.2.2.6 Compilation of a test program 6-8
6.2.2.7 Board orientation check 6-8
6.2.2.8 Execution of a test program 6-8
6.2.2.9 Debug of a test program 6-8
6.3 TEST PROGRAM DEBUG 6-8
6.3.1 General suggestion for the manual debug 6-8
6.3.1.1 To debug a RESISTOR 6-9
6.3.1.2 To debug a CAPACITOR (using the #CON, #CAP macro) 6-9
6.3.1.3 To debug a CAPACITOR (using the #CON_AC) 6-9
6.3.1.4 To debug a CAPACITOR (using the @CON macro) 6-9
6.3.1.5 To debug a INDUCTOR (using the #IND macro) 6-10
6.3.1.6 To debug a INDUCTOR (#IND_AC macro) 6-10
6.3.1.7 To debug a DIODE 6-10
6.3.1.8 To debug a TRANSISTOR 6-10
6.3.1.9 To debug a MOS 6-10
6.3.1.10 To debug a RELAY 6-10

7. UNDER BOARD TEST AND SYSTEM FILES 7-1

7.1 THE .ADJ FILE 7-2
7.2 THE .AUT FILE 7-3
7.3 THE .CON FILE 7-6
7.4 THE .LIB FILE 7-6
7.5 THE .PAR FILE 7-8
7.6 THE .NOD FILE 7-9
7.7 THE .POL FILE 7-10
7.8 THE .SHI FILE 7-10
7.9 THE .SHO FILE 7-11
7.10 THE .SRG FILE 7-11

7.10.1 Example of File.srg 7-12
7.11 THE .TAB FILE 7-14
7.12 THE S20.CAD FILE 7-15
7.13 THE S20ANA.LIB FILE 7-19

8. APPENDIX A 8-1

8.1 SYSTEM EDITOR 8-1
8.1.1 MAIN COMMANDS 8-2
8.1.2 STRING SEARCH 8-4
8.1.3 STRING SUBSTITUTION 8-6

Section 1

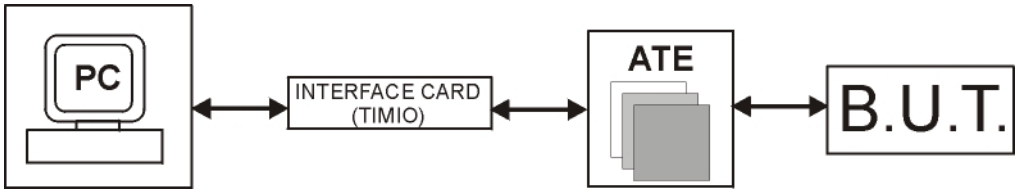
1. GENERAL INFORMATION

This manual shows the in-circuit test type, the technique, and the programming language employable on flying probe systems or systems with bad of nails adapter. The showed test technique is referred to system measure section, independently from the connection mode with the unit under test. The information relative to test technique, contains useful suggestions about practical application of these techniques.
Information about every single command and programming instruction, are besides described with the syntax and examples.
This manual contains references to *User Manual* about the Microsoft Operative System installed on the test system PC.

Section 2

2. HARDWARE ARCHITECTURE

This section presents and describe the hardware architecture of the test system, showing the essential elements for work, giving a valid guide support for understand and apply the commands and instructions presented in the following sections. The most important characteristic of the boards that constitute the ATE, are besides supplied.



2.1 PC

The personal computer handles all the system operations, and via S20 Windows software, lets the user use all the system expedients. The minimum PC characteristics required are:

- CPU: Intel Celeron 2 GHz
- RAM: 256 Mbyte
- HDU 40 Gb
- FDU 3,5” - 1,44 Mb
- CD ROM
- Monitor: 15”/17” CRT or 15” LCD colors display
- Keyboard and mouse
- OS Window NT 4.0 or Window 2000 Pro

2.2 Interface Card (TIMIO)

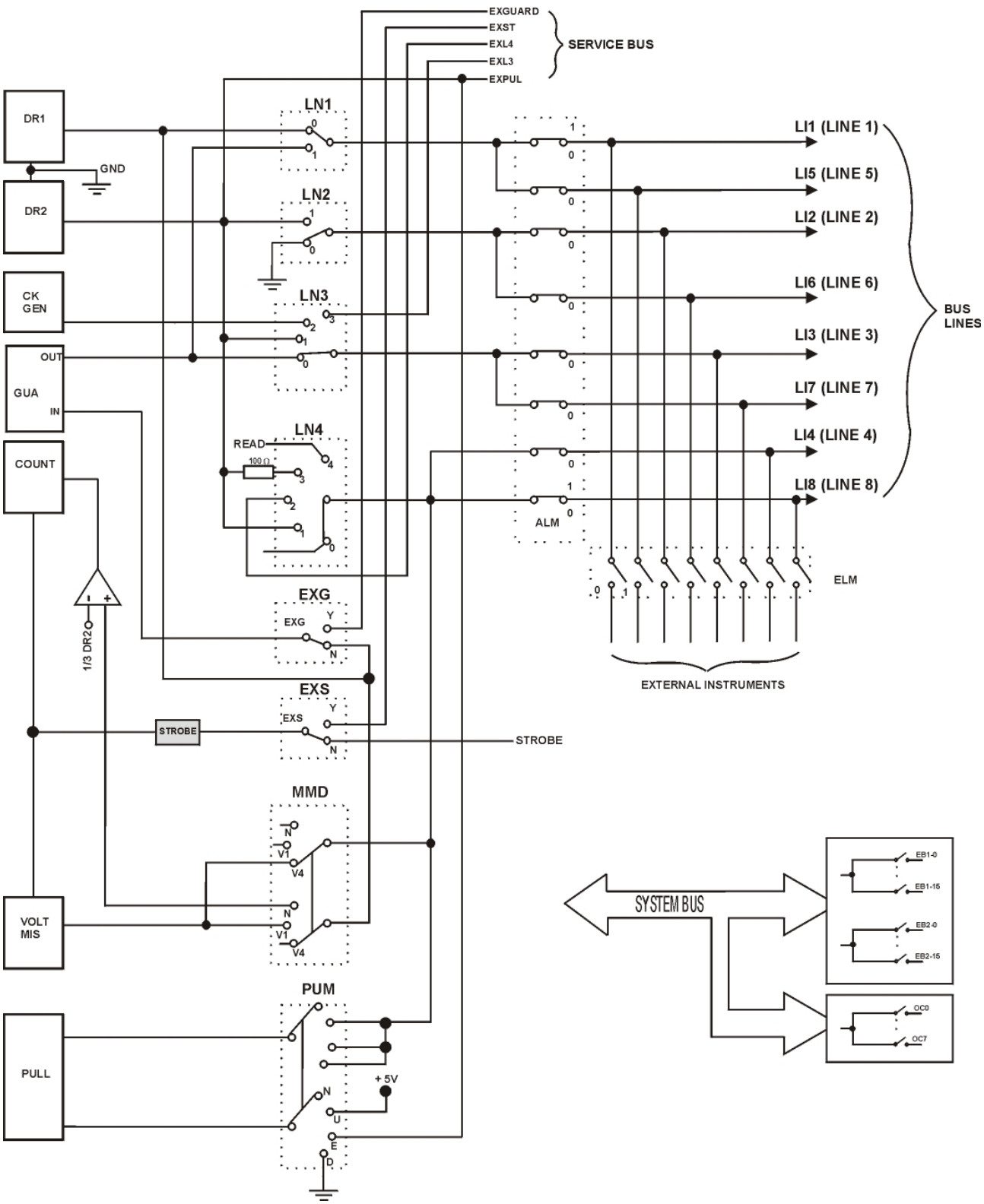
Control the ATE - PC communication.

2.3 ATE

The ATE group is constituted by all the elements that are essential for the test of a board. The instruments board, the channel boards, the user power supply manager board and other are in fact installed in ATE rack.
The most important element of ATE group is the instruments board, that have the stimulate and measure elements useful for the test.

2.3.1 Instruments board (LAM)

The LAM is the most important ATE instruments board, and looking the block diagram is clear that is the most complex. The LAM includes voltage/current generator, clock generator, pulse counter, voltage measurement, guarding generator and pull up/down resistor. In the following a LAM block diagram is showed with the instruments characteristics.
The LAM instruments are connected with the UUT nets via a relay matrix, represented by the channel board/s. Depending on the kind of test, the LAM instruments are connected with UUT nets by closing some relay, that are drived by S20 Windows software.



DR1 VOLTAGE/CURRENT GENERATOR INSTRUMENT

Current generator:	25nA	÷ 100µA	Accuracy 25nA
	100µA	÷ 1mA	Accuracy 250nA
	1mA	÷ 10mA	Accuracy 2.5µA
	10mA	÷ 100mA	Accuracy 25µA
Clamp voltage:	-5V	÷ +5V	Accuracy 5mV
	-1V	÷ +1V	Accuracy 0.5mV
Programmable voltage generator:	-50V	÷ +50V	Accuracy 25mV
	-10V	÷ +10V	Accuracy 5mV
Clamp current:	1mA - 10mA - 100mA - 200mA - 300mA - 400mA - 500mA		

DR2 VOLTAGE GENERATOR INSTRUMENT

Voltage:	-10.240V	÷ +10.235V	Accuracy 5mV
Clamp voltage:	1A		

CKGEN CLOCK GENERATOR INSTRUMENT

Cycle:	500ns	÷ 60s	Accuracy 500ns
Pulse per step:	1	÷ 32767	
Clamp current:	500mA		
Voltage:	5V		
Rise time:	30ns		
Fall time:	30ns		

GUA GUARDING GENERATOR INSTRUMENT

Auxiliary generator for realize the fair potential between two or more nodes of the unit under test.

Current:	1A max	
Voltage:	-10V	÷ +10V

VOLTMIS VOLTAGE MEASURER INSTRUMENT

Voltage:	-2V	÷ +2V	Accuracy 1mV
	-10V	÷ +10V	Accuracy 5mV
	-50V	÷ +50V	Accuracy 25mV
Measure strobe:	2µs	÷ 60s	Accuracy 500ns

Depending on the measurement scale used, the input impedance of the measurer instrument may be:

-2V	÷ +2V	Rin = 10^{10} Ω
-10V	÷ +10V	Rin = 150K Ω
-50V	÷ +50V	Rin = 125K Ω

COUNT PULSE GENERATOR INSTRUMENT

Time interval:	500ns	÷ 60s	Accuracy 500ns
Input voltage:	±5V		
Threshold voltage:	1/3 VDR2		
Count:	up to 4095		

PULL PROGRAMMABLE RESISTANCE MATRIX INSTRUMENT

Range:	100Ω	÷ 1,5KΩ	Accuracy 100Ω
	1KΩ	÷ 15KΩ	Accuracy 1KΩ
	10KΩ	÷ 0,15MΩ	Accuracy 10KΩ
	100KΩ	÷ 1,5MΩ	Accuracy 100KΩ

2.3.2 Channel board (S64)

The channel card performs the connection between the instrument card (LAM) and the board nodes. Each S64 card has got 64 not multiplexed channels for the connection to the board nodes. Each channel has 4 internal lines for switching the integrated instrumentation. The channels have the following characteristics:

Current:	1A max.
Voltage:	100V max.
Power:	10W max.
Maximum frequency measurable:	5MHz
Contact resistance:	85mΩ tip. 150mΩ max
Life:	200.000.000 operations

2.3.3 Channel board (P32)

The channel board realizes the connection between the instruments board and the nodes of the unit under test. Every P32 board has 32 direct hybrid channels, where every channels has 4 analog lines, where it is possible connect one of the 4 line of instruments board and one bi-directional digital line.
The channels have the following characteristics.

Analog channels:

Current:	1A max.
Voltage:	100V max.
Power:	10W max.
Max. Measurable frequency:	5MHz
Contact resistance:	85mΩ tip. 150mΩ max
Life:	200.000.000 operations

Driver digital channel:

High level:	-1V ÷ +12V		(-500mA <IOH<0)
Low level:	+2V ÷ -12V		(0<IOL<500mA)
Current clamp:	Programmable from 50 to 500mA by step of 2mA (guaranteed connection in 10µs)		
Operating frequency:	• Static (without DHF board) • Static – 10 MHz (with DHF board)		
Slew rate:	Programmable by 256 steps		

Sensor digital channel:

Strobe:	Programmable by step of 50ns		
Input capacitance:	50pF		
Input impedance:	> 30KΩ		
High voltage level:	-12.5V	÷ +12.5V	by steps of 100mV
Low voltage level:	- 12.5V	÷ +12.5V	by steps of 100mV

2.3.4 Board (SCAFP)

The channel board realizes the connection between the instruments board and the nodes of the unit under test. Every SCAFP board has 16 fix channels, where it is possible connect one of the 4 line of instruments board, consequently is possible connect every instruments of the LAM board. The 16 channels are available using address from 4097. If the SCAFP is installed on a flying probe system, it reserves 8 channels for test probe, consequently not available. Other instruments are besides available on SCAFP for use on system lines, described in the following with channels characteristics.

Channel characteristics:

Current:	1A max.
Voltage:	250V max.
Power:	10W max.
Max. Measurable frequency:	5MHz
Contact resistance:	85mΩ tip. 150mΩ max.
Life:	200.000.000 operations

Programmable voltage/current DC/AC generator, 4 quad:

DC current generator:	Progr. Current:	±1A	resolution 244µA
	Progr. Clamp voltage:	±5V	resolution 2.44mV
	Precision:	typ. 1% of full scale	
DC voltage generator:	Progr. Voltage:	±5V	resolution 2.44mV
	Progr. Clamp current:	±1A	resolution 244µA
	Precision:	typ. 1% of full scale	
AC voltage generator:	50 KHz (sinusoidal wave)		
	Progr. Voltage:	±5V	resolution 2.44mV
	Current:	300mA RMS	
	Progr. Offset:	±5V	resolution 2.44mV
	Precision:	typ. 1% of full scale	

Current meter:

Measure scale	Resolution	Typ. precision
0÷50µA	0.02µA	1% of the measured value ±0.06
0÷1mA	0.48µA	1% of the measured value ±1.44
0÷10mA	4.88µA	1% of the measured value ±14.64

- Capability to drive up to 16 SCAMP0 boards used for measures with fixed Capacitive Senses

- Timer for the ATGEN generator

  Progr. Current: 1mA/5mA
  Progr. Voltage: from 1V to 250V

- Output signals for the ATGEN generator

2.3.5 User power supply and User power supply control card (SERVIZI)

The SERVIZI control card, can support up to 6 power supplies that are used to power the board under test. The power supplies are mounted two modules. Each module is made of 3 single power supplies.

Module 1	
PW1	0V ÷ 6V 6A
PW2	0V ÷ 18V 1.2A
PW3	0V ÷ 18V 1.2A

Module 2	
PW1	0V ÷ 6V 6A
PW2	0V ÷ 30V 1.2A
PW3	0V ÷ 30V 1.2A

In the systems can be present all combination of Module 1 and Module 2 (Example. only Module 1, Module1 and Module 2, two Module 1, etc…). Customized power supplies, that are not included in the previous list, can be also mounted in the systems. The max current supported by the user power supply control card (servizi card) is 10A.

Section 3

3. TESTING METHODS

This section describes the techniques used to test the most common components, the sequence of a measurement and the guarding technique to isolate a component from the rest of the circuit.

3.1 "Standard" In-circuit measurements

We will now examine the different in-circuit techniques used to measure the most common components.
It is advisable to read this section in conjunction with the systems hardware architecture description, in particular the instrument card that is directly involved in the measurements.

3.1.1 Resistor measurement

The resistor measurement is carried out by injecting on one pin, an amount of current necessary to cause a determinate voltage drop (usually 0.3V). The other resistor pin is connected to the ATE ground.

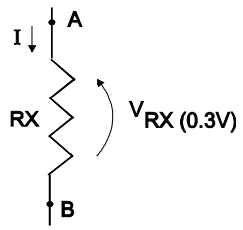


Figure 3-1 Resistor Measurements

In order to perform this test, the instruments/lines on the LAM card must be connected to the resistor in the following way:

- DR1 (current generator) connected on LINE 1 (LI1) to point A
- The ATE ground is connected on LINE 2 (LI2) to point B
- VOLTMIS (volt meter) is connected on LINE 4 (LI4) to point A as showed in figure below

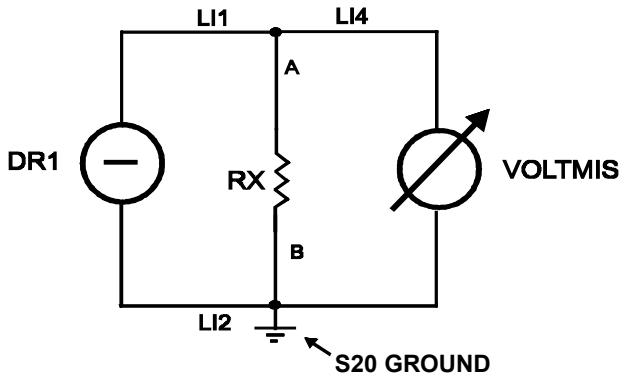
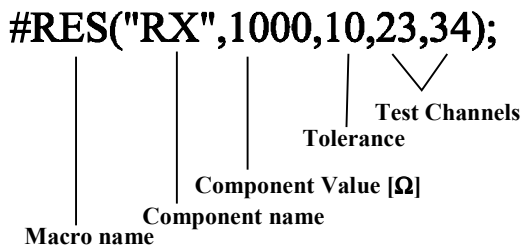


Figure 3-2 Resistor measurement connection

The measurement is executed by injecting a current on LINE 1 whose value was previously calculated with the formula:

$$I = V/RX \qquad I = 0.3/RX$$

and measuring the value of the voltage drop across the resistor with the VOLTMIS on LINE 4.
All operations to perform the measurement i.e. the current calculation, the instrument and line connections to the resistor are enclosed in a user macro, the #RES.
Let's suppose that the RX resistor previously mentioned, has got a value of 1KΩ, a tolerance of 5% and that we will use test channels 23 and 34 to measure it. The user macro will result like this:



This macro is written in the File.srg (source file) that contains all component tests for a board while the correspondence between the #RES macro and the software commands that drive the instruments/lines is written in the File.lib (library file). Here below is given the description of the source and library file. Refer to section 5 for any further details about software commands listed in the library file.

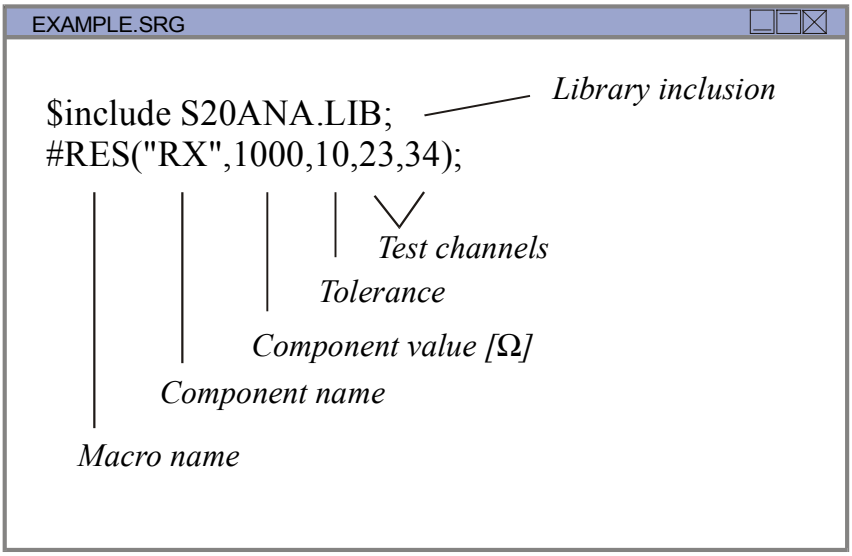


Figure 3-3 Source file description

S20ANA.LIB

```
#RES(COMM,VALO,PERC,P1,P2)
!A,1,2;
!2,0,0,2,0,P1=1,P2=2;
LOCAL VAL;
VAL=VALO+CABIMP;
NER=N,BEE=N,VIW=N,STP=N;
ALM=0X00,ELM=0XFF,PUM=N;
VIM=O;
DRM=I,DRV=1.1,DRI=(.5/VAL)*1000;
MES=(DRI*VAL)/1000;
TIM=VAL*.000025;
LOW=MES-(MES*PERC)/100;
HIG=MES+(MES*PERC)/100;
LN1=0;
LN2=0;
LN3=0;
LN4=0;MMD=V4;
LI1=P1,LI2=P2,LI3=0,LI4=P1;
LAB=COMM;
```

Macro header

Component structure

Variable declaration

Default values

DR1 programming

VOLTMIS programming

Connection of DR1 to LINE1

Connection of s20 ground to LINE2

Connection of GUArding gen. to LINE3

Connection of VOLTMIS to LINE4

Connection of channel 23 to LINE1,and LINE4 and of channel 34 to LINE2

Figure 3-4 Library file description

3.1.2 Capacitor measurement

The measurement of a capacitor is carried out by supplying the component with a constant current for a pre-defined time, to charge it up to 0,3V.
In order to perform this test, the capacitor must be connected to two lines:

- DR1 and VOLTMIS connected on LINE 1 to a capacitor pin
- ATE ground connected on LINE 2 to the other capacitor pin

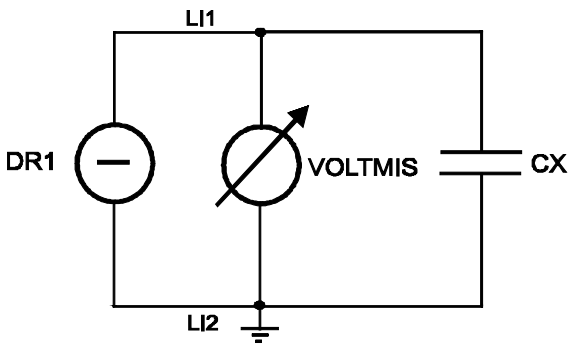
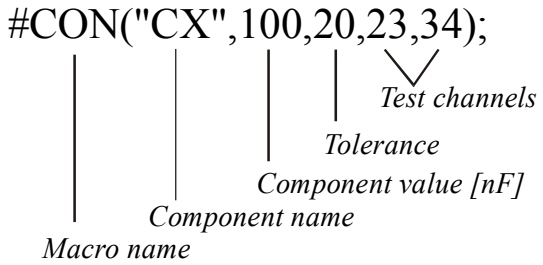


Figure 3-5 Capacitor measurements

The measurement is executed by injecting the calculated current on LINE1, and measuring the capacitor voltage charge at a determinate time T according to the formula:

I=V∗C/T I=0.3*C/T

There are two user macros that can be used to measure a capacitor: the #CON macro (for not-polarized capacitor) and the #CAP macro (for polarized capacitors).



In the case of low-value capacitors (below 40nF), the macro executes two measurements illustrated in the following figure. In the first one, the CX + CM capacitance is measured. During the second measurement the capacitor CX is disconnected from the instruments so that only the CM is measured. The subtraction between the two reads, gives the real capacitor value.

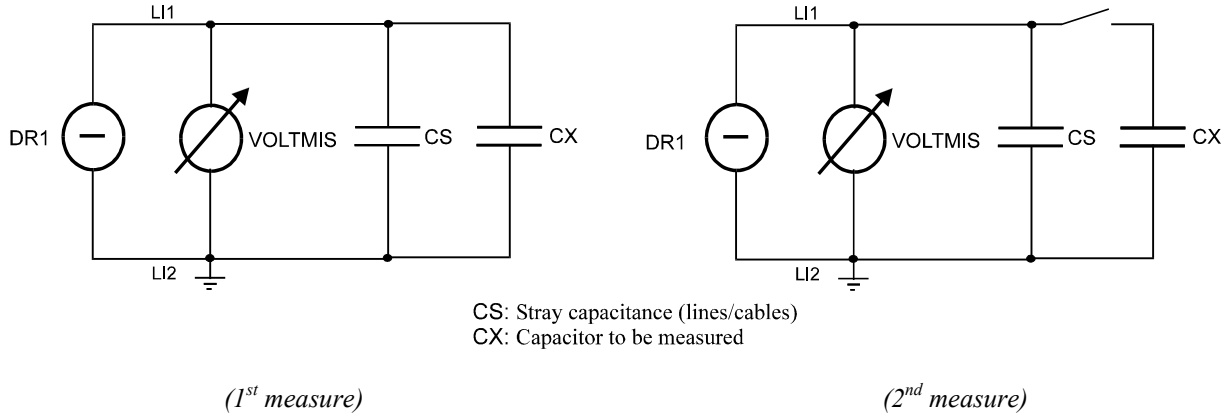
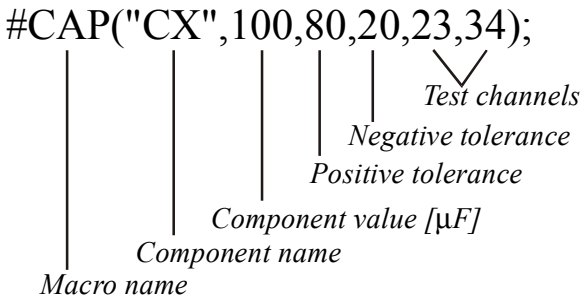


Figure 3-6 Low value capacitor measurement

The CAP macro is used for polarized capacitors and the syntax is given here below:



3.1.3 Inductor measurement

The measurement of an inductor is carried out by connecting in parallel to the component, a pre-charged capacitor and let it discharge on the inductor. The measure reads the voltage on the inductor at a determinate time T of the discharge transient according to the formula:

$$T = \left(\sqrt{\frac{4{,}7 \cdot L \cdot 2\pi + 4}{4000}} \right)$$

where:

L = inductor value to be measured

In order to perform this test, the instruments must be connected like this:

- DR1 and VOLTMIS connected on LINE 1 to one inductor pin
- ATE ground connected on LINE 2 to the other inductor pin
- The pre-charged capacitor connected in parallel to the inductor

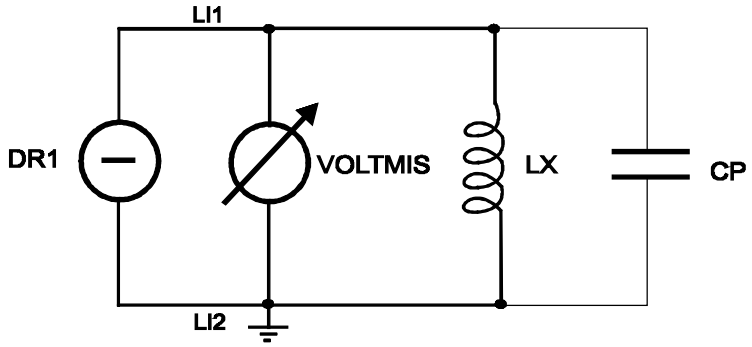


Figure 3-7 Inductor measurement

The user macro used to measure the inductor is the #IND.

#IND("LX",10,0.3,20,23,34);

Test channels

Tolerance

Test voltage

Component value [µH]

Component name

Macro name

3.1.4 Diode and Zener diode measurement

The measurement of diodes is carried out with two tests:

Direct polarization of the diode for which the following connections are performed:

- DR1 connected on LINE1 to the anode of the diode
- ATE GROUND connected on LINE 2 to the cathode of the diode
- VOLTMIS connected on LINE 4 to the anode of the diode

The measurement is executed by injecting a current on LINE 1 and measuring the value of the voltage drop on the terminals of the component with the voltmeter connected on LINE 4. This voltage is the voltage drop across the junction diode.

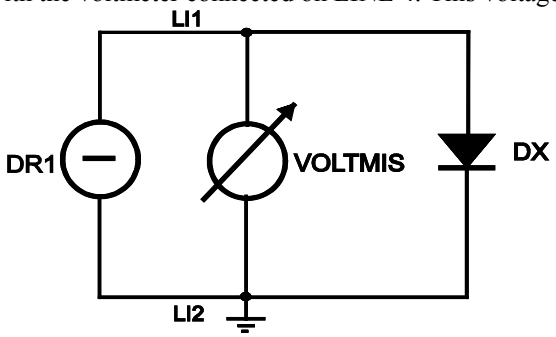


Figure 3-8 Direct polarization

Inverse polarization of the diode for which the following connections must be made:

- DR1 connected on LINE 1 to the cathode of the diode
- S20 GROUND connected on LINE 2 to the anode of the diode
- VOLTMIS connected on LINE 4 to the cathode of the diode

The measurement is executed by injecting a current on LINE 1 and measuring the value of the voltage drop on the terminals of the component with the voltmeter connected on LINE 4. This voltage must assume the clamp value of the current generator.

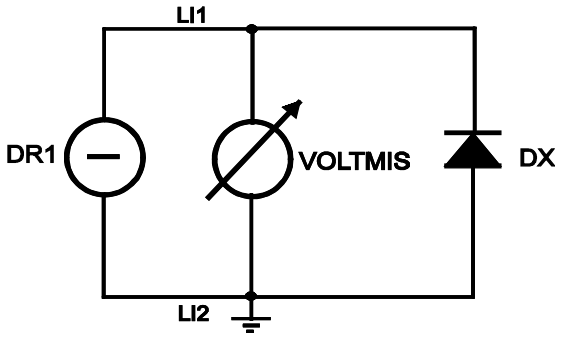


Figure 3-9 Inverse polarization

The macro used to test a diode is the:

#DIO("DX",37,23);

Channel on cathode

Channel on anode

Component name

Macro name

The measurement of zener diodes is carried out with two tests:

Direct polarization for which the following connections must be made:

- DR1 connected on LINE1 to the anode of the diode
- ATE GROUND connected on LINE 2 to the cathode of the diode
- VOLTMIS connected on LINE 4 to the anode of the diode

The measurement is executed by injecting a current on LINE 1 and measuring the value of the voltage drop on the terminals of the component with the voltmeter connected on LINE 4 This voltage is the voltage drop across the diode.

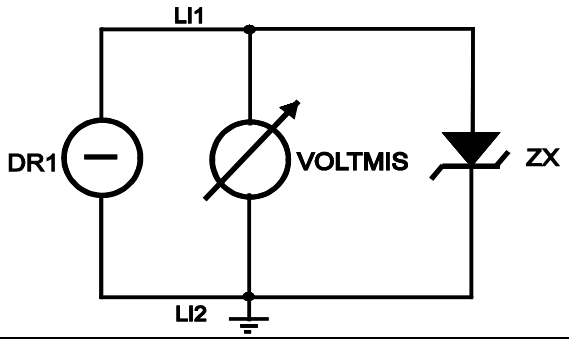


Figure 3-10 Direct polarization

Inverse polarization for which the following connections must be made:

- DR1 connected on LINE 1 to the cathode of the diode
- ATE GROUND connected on LINE 2 to the anode of the diode
- VOLTMIS connected on LINE 4 to the cathode of the diode

The measurement is executed by injecting a current on LINE 1 and measuring the value of the voltage drop on the terminals of the component with the voltmeter connected on LINE 4. This is the zener voltage.

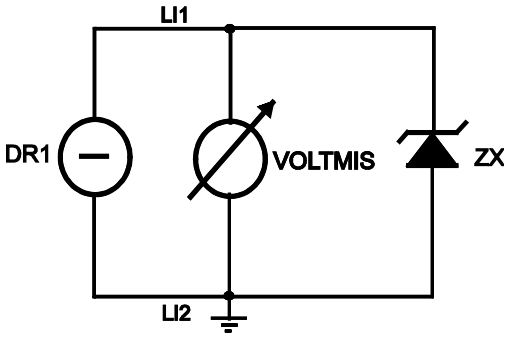
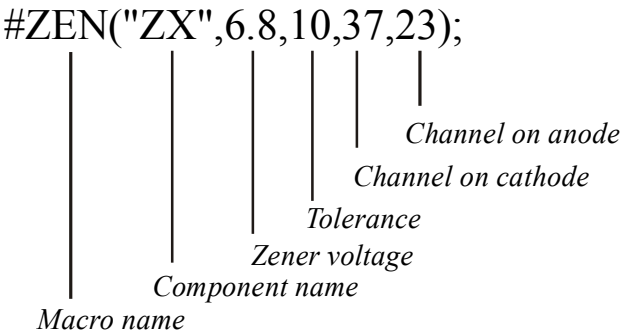


Figure 3-11 Inverse polarization

The macro used to test a zener diode is the:



3.1.5 Transistor measurement

The measurement of a transistor is performed by supplying the component with a constant current and verifying the junction voltage values.
Four tests are implemented in the measurement of a transistor:

Direct polarization of the BE junction, for which the following connections must be made:

- DR1 connected on LINE 1 to the transistor base
- ATE ground connected on LINE 2 to the transistor emitter
- VOLTMIS connected on LINE 4 to the transistor base

The measurement is executed by injecting a current on LINE 1 and measuring the value of the voltage drop on the terminals of the transistor. This voltage must be equal to the conducting value of this part of the transistor.

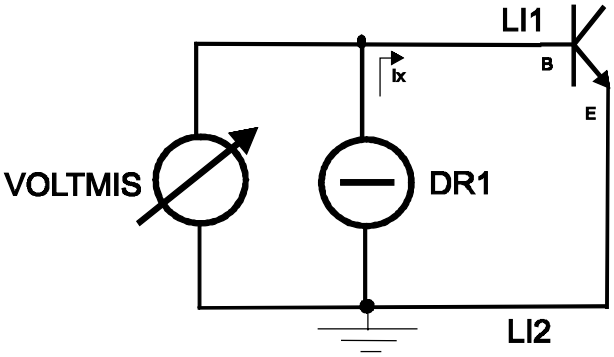


Figure 3-12 Direct polarization of the BE junction

Direct polarization of the BC junction, for which the following connections must be made:

- DR1 connected on LINE 1 to the transistor base
- ATE ground connected on LINE 2 to the transistor collector
- VOLTMIS connected on LINE 4 to the transistor base

The measurement is executed by injecting a current on LINE 1 and measuring the value of the voltage drop on the terminals of the transistor. This voltage must be equal to the conducting value of this part of the transistor.

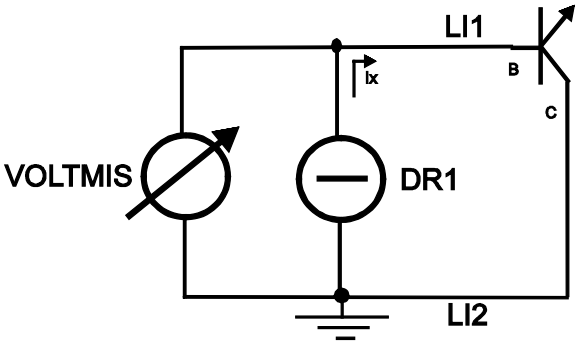


Figure 3-13 Direct polarization of the BC junction

Gain measurement. To measure the transistor gain, we must proceed as follows:

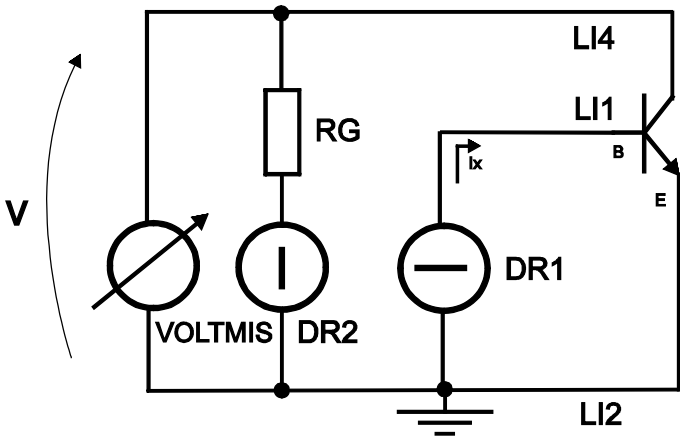


Figure 3-14 Measurement of the transistor gain

Example: if the gain of the transistor is β = 100. The following values are used to verify the gain:

DR1 = 10/β, DR1=0.1mA

DR2 = 1.5V

RG = 100Ω

From the circuit in figure 3-14, the following equation must be verified:

$$V = VDR2-(DR1*\beta)*RG$$

From which

$$\beta=V-VDR2/(DR1*RG)$$

V is set to 0,3V so we have

$$100=0.5-1.5/(0.0001*100)$$

Transistor in the OFF state. This test uses the same connections as the gain test but DR1 in this case is programmed to 0V so that the transistor is in its OFF state.

There are two user macros that can be used to test a transistor: the #TRN and the #TRP according to the transistor type. The two macros have got the same syntax. Here below is illustrated the #TRP macro:

```
#TRN("TX",100,37,23,34);
```

Channel on collector
Channel on base
Channel on emitter
Transistor gain
Component name
Macro name

3.1.6 Mosfet measurement

The MOSFET test is performed with two measurements:

The first one puts the MOSFET in **OFF** state and connects the instruments like this:

- DR1 is connected to LINE 1 to the Mosfet DRAIN
- ATE GROUND is connected to LINE 2 to the Mosfet SOURCE
- DR2 is connected to LINE 3 to the Mosfet GATE
- VOLTMIS is connected on LINE 4 to the Mosfet DRAIN

The measure applies 0V on the Mosfet Gate and the VOLTMIS measures between Drain and Source. The voltage read is the DR1 clamp because the Mosfet is in the OFF state.

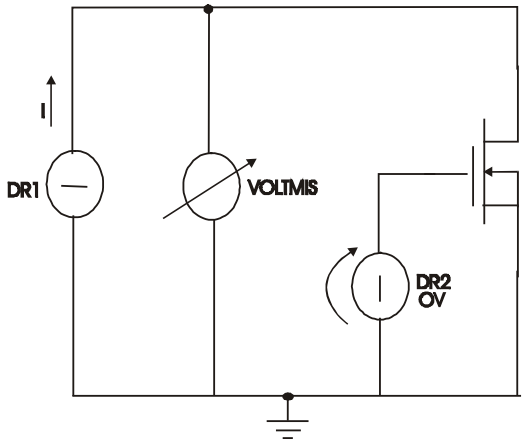


Figure 3-15 MOSFET in OFF state

The second one puts the MOSFET in **ON** state and connects the instruments like this:

- DR1 is connected to LINE 1 to the Mosfet DRAIN
- ATE GROUND is connected to LINE 2 to the Mosfet SOURCE
- DR2 is connected to LINE 3 to the Mosfet GATE
- VOLTMIS is connected on LINE 4 to the Mosfet DRAIN

The measure applies 5V on the Mosfet Gate and the VOLTMIS measures the voltage drop between Drain and Source.

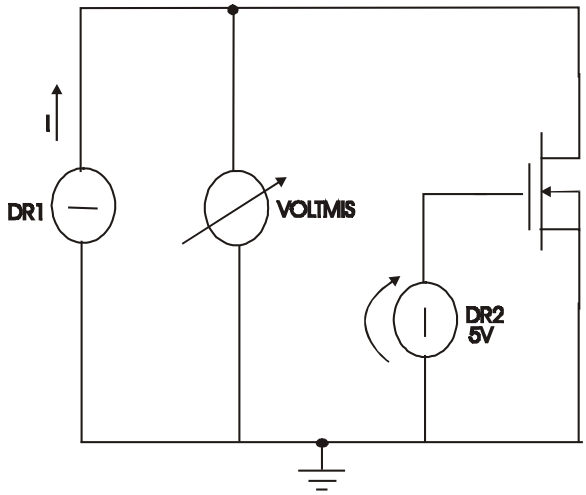


Figure 3-16 MOSFET in ON state

3.1.7 Relay measurement

The relay measurement performs two tests: the relay closed and the relay open.

Relay closed. To close the relay, the following connections are made:

- DR2 connected on LINE 3 to one terminal of the coil
- ATE ground connected on LINE 2 to the other terminal of the coil and to one relay contact
- DR1 connected on LINE 1 to the other relay contact
- VOLTMIS connected on LINE 4 to the other relay contact

The measurement is performed by exciting the coil and measuring the voltage drop at the terminals of the contacts. This voltage must be equal to V=Rc*I where Rc represents the relay contact resistance.

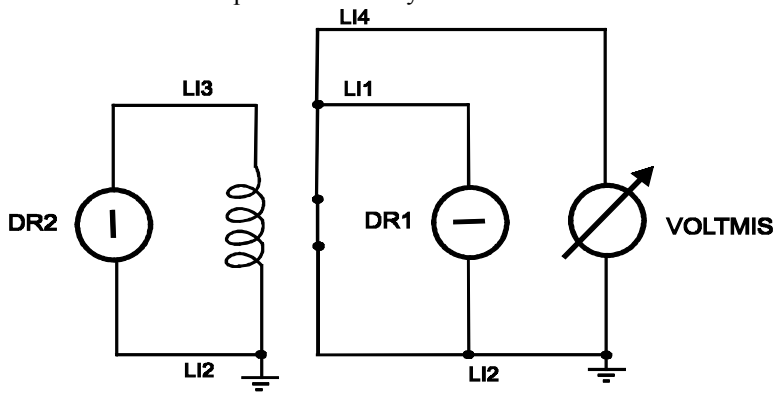


Figure 3-17 Relay closed

Relay open. The same connections previously described are used to open the relay. In this case, the DR2 generator is programmed to 0V so that the coil is not excited. The voltage measure is equal to the value of the generator clamp.

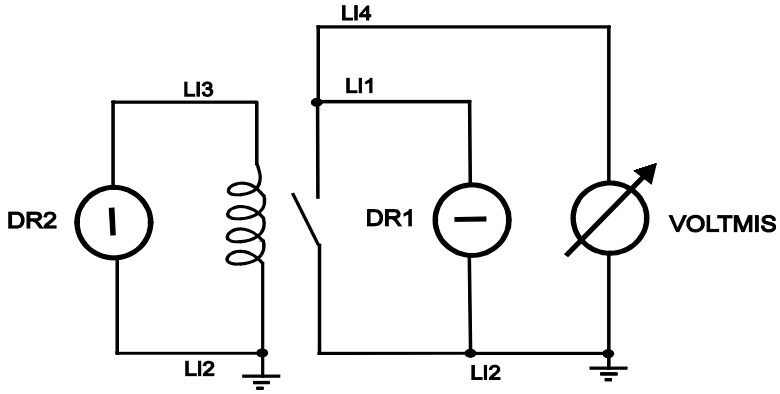
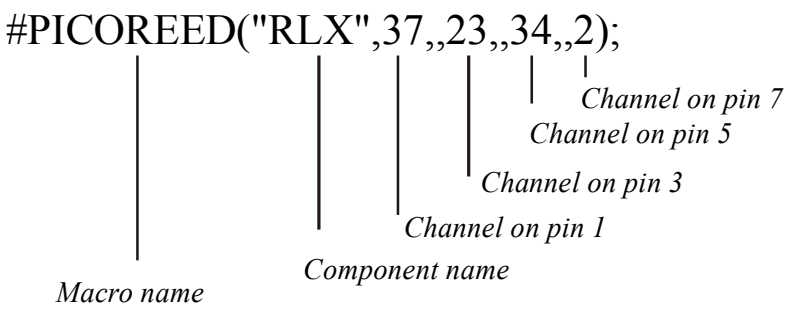


Figure 3-18 Relay open

The user macro used to test the Pico reed relay is:



3.1.8 Transformer measurement

The measurement of a transformer is carried out using a voltage generator programmed in a pulse mode, and measuring the induced voltage on the secondary winding.

To perform this test, the following connections must be made:

- primary: DR1 is connected on LINE 1 to one primary pin
  ATE GROUND connected on LINE 2 to the other primary pin
- secondary: VOLTMIS connected on LINE 4 to one secondary pin
  ATE GROUND connected on LINE 2 to the other secondary pin

The measurement is performed by supplying a determinate pulsed voltage value on LINE 1 and measuring the voltage that is generated on the secondary winding. This voltage will depend on the input voltage and the winding ratio as indicated in the equation:

$$\frac{V2}{V1} = \frac{n2}{n1}$$

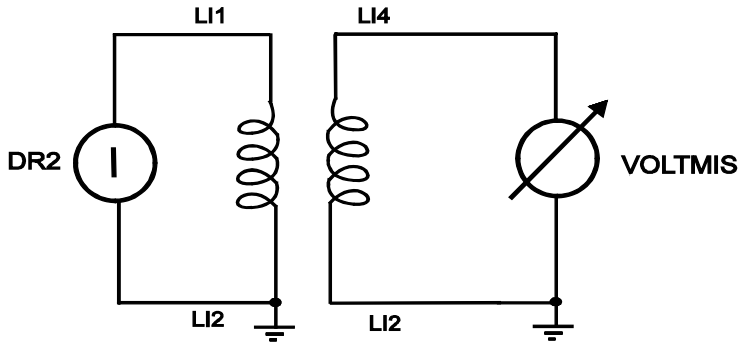


Figure 3-19 Measurement of a transformer

The user macro for the transformer is:

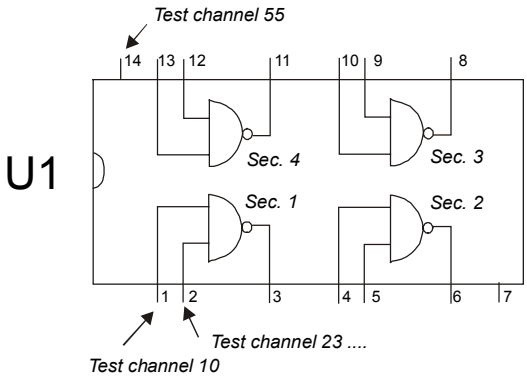
#TRASF("TX",20,5,15,10,23,34,45,67);

Test channels on secondary
Test channels on primary
Tolerance
Time [ms]
Voltage on secondary
Voltage on primary
Component name
Macro name

3.1.9 Digital component measurement

For digital components, the test must verify the true table. The board under test is powered on and the four probes supply the test stimuli on the input pins and verify the output responses according to the true table of the component. The low and high input levels are given by DR1 that perform the low logic level and DR2 that performs the high logic level. The low and high output levels are measured by the VOLTMIS on the LAM card. Let's now write the test for

component U1 (7400) illustrated in the following figure.



The macro in the File.srg will result like this:

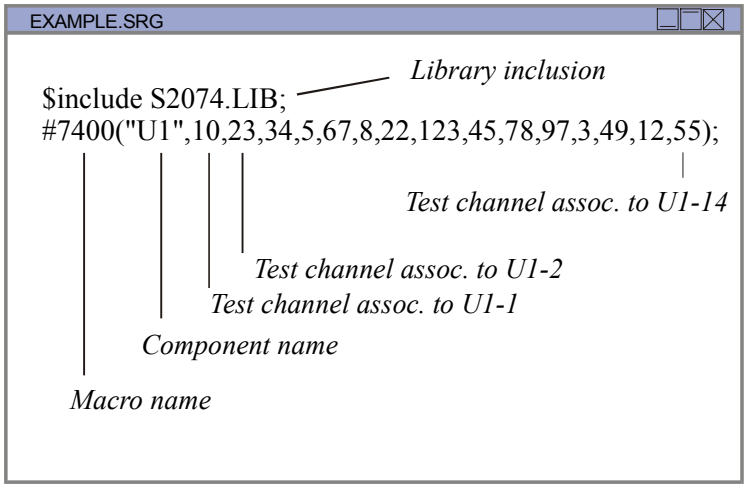


Figure 3-20 Source file description

and the #7400 macro in the library file will be:

S20ANA.LIB

```
#7400(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14);

!D,4,14;                 D= digital comp.  4= nr. of internal sections, 14= nr.of pins of the component

!3,2,1,0,0,P1,P2,P3;     Section 1: 3=nr. of pin of the section, 2=nr. of input pin, 1=nr. of output pin
                                    0=nr. of bidir. pin, 0=nr. of enable pin, P1,P2,P3=list of pin of section 1

!3,2,1,0,0,P4,P5,P6;     Section 2: 3=nr. of pin of the section, 2=nr. of input pin, 1=nr. of output pin
                                    0=nr. of bidir. pin, 0=nr. of enable pin, P4,P5,P6=list of pin of section 2

!3,2,1,0,0,P9,P10,P8;    Section 3: .....

!3,2,1,0,0,P12,P13,P11;  Section 4: .....

@C(4.7,0,2.4,.8,.1,1,P,C);  4.7= voltage for high level inputs, 0= voltage for low level inputs,
                            2.4=voltage for high level output, 0.8=voltage for low level output
                            .1=test time [ms], 1 nr. of clock pulse from the CKGEN,
                            P= clock edge: rising,C=true table type:for combinatorial components

@L(HH0...........);      H= high input level on pin 1, H= high input level on pin 2,
                         0= low output level on pin 3

@L(Lh0...........);
@L(hL0...........);
@L(...HH0........);      H= high input level on pin 4, H= high input level on pin 5,
@L(...Lh0........);      0= low output level on pin 6
@L(...hL0........);
@L(......0HH.....);      As a general rule, in the in-circuit programs:
@L(.....1Lh......);      H or  h = Driver High
@L(......1hL.....);      L or  l = Driver Low
@L(..........0HH.);      1       = Sensor High
@L(..........1Lh.);      0       = Sensor Low
@L(..........1hL.);
```

Figure 3-21 Library file description

3.2 The analog guarding technique

The guarding technique permits to isolate a single component from the rest of the circuit while testing it. The principle used is that of balancing the voltages, which surround the component under test so that there is no current flow through the components on the UUT connected to it.
This operation is made possible using a guarding generator (GUA), composed of a voltage follower that follows the voltage on DR1 which will be connected to all components connected to the measured node. In this way, the injected current flows entirely through the component under test and there are no current leaks through the adjacent nodes (guarding points).
The following is an example of the application of the guarding technique.

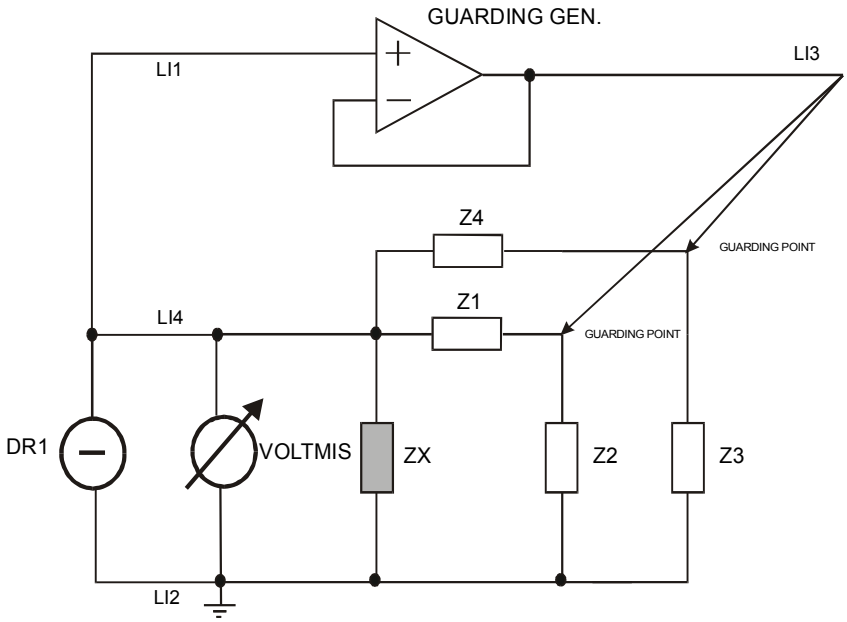


Figure 3-22 Application of the guarding technique

In the circuit above, Z1 and Z4 are subjected to a null difference in potential, no current flows through them but it all flows through the component under test (ZX).

3.3 The digital guarding technique

It is a common condition on digital boards that many components are connected on the same bus. While performing the digital testing the board must be powered so that the components try to talks together. With the digital guarding technique the test system can test each component separately by disabling the others on the bus. This concept is better explained by the example that follows:

Let's suppose to have this condition:

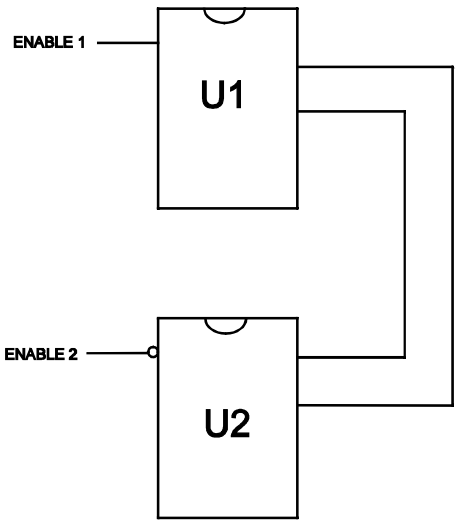


Figure 3-21 Two components talking on the same bus

The digital guarding disables the component U2 by forcing a high level on ENABLE 2 while testing U1 and it forces a low level on ENABLE 1 while testing U2. In this way the test of the two components can be done separately.

3.4 Sequences of measurements

The tests of board components are developed on the basis of a specific timing determined by the system.

This timing includes the following steps:

1. Closure of the scanning circuits of the channels relative to the device to be measured
2. Activation of the stimulus generators
3. Measurement strobe
4. End of measurement and indication to the control unit
5. Disable of the generators with consequent discharge of the capacitors and inductors present on the board
6. Opening of the scanning circuits

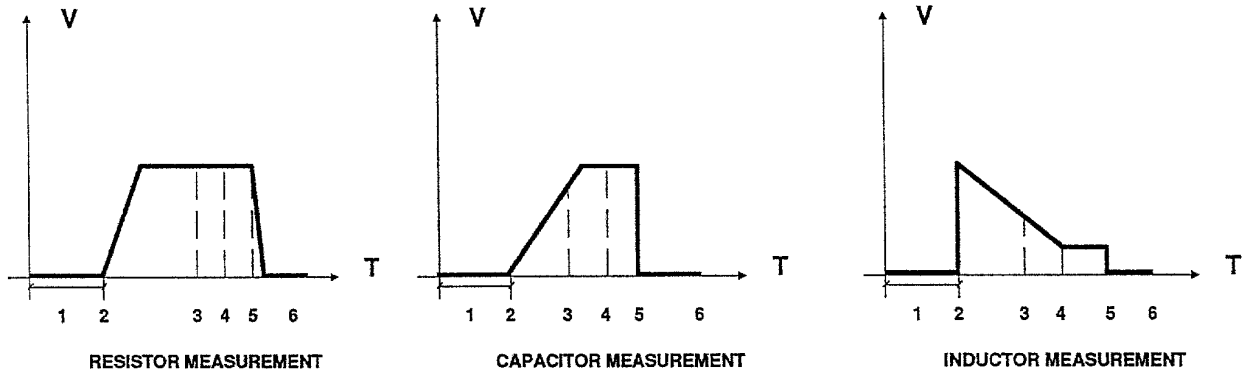


Figure 3-24 Measurement sequences

The timing relative to the above-mentioned steps is as follows:

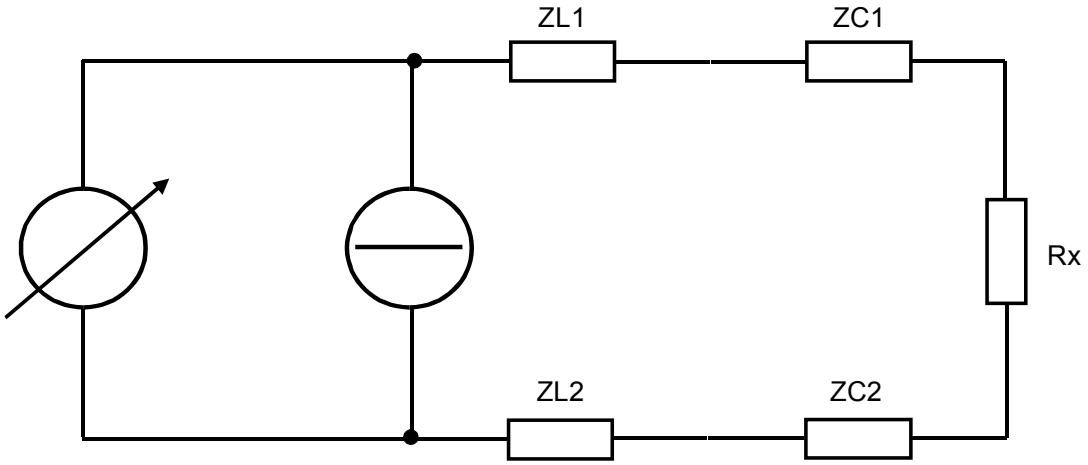
1	approximately 1000µs
2	programmed according to the TIM command that defines the test time
4	approximately 20µs
5	programmed according to the WAI command that defines the wait time at the end of the test
6	approximately 1000µs

3.5 Types of measurement

According to how the 4 lines (LINE 1 - LINE 4) on the LAM card are connected, different types of measurements can be executed.
The type of measurement to choose depends on the value of the component, its position in the circuit and its test requirements.

3.5.1 2 lines, 2 points measurement

The generator and the measurement magnitude are in parallel and are connected to the component via 2 lines and 2 points.



Rx = Resistor under test

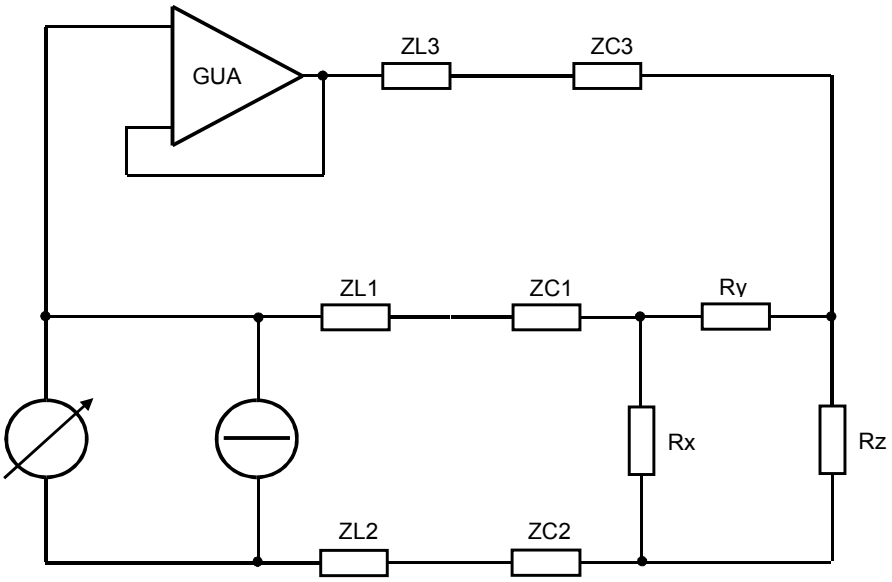
ZL = Line impedance (reed)

Zc = Contact impedance (contacts, nails, connections)

Value measured = ZL 1 + ZC1 + Rx + ZC2 + ZL2

Figure 3-25 2-lines, 2 points measurement

3.5.2 2-lines, 2 points measurement with guarding



Rx = Resistor under test

Ry = Resistor to be guarded

Rz = Resistor to be guarded

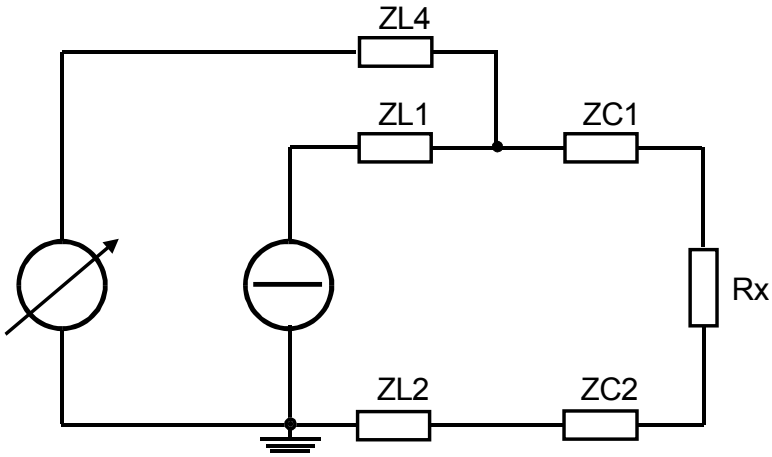
ZL = Line impedance (reed)

Zc = Contact impedance (contacts, nails, cabbs)

Figure 3-26 2-lines, 2 points measurement with guarding

3.5.3 3-lines measurement

Two different lines are used for the stimulus (forcing) and measurement (sensing). Another line is used for the ground reference, common to both the generator and measurement magnitude.

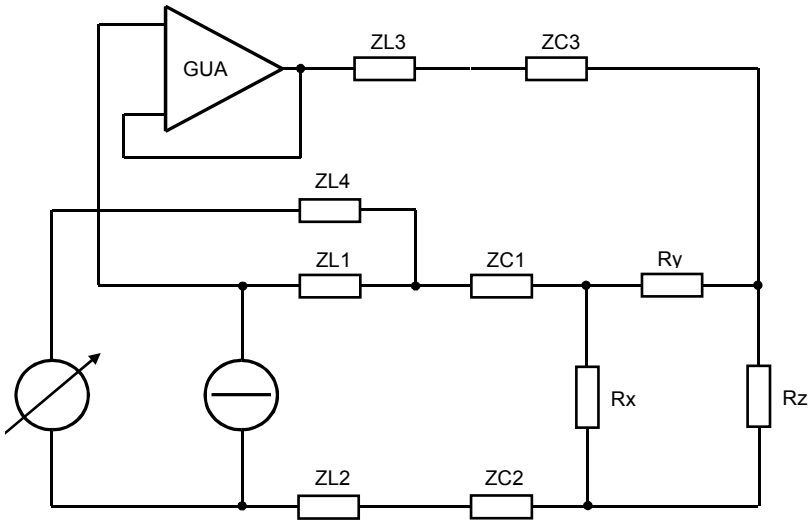


Rx = Resistor under test
ZL = Line impedance (reed)
Zc = Contact impedance (contacts, nails, cabbs)

Value measured = ZC1 + Rx + ZC2 + ZL2

Figure 3-27 3-lines, 2 points measurement

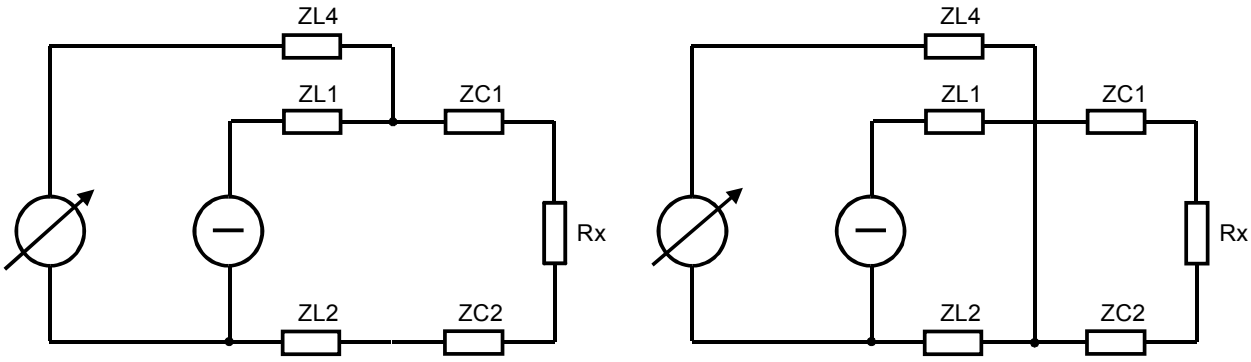
3.5.4 3-lines, 2 points measurement with guarding



Rx = Resistor under test
Ry = Resistor to be guarded
Rz = Resistor to be guarded
ZL = Line impedance (reed)
Zc = Contact impedance (contacts, nails, cabbs)

Figure 3-28 3-lines, 2 points measurement with guarding

3.5.5 Double read measurement



Rx = Resistor under test

ZL = Line impedance (reed)

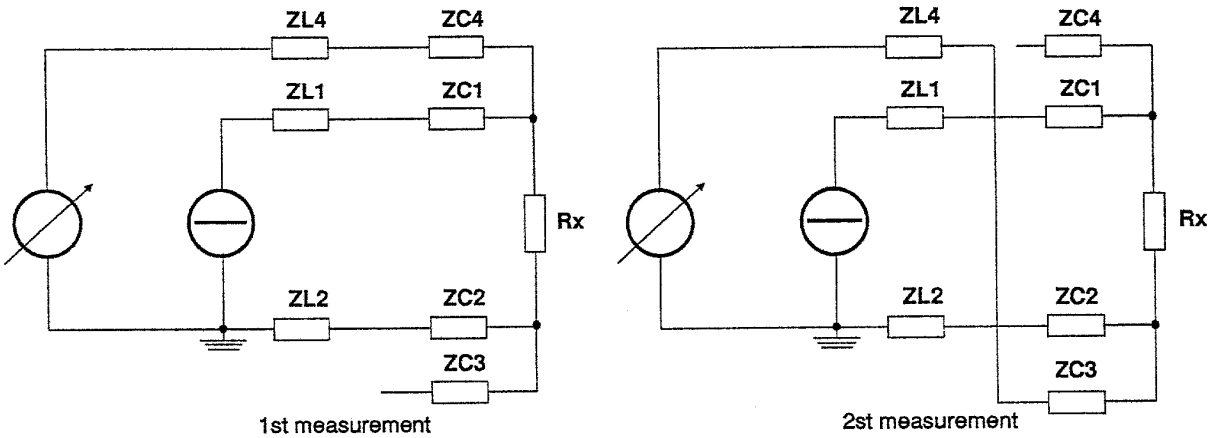
Zc = Contact impedance (contacts, nails, cables)

$$\text{Value measured} = \underbrace{ZC1 + Rx + ZC2 + ZL2}_{\text{1st measurement}} - \underbrace{ZL2}_{\text{2st measurement}} = Rx + ZC1 + ZC2$$

Figure 3-29 2-points, double read measurement

With this measurement technique, the imprecision is due only to the ZC impedance's. It can be used when a high level of precision is required. Guarding can be used on line 3 to isolate the component from the rest of the circuit.

3.5.6 4 points, double read measurement



Rx = Resistor under test

ZL = Line impedance (reed)

Zc =Contact impedance (contacts, nails, cabbs)

$$\text{Value measured} = \underbrace{Rx + ZC2 + ZL2}_{\text{1st measurement}} - \underbrace{(ZL2 + ZC2)}_{\text{2st measurement}} = Rx$$

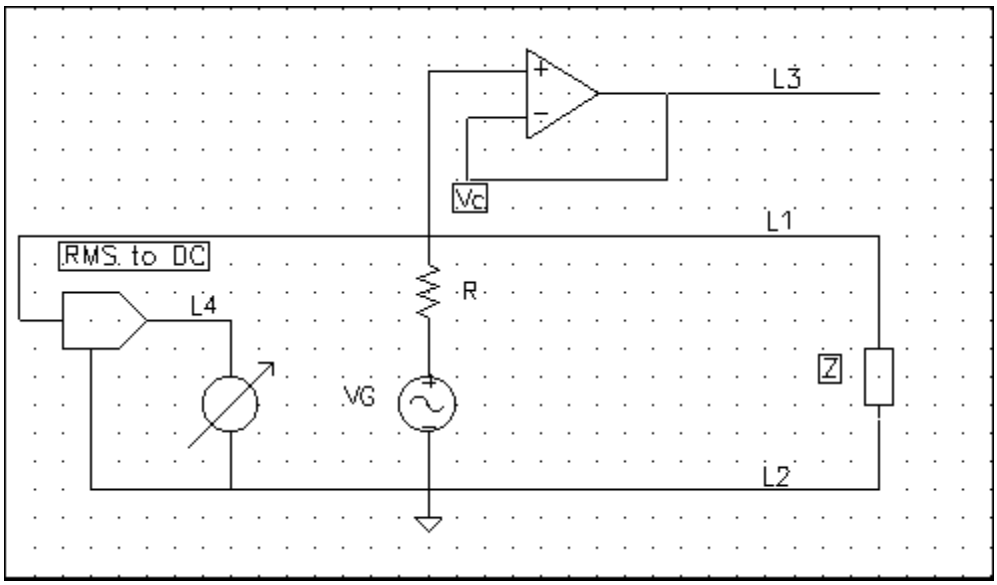
Figure 3-30 4-points, double read measurement

This technique allows very high levels of precision, because it eliminates all of the undesired impedance's from the measurement.
Guarding can be used on line 3 to isolate the component from the rest of the circuit.

3.6 In-circuit ac measurements

We will now examine the in-circuit AC techniques used to measure capacitors and inductors.
The resources used to perform this kind of measurements are the USER BOARD and the USER GEN.

The circuit used to perform ac measurements is the following:



The Vc voltage on the measurement point is regulated by the formula:

$$V_C = HV_G$$

with

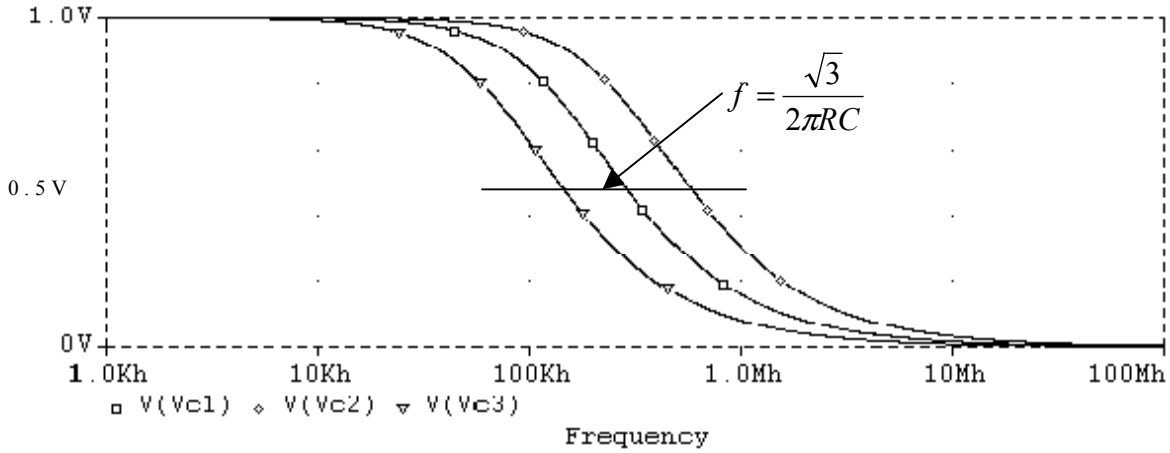
$$H = \frac{Z}{Z + R} \qquad (1)$$

3.6.1 Capacitor AC measurement

With a capacitor, the magnitude of the transfer function (1) becomes:

$$|H_C| = \frac{1}{\sqrt{1 + (2\pi f)^2 R^2 C^2}} \qquad (2)$$

the graph for this equation, with R=1KOhm, C1=0.5nF C2=1nF, C3=2nF, is shown below:



We have a pole in this transfer function for:

$$f = \frac{1}{2\pi RC}$$

From the graph, we deduce that is isn't worthwhile to make a measurement at a very low or at a very high frequency because the magnitude variations as a function of the capacitance are very small. If we make the measurement in the middle point (where the magnitude it half its value), we have:

$$f = \frac{\sqrt{3}}{2\pi RC} \tag{3}$$

$$C = \frac{\sqrt{V_G^2 - V_C^2}}{V_C R(2\pi f)} \tag{4}$$

The user macro used to test the capacitor using the AC technique is:

```
#CON_AC("C1",100,10,34,2,24);
```

Canale collegato al punto di guarding (0 se il guarding non è presente
Canale collegato al pin 2
Canale collegato al pin 1
Tolleranza
Valore componente (espresso in nF)
Nome componente
Nome macro

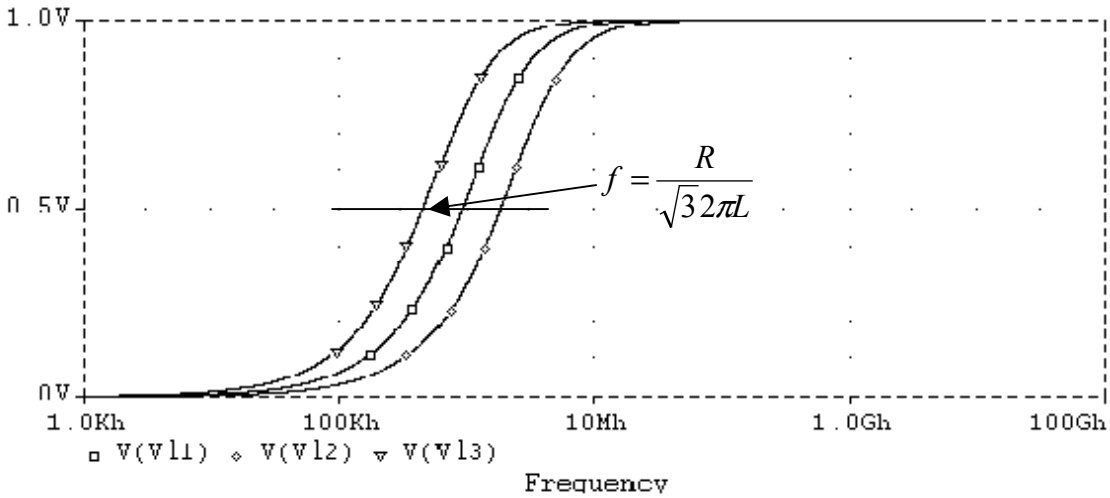
NOTE: The UGEN.LIB library must be included in the File.srg to use this macro.

3.6.2 Inductor AC measurement

With an inductor, the magnitude of the transfer function (1) becomes:

$$|H_L| = \frac{(2\pi f)L}{\sqrt{R^2 + (2\pi f)^2 L^2}} \tag{5}$$

the graph for this equation, with R=100Ohm, L1=5uH L2=10uH, L3=20uH, is shown below:



this transfer function has got a zero in the origin and a pole for $f = \frac{R}{2\pi L}$

From the graph, we deduce that is isn't worthwhile to make a measurement at a very low or at a very high frequency because the magnitude variations as a function of the inductance are very small. If we make the measurement in the middle point (when we have half the magnitude), we have:

$$f = \frac{R}{\sqrt{3} \cdot 2\pi L} \tag{6}$$

$$L = \frac{V_C R}{\sqrt{V_G^{\ 2} - V_C^{\ 2}}(2\pi f)} \tag{7}$$

The user macro used to test the inductor using the AC technique is:

#IND_AC("L1",100,10,34,2,24);

- *Macro name*
- *Component name*
- *Component value (expressed in μH)*
- *Tolerance*
- *Channel on pin 1*
- *Channel on pin 2*
- *Channel on guarding point (0 if no guarding is present)*

NOTE: The UGEN.LIB library must be included in the File.srg to use this macro.

3.6.3 Capacitor and inductor in parallel

If the impedance Z is a capacitor in parallel to an inductance, the transfer function magnitude is:

$$|H_{LC}| = \frac{(2\pi f)L}{\sqrt{R^2 + R^2L^2C^2(2\pi f)^4 + (L^2 - 2R^2LC)(2\pi f)^2}} \tag{8}$$

if we put:

$$\omega_n = \frac{1}{\sqrt{LC}},\ f_n = \frac{1}{2\pi}\frac{1}{\sqrt{LC}},\ \delta = \frac{1}{2R}\sqrt{\frac{L}{C}} \tag{9}$$

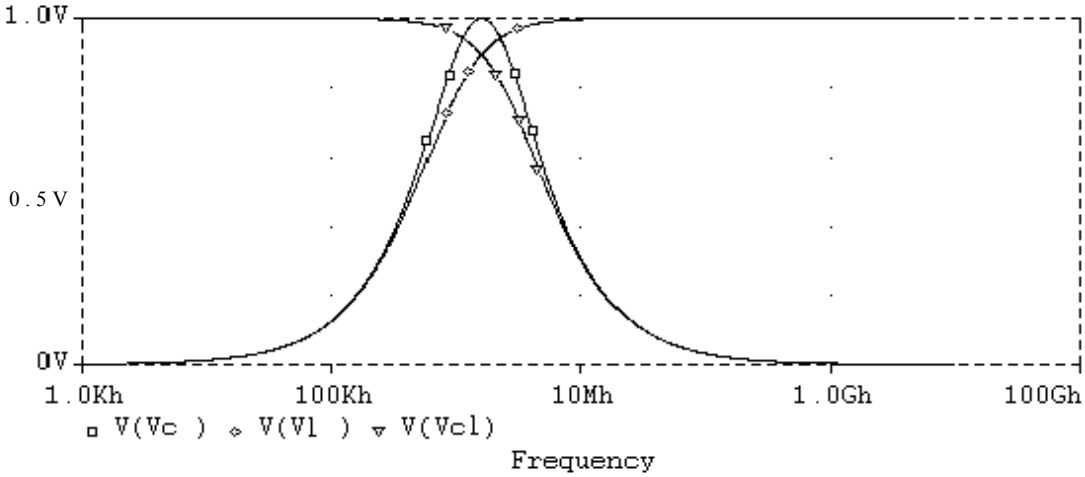
the equation becomes:

$$\left|H_{LC}\right| = \frac{2\delta \dfrac{f}{f_n}}{\sqrt{1 + \dfrac{f^4}{f_n{}^4} + \left(4\delta^2 - 2\right)\dfrac{f^2}{f_n{}^2}}} \tag{10}$$

This transfer function has got a zero in the origin and, if δ≥1, it has got two real poles for:

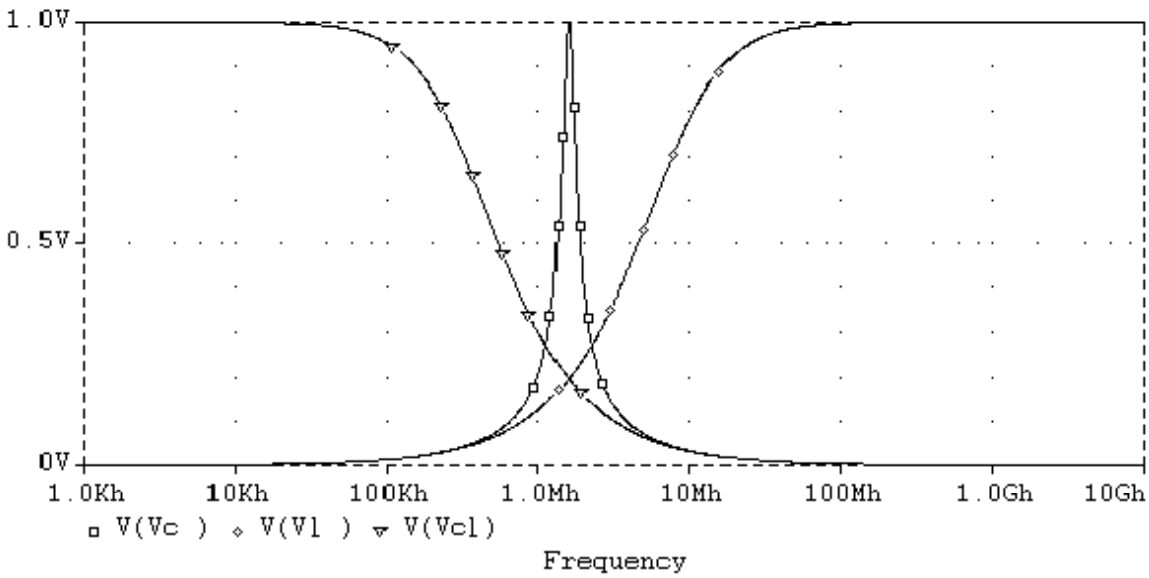
$$f_{1,2} = -\delta \mp \sqrt{\delta^2 - 1} \tag{11}$$

The graph for this equation with, R=50Ohm, L1=10uH, C=1nF δ=1, is shown below:



In the graph above, the three curves represent the transfer function related to the single capacity, to the single inductance and to the parallel of the two. It is worth noting that, if δ≥1, the first piece of the green curve is confused with the red curve. On the other hand, the last piece of the green curve is confused with the blue curve.

In the case of complex conjugate poles we have:
R=500Ohm, L1=10uH, C=1nF δ=0.1



In this case, the resonance peak is as much emphasized as δ approaches to zero and it is not possible to confuse the two curves any more.
This means that, if it is possible, it is advisable to put δ ≥ 1 so that we measure the inductor and the capacitor, individually.

In both cases, if we make the measurement for $|H| = \frac{1}{2}$, we have:

$$f_L = \left(-\sqrt{3}\delta + \sqrt{3\delta^2 + 1}\right) f_n \qquad f_C = \left(\sqrt{3}\delta + \sqrt{3\delta^2 + 1}\right) f_n$$

$$C = \frac{V_C \pm (2\pi f_C) L \sqrt{V_G{}^2 - V_C{}^2}}{V_C L R (2\pi f_C)^2} \qquad L = \frac{V_C R\left((2\pi f_L) C \pm \sqrt{V_G{}^2 - V_C{}^2}\right)}{\left(V_C{}^2 R^2 C^2 + V_C{}^2 - V_G{}^2\right)(2\pi f_L)} \tag{12}$$

The user macro used to test the inductor and capacitor using the AC technique is:

```
#LC_AC0("L1//C1",0.8,0,10,1,100,10,23,25,4);
```

Channel on guarding point (0 if no guarding is present
Channel on pin 2
Channel on pin 1
Tolerance
Capacitor value (nF)
Inductor value (μH)
Number of test recycle
AC generator resistor (Ω)
AC generator voltage(V)
Component names
Macro name

NOTE: The UGEN.LIB library must be included in the File.srg to use this macro

Section 4

4. IN-CIRCUIT PROGRAMMING LANGUAGE

The in-circuit programming language programs the system hardware to supply the stimuli to the UUT and to perform the needed measurements.
The in-circuit language is composed of:

- Instructions
- Commands
- Macros

The **instructions** are used in the library environment (with the exception of the $include / $section instructions) to process data and to define the flow of execution of a test program.

Commands are used to define all the hardware and software parameters of the in-circuit program.
Commands act directly on the hardware of the system programming it to give the stimuli to the board and do the necessary measurement.

Macros are a group of instructions and commands that execute a specific function, for example the test of a component. The instructions that constitute the macro are included in a library. To execute a macro from the file.srg it is only necessary to call it and give the macro arguments but not re-write down the entire macro code in the program.

In the following paragraphs the instructions the macros the commands, are described in the right- hand corner of the page it is indicated the environment in which the command can be used: File.srg or File.lib.

4.1 Instructions

The instructions are divided in:

- condition instructions: if-else-endif
switch-case-endcase
- iteration instructions: for-endfor
- jump instruction: goto-label
- declaration instruction: $include
$section
GLOBAL
LOCAL

if-else-endif

File.SRG [ ] File.LIB [X]

FUNCTION

Allows the verification of a condition. If the condition is true, the instructions which follow are executed, otherwise the program continues with those following the ELSE or ENDIF.

SYNTAX

if (condition)
...
...
else (condition)
...
...
endif

NOTE

The part relative to the ELSE branch is optional and can also be omitted.
The operators allowed to specify the condition are:

=	equal to
>=	greater than or equal to
<=	lesser than or equal to
>	greater than
<	lesser than
<>	different from

EXAMPLE

```
if (VAL>1000)
TIM=10;
else
TIM=1;
endif
```

switch-case-endswitch

File.SRG [ ] File.LIB [X]

FUNCTION

Allows multiple decisions to be made, verifying whether the check expression corresponds to one of the constant values defined in the various branches of the instruction.

SYNTAX

switch (variable)
case (value 1)
...
...
case (value 2)
...
...
case (value 3)
...
...
endswitch

NOTE

The switch instruction evaluates the whole expression in parentheses (variable) and compares its value with the value defined in the “case” branches. Each “case” must be defined with a numerical variable. If a “case” does not correspond to the value of the expression, the execution continues with the verification of the next case. If a “case” corresponds to the value of the expression, all the instructions that make up the body are executed and after that, the execution is terminated with the ENDSWITCH instruction. If no case corresponds to the value of the expression, the ENDSWITCH instruction is executed.

EXAMPLE

```
GLOBAL VAR,I;
VAR=2;
for (I=0;I<=10;I=I+1;)
    VAR=VAR*I;
switch (VAR)
    case (0)
            DRV=10;
            TIM=100;
            LI1=3;
    case (5)
            DRV=I;
            TIM=VAR;
            LI1=5;
    case (7)
            DRV=I+7;
            TIM=VAR+7;
            LI1=5-7;
endswitch
endfor
```

for-endfor

File.SRG [ ] File.LIB [X]

FUNCTION

Indicates that an operation must be executed repeatedly, until the exit condition occurs. The initial assignment and increment instructions of the variable under test are used to attain the condition.

SYNTAX

```
for (assignment; condition; increment)
...
...
...
endfor
```

assignment: enables the initialization of the cycle variable

condition: condition verified in each cycle. If the condition is true, the increment operation is executed and the system remains in the cycle. If the condition is false the system exits from the cycle.

increment: operation which increments the cycle variable

NOTE

The operators allowed to specify the condition are:

=	equal to
>=	greater than or equal to
<=	lesser than or equal to
>	greater than
<	lesser than
<>	different from

EXAMPLE

```
GLOBAL I;
for (I=0; I<=10; I=I+2;)
    if (I>5)
            DRV=15;
    else
            DRV=30;
    endif
endfor
```

goto-label

File.SRG [ ] File.LIB [X]

FUNCTION

Indicates that the execution must continue in another point in the program where the label is found; this label is identified by the label command.

SYNTAX

```
goto ("label_name")
...
...
...
label=("label_name")
```

NOTE

The label instruction can also precede the goto instruction, in this case the program will jump backwards.

EXAMPLE

```
GLOBAL I;
DRV=5;
label ("jump1")
if(DRV>15)
  goto("jump2")
endif
for (I=0;I<+10;I=I+2;)
  if(I>5)
    DRV=15;
  else
    DRV=30;
  goto ("jump1")
  endif
endfor
label ("jump2")
```

label

File.SRG [ ] File.LIB [X]

FUNCTION

It defines a label in the program that can be the destination of a goto-label instruction, changing the program sequence.

SYNTAX

```
label=("label_name1")
...
...
goto ("label_name2")
...
...
goto ("label_name1")
...
label=("label_name2")
...
```

NOTE

The label instruction can be also placed before the GOTO instruction. In this way it is possible to jump backwards.

EXAMPLE

```
GLOBAL I;
DRV=5;
label ("jump1")
if(DRV>15)
      goto("jump2")
endif
for (I=0;I<+10;I=I+2;)
      if(I>5)
        DRV=15;
      Else
        DRV=30;
      goto ("jump1")
      endif
endfor
label ("jump2")
```

$include

File.SRG [X] File.LIB [ ]

FUNCTION

Allows the declaration of the file containing the library in order to use the macro it contains.

SYNTAX

$include library name

library name: file to be included in the source program.

NOTE

This section must precede the call to the macro contained in the file to be included, and is usually at the top of the source program. A maximum of 10 libraries can be included.

EXAMPLE

```
$include S20ANA.LIB;
$include S20FUN.LIB;
$include OVPN.LIB;
...
...
#RES("R1",1000,10,1,2);
```

$section

File.SRG [X] File.LIB [ ]

FUNCTION

Allows the separation of the .srg file in different section for each component or test type (resistors, diode, capacitors discharge, adjacent pin short test, ….)

SYNTAX

$section comment**;**

comment: alphanumeric string long up to 30 characters that identifies any different section (resistors, diode, capacitors discharge, adjacent pin short test, ….).

NOTE

The source program generation automatically generates the various test sections. During the optimization process, the order of the measurements within the various sections is changed in order to reduce the number of test heads movements required.

EXAMPLE

```
$section CAPACITORS DISCHARGE;

#DISCAP("DISC11",10,53,306);
#DISCAP("DISC8",10,136,137);
#DISCAP("DISC9",100,145,146);
….
….
$section RESISTORS;
….
$section CAPACITORS;
….
```

GLOBAL

File.SRG [ ] File.LIB [X]

FUNCTION

Allows the declaration of one or more global variable.

SYNTAX

GLOBAL var1[=VAL],var2[=VAL],...**;**

var1,var2,.... Global variable names. The name can be up to 9 characters.

NOTE

A global variable can be “seen” everywhere in the library file.

EXAMPLE

GLOBAL CONTOL,ZENTOL,ADDR;

LOCAL

File.SRG ☐ File.LIB X

FUNCTION

Allows the declaration of one or more local variable.

SYNTAX

LOCAL var1[=VAL],var2[=VAL],...**;**

var1,var2,....: local variable names. The name can be up to 9 characters.

NOTE

A local variable can be “seen” only in the macro where it is defined.

EXAMPLE

```
#RES(COMM,VALO,PERC,P1,P2)
!A,1,2;
!2,0,0,2,0,P1=1,P2=2;
LOCAL VAL;
VAL=VALO+2;
NER=N,BEE=N,VIW=N,STP=N;
ALM=0X00,ELM=0XFF,PUM=N;
VIM=O;
...
...
...
```

4.2 Commands

In-circuit language interacts directly on the hardware of the system by programming it so that it transmits the required impulses to the board under test and carries out the necessary measurements. In-circuit instructions are later grouped within summarized tables that are displayed in alphabetical order depending on the function they carry out.

DEFINITION OF A MEASUREMENT

LAB	Defines a measurement

DR1 PROGRAMMING

DRM	Defines the programming mode (voltage/current) of DR1
DRI	Defines the (clamp/programming) current of DR1
DRV	Defines the (clamp/programming) voltage of DR1
FIM	Sets the DR1 as impulsive or constant

DR2 PROGRAMMING

DR2	Defines the programming voltage of DR2
D2P	Sets the DR2 as impulsive or constant

CKGEN PROGRAMMING

NCK	Defines the number of clock intervals of ckgen
CKN	Defines the first front of the signal output by ckgen (lead-in/lead-out)

ANALOGUE GUARDING PROGRAMMING

GUA	Inserts one or more analogue guarding points
EXG	Establishes whether GUA follows DR1 or defines the voltage on line EXGUARD

DIGITAL GUARDING PROGRAMMING

ONE	Inserts one or more digital guarding points for the high level
ZER	Inserts one or more digital guarding points for the low level

COUNTER PROGRAMMING

MMD	Defines the measuring mode (voltage on LI1/LI4/impulses)
LOW	Defines the lower threshold of the measuring range
HIG	Defines the upper threshold of the measuring range
MES	Defines the expected value for the measurement
EXS	Specifies that the measuring strobe is external (on line EXST)
ESF	Specifies the front (rise/lowering) used for the measurement strobe

VOLTMIS PROGRAMMING

MMD	Defines the measuring mode (voltage on LI1/LI4/impulses)
LOW	Defines the lower threshold of the measuring range
HIG	Defines the upper threshold of the measuring range
MES	Defines the expected value for the measurement
EXS	Specifies that the measuring strobe is external (on line EXST)
ESF	Specifies the front (rise/lowering) used for the measurement strobe

MEASUREMENT LENGTH

TIM	Defines the length of the measurement
WAI	Defines the interval between two measurements

MEASURING MODE

NRC	Defines the number of measurement recycles
MRC	Defines the measurement recycle mode (medium/fine for the first PASS/highest/lowest value)
VIM	Defines the unit of measurement of the value read
VIW	Displays the measured value in the test area

PUL MODULE PROGRAMMING

PUM	Specifies whether a pull-up/pull-down is carried out
PUL	Defines the resistance value

CHANNELS AND CHANNEL RELAYS PROGRAMMING

LN1	Defines the mode used to connect the instrumentation to line 1
LN2	Defines the mode used to connect the instrumentation to line 2
LN3	Defines the mode used to connect the instrumentation to line 3
LN4	Defines the mode used to connect the instrumentation to line 4
ALM	Closes/opens the relays that connect the LAM instrumentation to the measuring lines
ELM	Closes/opens the relays that connect the external instrumentation to the measuring lines
LI1	Connects one or more channels to line 1
LI2	Connects one or more channels to line 2
LI3	Connects one or more channels to line 3
LI4	Connects one or more channels to line 4
CLE	Disconnect the lines (LI1/LI2/LI3/LI4) from the channels to which they are connected
HLD	Maintains the connection of the channels to the lines between two measuring operations
HIM	Connects a high impedance amplifier between the channel and the measuring unit (for digital measurements)
ICH	Inverts the connection of the channels on line 1, line 2 and line 4 (used mainly for debug)
VCC	Defines the channel/s connected to VCC
GND	Defines the channel/s connected to VCC

EXTERNAL BUS/OPEN COLLECTOR

EB1	Closes/opens the relays on the external EB1 bus
EB2	Closes/opens the relays on the external EB2 bus
WRT	Writes on the external EB1/EB2 buses
OPC	Pilots open collector lines

POWER SUPPLY

PW1	Programs PW1 power supply voltage
PW2	Programs PW2 power supply voltage
PW3	Programs PW3 power supply voltage
PW4	Programs PW4 power supply voltage
PW5	Programs PW5 power supply voltage
PW6	Programs PW6 power supply voltage

JUMPS/SUBROUTINES

BER	Performs a jump if the partial error flag is enabled
BOK	Performs a jump if the partial error flag is disabled
JER	Performs a jump if the global error flag is enabled
JOK	Performs a jump if the global error flag is disabled
JUM	Performs an unconditional jump
SUB	Runs a subroutine
SER	Runs a subroutine if the partial error flag is enabled
SOK	Runs a subroutine if the partial error flag is disabled
RET	Defines the return point of a subroutine

STACK-RELATED OPERATIONS

PSH	Enters a value in the stack
POP	Extracts the last value entered in the stack and performs the operation defined by the instruction
FRC	Forces the measured value

UTILITIES

BEE	Generate an acoustic warning
SCR	Displays or prints a string
DEB	Indicates whether the debug environment has been selected or not
FAR	Defines the parameters for macro @OPIC (max. distance between two pins)
MAX	Defines the parameters for macro @OPIC (max. n. of pin pairs)
LNK	Links to another test program
POL	Displays the network associated to the faulty channel
PRN	Prints the information displayed in the test report window of the measurement
STP	Stops the program and waits for the operator to press a key to continue
SYN	Enables to synchronies the voltage used for insulations tests carried out with the mains voltage

ERROR FLAG

CFE	Resets the global error flag enabled by the measurement (used to test board benches), but issues an error report at the end of the program
NER	Masks the partial error flag enabled by the measurement
INV	Inverts the status of the global error flag (resetting it if enabled and vice versa)

FLYING PROBE

FLP	Pre-positions contact mobile probes
FLU	Pre-positions non contact mobile probes
FLY	Indicates that a test is in progress on a Flying Probe
MOV	Inhibits the pre-positioning of mobile probes
SCA	Defines the measuring scale used by FEMTO
UPG	Defines the mode used to ground the channel connected to line 2

ALM

File.SRG [X] File.LIB [X]

FUNCTION

Connects/disconnect the eight measurement lines to the internal modules of the system by closing/opening the corresponding relay.

SYNTAX

ALM = val

Val: value which indicates the relay configuration:

0X00 ≤ val ≤ 0XFF

The least significant bit of "val" (LSB) corresponds to line 1 (LI1) while the most significant bit (MSB) corresponds to line 8 (LI8).
To close the relay corresponding to the line, the relative bit must be put to 0 (zero).
To open the relay corresponding to the line, the relative bit must be put to 1 (one).

DEFAULT VALUE

ALM=0X00

SEE ALSO

ELM

NOTE

The bus lines can be connected both to the internal and the external instruments using the ALM and ELM commands in all the possible combinations.

EXAMPLE

EXAMPLE 1

LAB="PRV1",LN1=0,MMD=V4,ALM=0X06;
* Correspondence between the value of the ALM command and the lines:

ALM=0x06:	0	0	0	0	0	1	1	0
	LI8	LI7	LI6	LI5	LI4	LI3	LI2	LI1

In this case, line1 is connected to driver 1 (LN1=0) and line 4 is connected to the voltage meter (MMD=V4). Line 2 and 3 a disconnected.

EXAMPLE 2

ALM=0X00,ELM=0XFF,PUM=N

In this case the eight instruments line are connected to the internal module of the system (ALM=0X00) and are disconnected from the external modules (ELM=0XFF) the pull-up resistor is disconnected (PUM=N).

BEE

File.SRG [X] File.LIB [X]

FUNCTION

Inserts an acoustic signal during the execution of the current test.

SYNTAX

BEE = Y/N

Y: a beep is emitted during the current test.
N: Command disabled.

DEFAULT VALUE

BEE=N

SEE ALSO

STP, **SCR**, **VIW**

EXAMPLE

EXAMPLE 1

```
LAB="BOARD-1",NER=Y,SCR="\ -- TEST END -- ",WAI=500,BEE=Y;
*- it shows the message: ----- TEST END --*
- executes a wait loop of 500ms.*
- emits a beep
```

EXAMPLE 2

```
***   Trimmer   ****
LAB="ADJUST P1",BEE=Y,NER=Y,STP=Y,SCR=" Turn P1 clockwise until it stops";
#RES("P1-47K",47000,15,71,73),GUA=72;

*The first line in the example:
- shows the message,
- emits a beep,
- stops program execution (STP=Y).
The program execution restarts after the operator has pressed the START button or the space bar on
the keyboard. The P1 measurement is then executed.*
```

BER

File.SRG [X] File.LIB [X]

FUNCTION

Interrupts the sequential execution of the program and executes a jump to a specified label, if an error has occurred in the current test.

SYNTAX

BER = "label"

Label: label which identifies the test in the program to jump to.

SEE ALSO

BOK, **CFE**, **NER**, **JER**, **JOK**, **JUM**

NOTE

When this command is used in the library environment, it is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).
During debug, the modification of a test containing the BER command is not saved in the ".OBJ" file.
In the debug environment, this command is ignored by the system because one test at a time is considered and jumping to another point in the program serves no purpose.
Unlike JER, the BER command executes the jump if there is an error in the current test, independently from whatever has occurred at any other point in the program.
The NER=Y command does not influence the jump selection.
The name used as the command argument must always be in capital letters even if the destination label uses small letters.

EXAMPLE

```
***** Board presence test *****
#BOARD("NO BOARD_A",288,131),BER="M_7";
...
...
LAB="M_7",NER=Y,SCR="\\ ***** TEST NOT COMPLETED ***** ";
LAB="END",NER=Y;
```

If the board to be tested is not present, a jump to the M_7 label is executed. This label shows the message "TEST NOT COMPLETED" and the program ends.

BOK

File.SRG [X] File.LIB [X]

FUNCTION

Interrupts the sequential execution of the program and executes a jump to a specified label if no errors have occurred in the current test.

SYNTAX

BOK = "label"

label: label that identifies the test in the program to jump to.

SEE ALSO

BER, **CFE**, **NER**, **JER**, **JOK**, **JUM**

NOTE

When this command is used in the library environment, it is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).
During debug, the modification of a test containing the BER command is not saved in the ".OBJ" file.
In the debug environment, this command is ignored by the system because one test at a time is considered and jumping to another point in the program serves no purpose.
Unlike JOK, the BOK command executes the jump if there isn't an error in the current test, independently from whatever has occurred at any other point in the program.
The NER=Y command does not influence the jump selection.
The name used as the command argument must always be in capital letters even if the destination label uses small letters.

EXAMPLE

```
….
#CTO("JUMPER-1",100,97),BOK="MSG1";
#RES("R1",1000,10,1,2);
#RES("R2",10000,5,3,4);
#RES4("R3",0.5,5,5,6,7,8);
LAB="MSG1",NER=Y,TIM=1000,SCR="JUMPER-1 Present: No tested: R1, R2, R3!!",
#RES4("R4",0.5,5,9,10,11,12);
…..
```

* If there is the contact between test points 100 and 97, the R1, R2 and R3 will not be executed, it will be executed a jump to label MSG1 that shows the message "JUMPER-1 Mounted: R1, R2, R3!! Not tested". The test will go on testing R4.*

CFE

File.SRG [X] File.LIB [X]

FUNCTION

Clears the global error flag previously activated by a failing test. This command influences the behavior of the subsequent JER and JOK commands. This command is particularly used on multiple boards' test where there is a single program but each single board must be tested separately. The CFE=Y command used at the end of each board test, partially resets the error flag so that it will not be set while testing the subsequent board. Anyway, the total report on the board tests is always given at the end of the program.

SYNTAX

CFE=Y/N

Y: clears the global error flag activated by a previous failing test.
N: command disabled.

DEFAULT VALUE

CFE=N

SEE ALSO

NER, **JOK**, **JER**

EXAMPLE

```
LAB="PT1",SCR=" BOARD TEST 1",STP=Y;
#RES("R1-1",1000,10,23,56);
#RES("R2-1",470,10,46,48);
LAB="L1",CFE=Y,NER=Y;

LAB="PT2",SCR=" BOARD TEST 2",STP=Y;
#RES("R1-2",1000,10,43,96);
#RES("R2-2",470,10,67,18);
LAB="END",NER=Y;
```

* After the test of board 1, any errors are cleared (CFE=Y) to go on to test board 2*

CKN

File.SRG [X] File.LIB [X]

FUNCTION

Enables the negative clock in output from the clock generator.

SYNTAX

CKN=Y/N

Y: enables the negative clock.
N: command disabled

DEFAULT VALUE

CKN=N

SEE ALSO

NCK, **LN3** (=2)

EXAMPLE

***** TEST CLK LN3=2 MMD=N *****

LAB="NCK=100",NCK=100,LN3=2,LI3=409,LI4=409**,CKN=Y**,MES=100,HIG=100,LOW=100,DR2=5,MMD=N,TIM=.01;

* The clock generator is programmed to give a negative signal in output formed by 100 pulses (NCK=100) on channel 409*

CLE

File.SRG X File.LIB X

FUNCTION

Disconnect lines LI1,LI2,LI3,LI4 from the family of system channels they are connected to.

SYNTAX

CLE=channels [ALL]

channels: Family of channels to disconnect from the lines. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

ALL: When this option is used, all the system channels will be disconnected from the lines.

SEE ALSO

HLD

EXAMPLE

LAB="LAB1",CLE=1-10;
Disconnects channel 1 and 10 from the lines

LAB="LAB2",CLE=10/15
Disconnects channels 10,11,12,13,14,15 from the lines

LAB="LAB3",CLE=1-10/15
Disconnects channels 1,10,11,12,13,14,15, from the lines

D2P

File.SRG [X] File.LIB [X]

FUNCTION

Programs the operating mode of Driver 2 as a pulsing or a constant voltage generator.

SYNTAX

D2P=Y/N

Y: driver 2 is programmed as a pulsing voltage generator.
N: driver 2 is programmed as a constant voltage generator.

DEFAULT VALUE

D2P=Y

SEE ALSO

DR2

NOTE

When the command D2P=Y is used, Driver 2 generates a pulse with a duration equal to the time specified in the TIM command.

EXAMPLE

```
LAB="PRO1",DR2=5,D2P=N,TIM=500,LN3=1,
LI3=157,LI2=188,LI4=189,HLD=Y,NER=Y;
* Driver 2 generates a constant voltage of +5V *

LAB="PRO2",DR2=5, TIM=500,LN3=1,
LI3=157,LI2=188,LI4=189,HLD=Y,NER=Y;
*Driver 2 generates a pulsing voltage of +5V, the pulse duration is equal to 500ms*
```

DEB

File.SRG [ ] File.LIB [X]

FUNCTION

Permits the programmer to recognize the working environment (Test/Debug).

SYNTAX

DEB=Y/N

Y: it indicates: “Debug environment” active
N: it indicates: “Debug environment” not active

DEFAULT VALUE

DEB=N if the debug environment is active
DEB=Y if the test environment is active

SEE ALSO

FLY

NOTE

This command is principally used with the if-else-endif instruction in the library files or in the File.aut (Autobedug file) to recognize that the debug environment is active.

EXAMPLE

```
if (DEB=Y)
...
...
else
...
...
endif
```

DR2

File.SRG [X] File.LIB [X]

FUNCTION

Programs the voltage of Driver 2.

SYNTAX

DR2=val

val: value expressed in Volts, for programming Driver2 -10.240 ≤ val ≤ +10.235

N.B. The unit of measure must be omitted since the system considers it by default.

DEFAULT VALUE

DR2=0

SEE ALSO

D2P

EXAMPLE

LAB="TEST1",DR2=5,D2P=N,TIM=500,LN3=1, LI3=157,LI2=188,LI4=189,HLD=Y,NER=Y;
* Driver 2 is programmed to generate a constant voltage of +5V *

LAB="PW1",LI4=1,TIM=1000,LOW=-4.85,MES=-5.0, HIG=-5.1,VIW=Y, DR2=-5,LN2=1,LI2=6;
* Driver 2 is programmed to generate a pulsing voltage of -5V. (The command is not specified so D2P=Y is considered by default which means Driver 2 is pulsing)

DRI

File.SRG [X] File.LIB [X]

FUNCTION

Programs the current value of Driver 1, when driver 1 is programmed as a current generator, or programs the limitation current of Driver 1 when driver 1 is programmed as voltage generator.

SYNTAX

DRI=val

val: value expressed in mA, of the current to be programmed.

If **DRM=I** (Driver 1 programmed as a current generator), "val" indicates the current value of Driver 1. The current resolution varies according to the following ranges:

0 mA	≤ val ≤	0.1mA	steps of 25nA
0.1mA	< val ≤	1mA	steps of 250nA
1mA	< val ≤	10mA	steps of 2.5µA
10mA	< val ≤	100mA	steps of 25µA

If **DRM=V** (Driver 1 programmed as a voltage generator), "val" indicates the value of the current clamp and can assume the following values:

1mA, 10mA, 100mA, 200mA, 300mA, 400mA, 500mA.

DEFAULT VALUE

DRI=0 e **DRM**=I

SEE ALSO

DRV, **DRM**

NOTE

The unit of measure must be omitted since the system considers it by default.
When the DRM=I (Driver 1 as a current generator) and MMD=C (small capacity measurement) commands are specified in the same test, the current value is automatically considered to be in µA to obtain a better resolution.
The DRI=0 and VIM=O commands are incompatible if used in the same test because a resistance value can't be measures with a null current.

EXAMPLE

LAB="BOARD-1 CLOSE RL1",**DRM**=V,**DRV**=9,**DRI**=200,TIM=10,LI1=5,LI2=32,NER=Y;
* Driver 1 is programmed as a voltage generator (DRM=V) with a current clamp value equal to 200mA.*

LAB="BOARD-2 CLOSE RL1",**DRM**=I,**DRV**=4,**DRI**=100,TIM=10,LI1=5,LI2=32,NER=Y;
* Driver 1 is programmed as a current generator of 100mA (DRI=100) with a voltage clamp value of +4V*

LAB="C1",DRM=I,**DRI=5**,**MMD=C**,TIM=0.1,MES=0.5,LOW=0.45,HIG=0.55;
Measurement of a 1nF capacitor with a current of 5µA

DRM

File.SRG [X] File.LIB [X]

FUNCTION

Programs the operating mode of Driver 1.

SYNTAX

DRM=mode

mode: It indicates the operating mode of Driver 1 and can be:

I: indicates that Driver 1 is programmed as a current generator with voltage clamp.
V: indicates that Driver 1 is programmed as a voltage generator with current clamp
L: indicates that Driver 1 is programmed as a voltage generator with a 4.7µF capacitor in parallel to perform the inductor tests.

DEFAULT VALUE

DRM=I

SEE ALSO

DRV, **DRI**

EXAMPLE

EXAMPLE 1

LAB=" ZEN",**DRI=80**,**DRV=5**,**DRM=I**,
WAI=10,NRC=1000,MRC=J,TIM=50;
Driver 1 programmed as a current generator of 80mA with voltage clamp of 5V

EXAMPLE 2

LAB="BOARD-1 CLOSE RL1",**DRM=V**,**DRV=9**, **DRI=200**,TIM=10,LI1=5,LI2=32,NER=Y;
Driver 1 programmed as a voltage generator of 9Vwith current clamp of 200mA

EXAMPLE 3

LAB="Q3 R16 R17 R18 ON",LI4=54,LI1=58,MES=4.2,HIG =4.4,LOW=3.7,**DRM=L**,DRV=4, DRI=100, TIM=.01;
Driver 1 programmed as a voltage generator with a 4.7µF capacitor in parallel (inductor measurement).

DRV

File.SRG [X] File.LIB [X]

FUNCTION

Programs the Driver 1 voltage value.

SYNTAX

DRV=val

val: value, expressed in Volts, of the current to be programmed.

If **DRM=I** (Driver 1 programmed as a current generator), "val" indicates the voltage clamp. The voltage resolution varies according to the following ranges:

-5V	≤ val ≤	-1V	resolution 5mV
-1V	≤ val ≤	+1V	resolution 0.5mV
+1V	≤ val ≤	+5V	resolution 5mV

If **DRM=V** (Driver 1 programmed as a voltage generator), "val" indicates the voltage value to be programmed. The voltage resolution varies according to the following ranges:

-50V	≤ val ≤	-10V	resolution 25mV
-10V	≤ val ≤	+10V	resolution 5mV
+10V	≤ val ≤	+50V	resolution 25mV

DEFAULT VALUE

DRV=0

SEE ALSO

DRI, **DRM**

NOTE

The unit of measure must be omitted since the system considers it by default.

EXAMPLE

LAB="BOARD-1 CLOSE RL1",**DRM=V**,**DRV=9**, **DRI=200**,TIM=10,LI1=5,LI2=32,NER=Y;
Driver 1 programmed as a voltage generator of +9V with a current clamp of 200 mA.

LAB="BOARD-2 CLOSE RL1",**DRM=I**,**DRV=2**, **DRI=50**,TIM=10,LI1=5,LI2=32,NER=Y;
Driver 1 programmed as a current generator of 50mA and voltage clamp of +2V

EB1

File.SRG [X] File.LIB [X]

FUNCTION

Connects the 16 lines of the external bus 1 (EB1-0 ÷ EB1-15) to the S20 system bus by closing the correspondent relay. In this way it is possible to read and write data from any external module connected. The external bus 1 lines are on the LAM board output connector.

SYNTAX

EB1=val

val: Value which indicates the relays configuration: 0X0000 ≤ val ≤ 0XFFFF

The least significant bit of "val" (LSB) corresponds to the EB1-0 line while the most significant (MSB), corresponds to the EB1-15 line.

To close the relay corresponding to a certain line, it is necessary to put the relative bit to 0 (zero).
To open the relay corresponding to a certain line, it is necessary to put the relative bit to 1 (one).

DEFAULT VALUE

EB1=0XFFFF

SEE ALSO

EB2, **WRT**, **MMD** (=R), **LN4** (=4), **NCK**, **CKN**

NOTE

The write operation from the S20 bus to the external world is done using the WRT command.

The read operating from the external world to the S20 bus is done using the MMD=R command.

EXAMPLE

```
EB1=0X0000;
EB1=0X0001;
EB1=0X8003;
```

EB2

File.SRG [X] File.LIB [X]

FUNCTION

Connects the 16 lines of the external bus 2 (EB2-0 ÷ EB2-15) to the S20 system bus by closing the correspondent relay. In this way it is possible to read and write data from any external module connected. The external bus 1 lines are on the LAM board output connector.

SYNTAX

EB2=val

val: value which indicates the relays configuration: 0X0000 ≤ val ≤ 0XFFFF

The least significant bit of "val" (LSB) corresponds to the EB2-0 line while the most significant (MSB), corresponds to the EB2-15 line.

To close the relay corresponding to a certain line, it is necessary to put the relative bit to 0 (zero).
To open the relay corresponding to a certain line, it is necessary to put the relative bit to 1 (one).

DEFAULT VALUE

EB2=0XFFFF

SEE ALSO

EB1, **WRT**, **MMD** (=R), **LN4** (=4), **NCK**, **CKN**

NOTE

The write operation from the S20 bus to the external world is done using the WRT command.
The read operating from the external world to the S20 bus is done using the MMD=R command.

EXAMPLE

```
EB2=0X0000;
EB2=0X0001;
EB2=0X8003;
```

ELM

File.SRG [X] File.LIB [X]

FUNCTION

Connects/disconnects the eight measurement lines to the external system modules by closing/opening the corresponding relays.

SYNTAX

ELM = val

val: value which indicates the relays configuration: 0X00 ≤ val ≤ 0XFF

The least significant bit of "val" (LSB) corresponds to line 1 (LI1) while the most significant bit (MSB) corresponds to line 8 (LI8).

To close the relay correspondent to the line, the relative bit must be put to 0 (zero).
To open the relay correspondent to the line, the relative bit must be put to 1 (one).

DEFAULT VALUE

ELM=0XFF

SEE ALSO

ALM

NOTE

The bus lines can be connected both to the internal and the external instruments using the ALM and ELM commands in all the possible combinations.

EXAMPLE

EXAMPLE 1

LAB="PRV1",LN1=0,MMD=V4,**ELM=0X00**; * Correspondence between the ELM instruction value and the lines:

ELM=0x00:	0	0	0	0	0	0	0	0
	LI8	LI7	LI6	LI5	LI4	LI3	LI2	LI1

In this case, the line 1 is connected to Driver 1 (LN1=0) and line 4 is connected to the VOLT MIS (MMD=V4). Lines 2 and 3 are disconnected

EXAMPLE 2

ALM=0X00,ELM=0XFF,PUM=N

In this case the eight instruments line are connected to the internal module of the system (ALM=0X00) and are disconnected from the external modules (ELM=0XFF) the pull-up resistor is disconnected (PUM=N).

ESF

File.SRG X File.LIB X

FUNCTION

Defines the control edge of the measurement strobe signal, originating from outside the system.

SYNTAX

ESF=Y/N

Y: enables the external strobe on the falling edge.
N: enables the external strobe on the rising edge.

DEFAULT VALUE

ESF=N

SEE ALSO

EXS

EXAMPLE

```
ESF=Y;
ESF=N;
```

EXG

File.SRG [X] File.LIB [X]

FUNCTION

Connects the input of the Guarding generator to the external line EXGUARD of the service bus.

SYNTAX

EXG=Y/N

Y: connects the Guarding generator to the external line EXG of the service bus.
N: connects the Guarding generator to Driver 1.

DEFAULT VALUE

EXG=N

EXAMPLE

EXG=Y;
EXG=N;

EXS

File.SRG X File.LIB X

FUNCTION

Enables the use of the internal/external strobe signal for a certain test.

SYNTAX

EXS=Y/N

Y: connects the strobe signal to the external line EXST of the service bus.
N: the internal strobe signal is used for the test.

DEFAULT VALUE

EXS=N

SEE ALSO

ESF

EXAMPLE

EXS=Y;
EXS=N;

FAR

File.SRG [X] File.LIB [X]

FUNCTION

It defines the maximum distance between two selected pins, thus enabling to set the maximum distance between the two pins selected for the pairs of macros @OPICN and @OPICP. This value indicates the closest number of pins that have to be tested, but provides no unit of measurement because it is used to specify a limit for the maximum number of pairs that can be set-up for each pin. For a component with N pins, the maximum number of pairs that can be set-up is N-2. Therefore, command FAR is effective only if used with a measurement value of (N-2)/2. When it is used with command MAX=. in macro @DEF, it defines the necessary parameters for macros @OPICN and @OPICP, which indicate whether the component has correctly been soldered to the board. Command MAX defines the maximum number of pin pairs that macros @OPICN and @OPICP can form for each pin under test, while command FAR defines the maximum distance between the two pins selected to form the pair.

SYNTAX

FAR=val

val: indicates the max distance between the two pins chosen to form the pair in the @OPICN and @OPICP macros.

DEFAULT VALUE

FAR=0

SEE ALSO

MAX

NOTE

The unit of measurement must be omitted because it is considered by default.

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

EXAMPLE

```
@DEF (GND=7,VCC=14,MAX=22,FAR=5);
@OPICN(“IC1”,1,2,3,4,5,6,7,8,9,10,11,12,13,14);
```

Macro OPICN carries out the test on the open pins of component IC1, which has a n type sub-layer. The macro attempts to form 22 pairs of pins for each pin of the component by using a distance of 5 pins

FIM

File.SRG X File.LIB X

FUNCTION

Programs Driver 1 as a pulsing voltage generator or a constant voltage generator.

SYNTAX

FIM=Y/N

Y: driver 1 is programmed as a constant voltage generator.
N: driver 1 is programmed as a pulsing voltage generator.

DEFAULT VALUE

FIM=N

SEE ALSO

DRI, **DRM**, **DRV**

NOTE

When the FIM=N command is specified, Driver 2 generates a pulse with a duration equal to the time indicated in the TIM command.

EXAMPLE

LAB=“TEST1”,DRM=V,DRV=5,DRI=300,FIM=Y;
Driver 1 is programmed as a constant voltage generator

LAB=“TEST2”,DRM=V,DRV=5,DRI=300,FIM=N,TIM=100;
Driver 1 is programmed as a pulsing voltage generator. The pulse duration is equal to 100ms

FLP

File.SRG [X] File.LIB [X] Only for Flying Probe systems [X]

FUNCTION

This command is used only on Flying Probe test systems. The FLP command is used by the automatic optimization procedure to preset the probe position for the next test, thus optimizing the time of the test. **Note: there is a contact** between the point and the prod of the mobile probe.

SYNTAX

FLP=channel

channel: Channel or channels on which the flying probes must move to, in advance. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels. If more than four channels are specified, the exceeding ones will be ignored.

SEE ALSO

FLU

NOTE

This command is mainly used in the automatic path optimization process during the test. (FLYING PROBES\Optimization menu).

EXAMPLE

#RES("R1",1000,10,1,23),FLP=2-24;
#RES("R2",100,10,2,24);
During the R1 measurement, the ATE moves two flying probes to channels 1 and 23 and the other two to channels 2 and 24 thus positioning the two probes for the next test (R2 resistor)

FLU

File.SRG [X] File.LIB [X] Only for Flying Probe systems [X]

FUNCTION

This command is used only on Flying Probe test systems. It preset the probe on the point correspondent to the channel to optimizing the time of the test. **NOTE:** there is no contact between the point and the test nail.

SYNTAX

FLU=channel

channel: channel or channels on which the flying probes must move to, in advance. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels. If more than four channels are specified, the exceeding ones will be ignored.

SEE ALSO

FLP

NOTE

This command is mainly used in the automatic path optimization process during the test. (FLYING PROBES\Optimization menu).

EXAMPLE

#RES("R1",1000,10,1,23),FLU=2-24;
#RES("R2",100,10,2,24);
During the R1 measurement, the ATE moves two flying probes to channels 1 and 23 and the other two to channels 2 and 24 thus positioning the two probes for the next test (R2 resistor) but the two points aren't touched

FLY

File.SRG [X] File.LIB [X] Only for Flying Probe systems [X]

FUNCTION

Enables to determine whether the test is carried out on boards configured for Flying Probes or bed of nails. The command returns Y for boards configured for Flying Probes or N when this condition does not apply.

SYNTAX

(**FLY**=Y/N)

Y: condition that indicates a "board configured for Flying Probe equipment"
N: condition that indicates a "board configured for beds of nails"

VEDERE ANCHE

DEB

NOTE

This command is principally used with the if-else-endif instruction in the library files or in the File.aut (Autobedug file) to recognize if we are working on a Flying Probe or on a bed of nail equipment.

EXAMPLE

```
if (FLY=Y)
...
...
else
...
...
endif
```

FRC

File.SRG [X] File.LIB [X]

FUNCTION

Forces the value indicated in the present command in the temporary buffer, in place of the value read from the UUT. This command is mainly used to perform operations with the stack.

SYNTAX

FRC=val

val: value forced in the temporary buffer.

if **MMD=V1/V4/D/C:** -51.2V ≤ val ≤ +51.2V
if **MMD=N/R:** 0X0000 ≤ val ≤ 0XFFFF

SEE ALSO

POP, **PSH**

NOTE

When this command is used in the library environment, it is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).
The value of this command can be expressed both as decimal and hexadecimal value. In the first case, there is no need of specifying a prefix, while in the second one it is necessary to enter prefix "0X" before the value.
Commands FRC, POP and PSH can be used to carry out several tests, since it is sufficient to arithmetically process the results in order to attain the desired value.
The FRC, POP and PSH command, allows the execution of several tests arithmetically processing the results in order to obtain the desired one.
These commands can be used independently of each other. If more than one is used in the same test, they will be executed with the following priority: FRC, POP, PSH.
If the FRC and PSH=Y commands are used together in the same test, the value indicated in the FRC command will be stored in the stack.

EXAMPLE

The following tests execute the measurements on 7 diodes, calculating the average

```
#DIODIR("c-1 P5-PT1_SUM",1,4),PSH=Y,NER=Y;
#DIODIR("c-1 P5-T1_ SUM",1,5), POP=+,PSH=Y,NER=Y;
#DIODIR("c-1 P5-T2_ SUM",1,8), POP=+,PSH=Y,NER=Y;
#DIODIR("c-1 P5-T4_ SUM",1,9), POP=+,PSH=Y,NER=Y;
#DIODIR("c-1 P5-P7_ SUM",1,2), POP=+,PSH=Y,NER=Y;
#DIODIR("c-1 P5-P4_ SUM",1,3), POP=+,PSH=Y,NER=Y;
#DIODIR("c-1 P5-P2_ SUM",1,7), POP=+,PSH=Y,NER=Y;
LAB="c-1 AVERAGE",NER=Y,FRC=7,POP=/,VIW=Y;
```

The first measured value is stored in stack (**PSH=Y**)
The second measured value is added to the stack value; the result is stored in stack (**POP=+,PSH=Y**)
….
The seventh measured value is added to the stack value and the result is stored in stack (**POP=+,PSH=Y**).
Instruction LAB enables to place value 7 in the measure buffer (**FRC=7**). The result of the addition present in the stack is divided by the value that had been entered in the measure buffer. The corresponding result is placed in the measure buffer (**POP=/**).

GND

File.SRG [X] File.LIB [X]

FUNCTION

Specifies the channel/s connected to ground in the board to be tested. This definition is subsequently used in the macro calls and in the automatic debug procedures.

SYNTAX

GND=channel

channel: channel or channels connected to ground. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

SEE ALSO

VCC

NOTE

The definition of the channels connected to ground and power must be specified with the @DEF macro before using the @OPICN or @OPICP macros. Refer to the example illustrated below.

EXAMPLE

@DEF (GND=7,VCC=14);
@OPICN("IC1",1,2,3,4,5,6,GND,8,9,10,11,12,13,VCC);
After the execution of the @DEF macro, the 7 and 14 channels are considered as connected to ground and power respectively.

GUA

File.SRG X File.LIB X

FUNCTION

Connects line 3 to one or more system channels.

SYNTAX

GUA=channel [ALL]

channel: family of channels to connect to line 3. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

ALL: when this option is specified, all channels in the system will be connected to line 3.

SEE ALSO

LI1, **LI2**, **LI4**, **CLE**, **HLD**

NOTE

The GUA command is automatically inserted by the software procedures (TOOLS\Software menu) that write the guardings for those macros using the LN3=0 command.

It is possible to use the LI3 command as an alias of the GUA command when the LN3=0 command has been used.

EXAMPLE

LAB="BOARD-1 CLOSE RL1",DRM=V,DRV=9,DRI=200,TIM=10,LI1=5,GUA=37,NER=Y;
Channel 37 is connected to line 3 for which command LN3=0 has also been performed

LAB="PRO1",LN1=0,DRM=V,DRV=0,DRI=100,GUA=171-250;
Channels 171 and 250 are connected to the guarding generator

LAB="PRO2",LN1=0,DRM=V,DRV=0,DRI=100,GUA=171/250;
Channel from 171 to 250 are connected to the guarding generator

HIG

File.SRG [X] File.LIB [X]

FUNCTION

Programs the high threshold of the measurement. Above this threshold, the read value is considered as incorrect.

SYNTAX

HIG=val

val: high threshold of the measurement that can assume the following values:

if **MMD=V1/V4/D/C:** -51.2V ≤ val ≤ +51.2V

if **MMD=N/R:** 0X0000 ≤ val ≤ 0XFFFF

DEFAULT VALUE

HIG=0

SEE ALSO

LOW, **MES**, **VIW**

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

EXAMPLE

LAB="V-MEAS",MES=5,HIG=5.5,LOW=4.5,DRI=80.000000,MMD=V4,LI4=18;
*The measurement high threshold is +5.5V

LAB="IMP-MEAS",MES=100,HIG=105,LOW=95,MMD=N,LN3=2,LI3=47,LI4=47,NCK=100;
*This measurement reads the number of pulses generated by the clock generator. The high threshold of this measurement is 105 pulses

HIM

File.SRG X File.LIB X

FUNCTION

It permits the programmer to connect an high impedance amplifier between the channel present on LINE 4 in the current test, and the meter. In this way, the impedance and the capacitance of the meter are reduced. This configuration is suitable particularly during digital measurements where it is important to have a low capacitive load to reduce the noise to the minimum.

SYNTAX

HIM=Y/N

Y: the high impedance amplifier is connected between the channel and the meter.
N: the high impedance amplifier is not connected between the channel and the meter.

DEFAULT VALUE

HIM=N

EXAMPLE

%7400(“U1”,12,586,9,23,56,95,4,49,34,846,8,2,33,57),HIM=Y;

HLD

File.SRG [X] File.LIB [X]

FUNCTION

Maintains the same channel relays programming for the previous and the current tests.

SYNTAX

HLD=Y/N

Y: enables hold
N: command disabled

DEFAULT VALUE

HLD=N

SEE ALSO

All commands which operate on the relays.

EXAMPLE

LAB="TX1",DR2=5,D2P=N,TIM=500,LN3=1,LI3=157,LI2=188,LI4=189,NER=Y;

LAB="TX2",TIM=100,MMD=N,**HLD=Y**,DR2=5,D2P=N,PUL=10,PUM=D,NRC=10,MRC=J,MES=4000,HIG=4020,LOW=3980;

The same channel relays programming is maintained for the TX1 and for the TX2 tests

ICH

File.SRG [X] File.LIB [X]

FUNCTION

Inverts the connections of the channels on line 1, line 2 and line 4 in the following way:

LI1=LI2
LI2=LI1
LI4=LI1

that is: the channel connected to line 1 will be connected to line 2 and vice-versa. The channel connected to line 1 will be also connected to line 4.

SYNTAX

ICH=Y/N

Y it inverts the connections.
N command disabled

DEFAULT VALUE

ICH=N

SEE ALSO

LI1, **LI2**, **LI4**

NOTE

This command is mainly used by the automatic debug procedures.

EXAMPLE

The DIO macro measures a diode by connecting line 1 on channel 20, line 2 on channel 20 and the Voltmis module on line 4.

#DIO(“D1”,10,20);
If we write:
#DIO(“D1”,10,20),ICH=Y;

The diode measurement will be executed by connecting line 1 on channel 20 and line 2 on channel 10.

INV

File.SRG [X] File.LIB [X]

FUNCTION

Inverts the error flag state for the current test, that is: if the flag is active, the command will reset it and vice-versa.

SYNTAX

INV=Y/N

Y: inverts the error flag for the current test.
N: command disabled

DEFAULT VALUE

INV=N

SEE ALSO

CFE

EXAMPLE

#RES("R45",100000,10,15,1),TIM=80,INV=Y;
If the R45 measurement fails, the error flag of the current test is reset and the program will give: PASS. If the R45 measurement passes, the error flag of the current test is activated and the program will give FAIL

JER

File.SRG X File.LIB X

FUNCTION

Interrupts the sequential execution of the tests and executes a jump to the specified label if an error has occurred at any previous point in the program. This command checks the global error flag.

SYNTAX

JER="label"

label: label name that identifies the point to jump to in the program.

SEE ALSO

BER, **BOK**, **JOK**, **JUM**, **NER**, **CFE**

NOTE

When this command is used in the library environment, it is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

During debug, the modification of a test containing the JER command is not saved in the ".OBJ" file.

In the debug environment, this command is ignored by the system because it is considered one test at a time and jumping to another point in the program has no meaning.

Unlike BER, the JER command executes the jump if an error has occurred in any previous point in the program.

The NER=Y command does not influence the jump selection.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

EXAMPLE

```
#CTO("T1_PIN_2_4",257,356);
#CTO("T1_PIN_6_8",315,316),TIM=5;
#CTO("T1_PIN_8_10",315,357),TIM=10;


LAB="JUMP",NER=Y,JER="END";
...
...
...
LAB="END",PW2=-1,NER=Y;
```

A jump to the END label is executed if an error has occurred

JOK

File.SRG [X] File.LIB [X]

FUNCTION

Interrupts the sequential execution of the tests and executes a jump to the specified label if no errors have occurred at any previous point in the program. This command checks the global error flag.

SYNTAX

JOK = “label”

label: label name that identifies the point to jump to in the program.

SEE ALSO

BER, **BOK**, **CFE**, **NER**, **JER**, **JUM**

NOTE

When this command is used in the library environment, it is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

During debug, the modification of a test containing the JOK command is not saved in the “.OBJ” file.

In the debug environment, this command is ignored by the system because it is considered one test at a time and jumping to another point in the program has no meaning.

Unlike BOK, the JOK command executes the jump no errors have occurred in any previous point in the program.

The NER=Y command does not influence the jump selection.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

EXAMPLE

```
#RES("ADAPTER",1000,5,415,416);
LAB="ADAPTER OK",NER=Y,JOK="ADAPTER CONT";
...
...
...
LAB="ADAPTER CONT",TIM=100,NER=Y;
*A jump to the ADAPTER CONT label is executed if the fixture check test passed*
```

JUM

File.SRG [X] File.LIB [X]

FUNCTION

Interrupts the sequential execution of the program and performs an unconditioned jump to the specified label.

SYNTAX

JUM = “label”

label: label name that identifies the point to jump to in the program.

SEE ALSO

BER, **BOK**, **JER**, **JOK**

NOTE

When this command is used in the library environment, it is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

During debug, the modification of a test containing the JUM command is not saved in the “.OBJ” file.

In the debug environment, this command is ignored by the system because it is considered one test at a time and jumping to another point in the program has no meaning.

Unlike BER, BOK, JER and JOK, command JUM performs the jump unconditionally without being affected by the values of the flags related to partial or global errors.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

EXAMPLE

```
LAB="M_6",NER=Y,SCR="\\Powered component tests ---- COMPLETED ---- ", JUM="END";
LAB="M_7",NER=Y, SCR="\\ ***** test program ---- NOT COMPLETED ---- ***** ";
LAB="END",NER=Y;
```

A jump to the END label will be executed after the execution of the M_6 test

LAB

File.SRG [X] File.LIB [X]

FUNCTION

Defines a label, which in an in-circuit programming environment, corresponds to a test, associating to it the name defined in field "Comment".
The **test** represents the moment in which the impulses are transmitted to the board under test and indicates also if the related results are measured.
From the code standpoint, a test identifies a sequence of instructions and/or commands that are performed when command LAB is found during the execution of the program. If the measurement detects an invalid value during the execution of command LAB, the application sets two error flags:

Partial error flag:
- it is set when the current LAB command is executed
- it applies only to the current LAB
- it is repositioned every time the LAB command is executed.

Global error flag:
- it is set the first time an error occurs
- it is globally valid for the whole program
- it indicates that at least one error has occurred during the execution of the program.

SYNTAX

LAB=“Comment”

Comment: Alphanumeric string maximum 30 characters which defines the test name.

NOTE

The LAB command has got three different meanings according to the working environment. In fact it can be used:

- in the .srg file
- in the macro calls
- in the library environment

In the .srg File: command LAB represents a test. In this context, the execution of LAB forces the execution of all the commands comprised between the definition of command LAB and the colon, which ends the instruction. Example: LAB=“L1”,TIM=100,LI1=12,LI2=23,LI4=12,LOW=4.5,MES=5,HIG=5.5;

In the macro calls: it modifies the parameters of the label with the same name, inside the macro. Example: #DIO(“D1”,45,67),LAB=“INV”,LOW=.8,MES=1.1,HIG=1.4;
The DIO macro is composed of two internal tests, the DIR and the INV test. The following command: LAB=“ INV”,LOW=.8,MES=1.1,HIG=1.4 written after the macro DIO, will vary the parameters of the internal INV test.

In the library environment: command LAB defines the instant in which the test is carried out. In this context, the execution of command LAB forces the execution of all the commands that precede instruction LAB, which must be performed in preparation of the measurement to be carried out with instruction LAB.

Example:

```
LAB=”COMM”….
BEE=Y,SCR=“TRIMMER ON ”+COMM;
AB=COMM+“ 2”;
ER=N,BEE=N,VIW=Y,STP=Y;
AB=COMM+“ T”;
```

The LAB COMM+ “ 2” label includes the BEE=Y and SCR=“...” commands.
The COMM+ “ T” label includes the NER=N,BEE=N,VIW=Y,STP=Y commands.

EXAMPLE

EXAMPLE 1
LAB="BOARD-1 CLOSE RL1",DRM=V, DRV=9,DRI=200, TIM=10,LI1=5,LI2=32,NER=Y;
Definition of test BOARD-1 CLOSE RL1 in the source program

EXAMPLE 2
#TRN(“T1”,1,2,3),LAB=“T1-BE DIR”,TIM=15;
The length of the direct conduction test for the base joint - transistor emitter has been changed to 15ms

EXAMPLE 3
NER=N,BEE=N,VIW=N,STP=N;
ALM=0X00,ELM=0XFF,PUM=N;
MMD=V4,VIM=V;
DRM=I,DRV=.5,DRI=10;
TIM=1;
LOW=-.5,MES=.1,HIG=.25;
LN1=0,LN2=0,LN3=0,LN4=0;
LI1=P1,LI2=P2,LI3=0,LI4=P1;
LAB=COMM;

Definition of a library test. The execution of command LAB forces the execution of the commands that precede command LAB

LI1, LI2, LI3, LI4

File.SRG [X] File.LIB [X]

FUNCTION

Connects line 1, line 2, line 3, and line 4, to one or more system channels.

SYNTAX

LI1=channel [ALL]; **LI2**= channel [ALL]; **LI3**= channel [ALL]; **LI4**= channel [ALL];

channel: family of channels to connect to line 1, line 2, line 3, and line 4. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

ALL: When this option is used, all the system channels will be connected to line 1, line 2, line 3, line 4.

SEE ALSO

CLE, **HLD**

NOTE

It is possible to use the ZER command as an alias of the LI1 command.

It is possible to use the ONE command as an alias of the LI2 command.

It is possible to use the GUA command as an alias of the LI3 command.

EXAMPLE

LAB="CLOSE RL1",LI*n*=5,LI2=32,NER=Y;
*Channel 5 is connected to line *n* (LI*n*=5).

LAB="PRO1",LN1=0,DRM=V,DRV=0,DRI=100,LI*n*=171-250;
* Channels 171 and 250 are connected to line *n**

LAB="PRO2",LN1=0,DRM=V,DRV=0,DRI=100,LI*n*=171/250;
* Channels from 171 to 250 are connected to line *n**

**n* indicates the line number that may be 1, 2, 3, or 4.*

LN1

File.SRG [X] File.LIB [X]

FUNCTION

Defines the type of connection of line 1 and the internal instruments.

SYNTAX

LN1= mode

mode: identifies the type of connection between line 1 and the instruments and can be:

0 connection between line 1 and Driver 1 (DR1).
1 connection between line 1 and Guarding generator (GUA).

DEFAULT VALUE

LN1=0

SEE ALSO

CLE, **DRI**, **DRM**, **DRV**, **HLD**, **LI1**

EXAMPLE

LAB="BOARD-1",**LN1=0**,LI1=49,DRM=V,DRI=100, DRV=5,TIM=100;
Connection between Driver 1 and line 1. Driver 1 is programmed as a pulsing voltage generator of +5V with current clamp equal to 100mA. The pulse duration is 100ms

LAB="GUA=.1", **LN1=1**,DRM=I,DRV=.1,DRI=10;
Connection between line 1 and Guarding generator that is programmed as a current generator to 10mA and limited to +0.1V

LN2

File.SRG [X] File.LIB [X]

FUNCTION

Defines the type of connection of line 2 and the internal instruments.

SYNTAX

LN2= mode

mode: indicates the type of connection between LI2 and the instruments and can be chosen between:

0 connection between line 2 and Ground.
1 connection between line 2 and Driver 2.

DEFAULT VALUE

LN2=0

SEE ALSO

CLE, **D2P**, **DR2**, **HLD**, **LI2**

EXAMPLE

LAB="CHECK DISCHARGE_B", **LN2=0**,LI2=3;
Connects the S20 system ground to line 2

LAB="PW1",LI4=1,TIM=1000,LOW=4.85,MES=5.0, HIG=5.1,VIW=Y,DR2=5,**LN2=1**,LI2=6;
Connects Driver 2 to line 2

LN3

File.SRG [X] File.LIB [X]

FUNCTION

Defines the type of connection of line 3 and the internal instruments.

SYNTAX

LN3= mode

mode: Identifies the type of connection between line 3 and the instruments and can be:

0 connection between line 3 and Guarding generator GUA.
1 connection between line 3 and Driver 2 DR2.
2 connection between line 3 and Clock generator CKGEN.
3 connection between line 3 and the external line EXL3 of the service bus.

DEFAULT VALUE

LN3=0

SEE ALSO

CLE, **CKN**, **D2P**, **DR2**, **EXG**, **HLD**, **LI3**, **NCK**

EXAMPLE

LAB="LN3=0",**LN3=0**,DRM=I,DRV=2,DRI=10,LI3=4097;
The Guarding generator is connected to line 3. Driver 1 is programmed to 10mA and limited to +2V

LAB="LN3=1",**LN3=1**,DR2=2,LI3=4097;
Driver 2, programmed to +2V, is connected to line 3

LAB="LN3=2",NCK=10,**LN3=2**,LI3=4097;
The Clock generator, programmed to give 10 pulses, is connected to line 3

LAB="LN3=3",**LN3=3**;
It connects the EXTLN3 signal, to line 3. An external instrument can be connected to EXTLN3

LN4

File.SRG [X] File.LIB [X]

FUNCTION

Defines the type of connection of line 4 and the internal instruments.

SYNTAX

LN4= mode

mode: Identifies the type of connection between line 4 and the instruments and can be:

0 line 4 is disconnected from the internal instruments.
1 connection between line 4 and Driver 2 DR2.
2 connection between line 4 and the external line EXL4 of the service bus.
3 connection between line 4 and Driver 2 DR2. Driver 2 has got a 100Ω resistor in series.
4 connection between line 4 and the Personal Computer READ signal. This command is mainly used to execute a synchronized read from the external bus *

DEFAULT VALUE

LN4=0

SEE ALSO

CLE, **D2P**, **DR2**, **HLD**, **LI4**, **MMD** (=R), **PUL**, **PUM**

EXAMPLE

LAB="+5 KO ",MES=5,LOW=4.5,HIG=5.5,LI4=215, **LN4=0**;
Line 4 is connected to the VOLT MIS module, (MMD=V4 by default) to execute a voltage measurement

LAB="DR2=.1",DR2=.1,**LN4=1**;
Driver 2 that is programmed at +0.1V is connected to line 4

LAB="LN4=2",**LN4=2**;
The EXTLN4 signal is connected to line 4 where an external instrument can then be connected

LAB="CH 10 ON",NER=Y,TIM=500,DR2=2,**LN4=3;**
* Driver 2, with the 100Ω in series, is connected to line 4. Driver 2 is programmed to give a pulsing voltage of +2V. The pulse duration is equal to 500ms.*

LAB="L4",**LN4=4**;
It executes a READ operation, synchronized with the Personal Computer READ signal

LNK

File.SRG [X] File.LIB [X]

FUNCTION

This command is used to switch the execution of the current program to another one, indicated in the LNK command.

SYNTAX

LNK="program name"

program name: name of the source program to be linked.

NOTE

The LNK command can be used in any point of the source program.

This command, when used in the library environment, is not maintained in the subsequent tests within the same macro.

The LNK command is not compatible with the SRC command. IF both are used in the same test, only the last one will be considered.

From the moment in which the LNK command is encountered, the system proceeds to execute the linked program starting from the first test.

N.B. The measurements that follow the LNK command in the first program, will not be executed.

At the end of the execution of the LNK command, the system returns to the beginning of the first program.

In the debug environment, this command is ignored by the system.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

EXAMPLE

We have got two source programs, PRG1.SRG and PRG2.SRG as follows:

```
PRG1.SRG
LAB="PW1 ",LI4=1,TIM=1000,LOW=4.85,MES=5.0,
HIG=5.1,VIW=Y,DR2=5,LN2=1,LI2=6;
LAB="PW2 ",LI4=2,TIM=1000,LOW=14.5,MES=15.0,
HIG=15.1,VIW=Y,DR2=5,LN2=1,LI2=6;
LAB="LINK ALI",NER=Y,LNK="PRG2";
```

After the PW1 and PW2 power supply test, the execution switches to PRG2.

```
PRG2.SRG
#RES("R1",100,10,12,14);
#RES("R15",270,10,12,14);
```

*After PRG2 has been executed, the execution of the link command stops, and the system returns to the beginning of the preceding program (PRG1).

LOW

File.SRG [X] File.LIB [X]

FUNCTION

Programs the low measurement threshold. Below this threshold the read value is considered as incorrect.

SYNTAX

LOW=val

val: value of the low threshold that can be:

if **MMD=V1/V4/D/C** -51.2V ≤ val ≤ +51.2V
if **MMD=N/R** 0X0000 ≤ val ≤ 0XFFFF

DEFAULT VALUE

LOW=0

SEE ALSO

HIG, **MES**, **VIW**

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

EXAMPLE

LAB="V-MEAS",MES=5, HIG=5.5,LOW=4.5, DRI=80.000000, MMD=V4,LI4=18;
* Voltage measurement, the low threshold is equal to +4.5V

LAB="IMP-MEAS",MES=100, HIG=105,LOW=95, MMD=N,LN3=2,LI3=47,LI4=47,NCK=100;
*Measurement of the pulses generated by the clock generator. The low threshold for this test is 95 pulses.

MAX

File.SRG [X] File.LIB [X]

FUNCTION

This command is used with the FAR=... command in the @DEF macro to define the parameters for the @OPICN and @OPICP macros. These two macros verify the correct solder joints of the component pin on the PCB. The MAX command defines the max number of pin pairs that the macro @OPICN e @OPICP can form for each pin. The FAR command sets the max distance between the two chosen pins to form the pair.

SYNTAX

MAX=val

val: Indicates the max number of pairs in the @OPICN and @OPICP macros.

$0 \leq val \leq 65535$

DEFAULT VALUE

MAX=0

SEE ALSO

FAR

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

EXAMPLE

@DEF (GND=7,VCC=14,MAX=22,FAR=5);
@OPICN(“IC1”,1,2,3,4,5,6,7,8,9,10,11,12,13,14);
Macro OPICN carries out the test on the open pins of component IC1. The component has an "n" type sub-layer. The macro tries to form 22 pairs of pins for each pin of the component using a distance of 5 pins

MES

File.SRG [X] File.LIB [X]

FUNCTION

Indicates the expected value of the measurement.

SYNTAX

MES=val

val: Expected value that can be:

if **MMD=V1/V4/D/C**	-50V < val < +50V
if **MMD=N/R**	0X0000 ≤ val ≤ 0XFFFF

DEFAULT VALUE

MES=0

SEE ALSO

HIG, **LOW**, **VIW**

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.
On the basis of the value indicated by the MES command, the voltage measurement range is automatically selected from the following:

0V ≤ val ≤ 2V	steps of	1mV
2V ≤ val ≤ 10V	steps of	5mV
10V ≤ val ≤ 50V	steps of	25mV

The value indicated in the MES command is used by the system to calculate the percentage of deviation of the measured value with respect to the expected value.

If MMD=R (reading from the external busses EB1, EB2) the value indicated in the MES command is logically multiplied with the value red by the system. (AND operation). In other words, the measured value is masked.

EXAMPLE

LAB="WRONG P. SUPPLY",MES=12,
LOW=11,HIG=13,LI4=129,BER="M_7";
The expected measurement value is +12V

LAB=“READING”,EB1=0X000E,MMD=R,MES=0X0FFF;”
External bus reading: in this case bit 12, 13, 14 and 15 are masked because a logic AND between the read value and the one indicated by MES is executed

MMD

File.SRG [X] File.LIB [X]

FUNCTION

Programs the type of measurement to execute.

SYNTAX

MMD=mode

mode: Letter that indicates the type of measurement. It can be:

- **V1** voltage measurement on line 1.
- **V4** voltage measurement on line 4.
- **N** pulse measurement.
- **R** read of data on the external buses EB1 and EB2.
- **C** low value capacitor measurement. For the S20 series test systems a value of less than 40nF is usually considered as a low value capacitor measurement.
- **D** four-wire resistor measurement.
- **U** used together with the WAI=.... command, it permits a read on the eight optocoupled inputs or the data bus of the USB board. The read operations are executed as described below.

With MMD=U and	Is executed the...
WAI=1	USB nr. 1 read of the optocoupled inputs
WAI=2	USB nr. 1 read of the data bus
WAI=3	USB nr. 2 read of the optocoupled inputs
WAI=4	USB nr. 2 read of the data bus
WAI=5	USB nr. 3 read of the optocoupled inputs
WAI=6	USB nr. 3 read of the data bus
WAI=7	USB nr. 4 read of the optocoupled inputs
WAI=8	USB nr. 4 read of the data bus

DEFAULT VALUE

MMD=V4

SEE ALSO

EB1, **EB2**, **HIG**, **LN4** (=4), **LOW**, **MES**, **WAI**

EXAMPLE

LAB="K15_ON",DRV=5,DRI=200,DRM=V,TIM=100,**MMD=V1**,LI2=2-99-104,LI1=109,MES=0,
LOW=-0.5,HIG=1;
Voltage measurement on line 1

LAB="DISCHARGE_B CHECK",**MMD=V4**,LN4=0,LI4=2,LN2=0,LI2=3,TIM=10,NRC=4095,
MRC=J,LOW=0,MES=0.3, HIG=0.5,BER="FINE";
Voltage measurement on line 4

LAB="TXUS",TIM=100,**MMD=N**,HLD=Y,DR2=5,D2P=N,LN3=1,LI3=158,LI2=188,LI4=189,
PUL=10,PUM=D,NRC=10,MRC=J,MES=4000, HIG=4020,LOW=3980;
Pulse measurement

LAB=“C1”,TIM=.2,NRC=10,MRC=M,**MMD=C**,MES=0.5,LOW=0.45,HIG=0.55,LI1=45,LI2=78;
Low value capacitor resistor

#RES("T1 RANGE 3 RX",604,1,166,191),TIM=10,**MMD=D**,SCR="\CODICE --> 47",BOK= "END TEST RX",NER=Y;
Four-wire resistor measurement

LAB=“M1”,**MMD=U**,**WAI=2**;
Read on the data bus of USB nr 1

MOV

File.SRG [X] File.LIB [X] Only for Flying Probe systems [X]

FUNCTION

This command is used only on Flying Probe test systems. During the optimization procedure, the order of the tests is changed to reduce the probe movement time. If the MOV=N command is used in a test, its position will not be changed.

SYNTAX

MOV=Y/N

Y: the order of the test can be changed by the optimization procedure.
N: the order of the test cannot be changed by the optimization procedure.

DEFAULT VALUE

MOV=Y

NOTE

This command is mainly used in the automatic path optimization process during the test. (FLYING PROBES\Optimization menu).

EXAMPLE

```
#CAP("C14",100,50,20,87,66);
#RES("R1",4700,10,34,56),MOV=N;
#RES("R2",1000,10,57,89),MOV=N;
#CAP("C17",100,50,20,88,66);
*The order of R1 and R2 tests will not be changed during optimization*
```

MRC

File.SRG [X] File.LIB [X]

FUNCTION

Defines the test iteration mode.

SYNTAX

MRC=mode

mode: Letters that identify the iteration mode. These be:

- **M** the result of the measurements is stored at every recycle. The iteration stops when the programmed number of recycles has been reached, and the average value of the results is displayed.
- **W** the iteration stops when the programmed number of recycles has been reached, and only at this point the system executes the measurement.
- **J** the iteration stops when the test has given a positive result.
- **H** the iteration stops when the programmed number of recycles has been reached and the highest measurement value is stored.
- **L** the iteration stops when the programmed number of recycles has been reached and the lowest measurement value is stored.
- **P** the iteration stops when the programmed number of recycles has been reached and the difference between the maximum and minimum value of the measurement is stored.
- **Z** defines the starting point of a recycle executed on more than one test (the ending point of the recycle is defined by the MMD=R command). Refer to EXAMPLE 7.
- **R** defines the ending point of the recycle executed on more than one test (the starting point of the recycle is defined by the MMD=Z command). Refer to EXAMPLE 7.

The MRC=Z and MRC=R commands are used to solve any critical conditions in the circuit to be tested, in fact they execute a recycle on a certain number of failing tests.

N.B. The MRC=Z and MRC=R commands must always be used together otherwise the result of the program may not be real.

DEFAULT VALUE

MRC=M

SEE ALSO

NRC

EXAMPLE

EXAMPLE 1
#RESV("RR4_4",0.3,4700,5,157,131),NRC=200,MRC=W,TIM=1;
The iteration stops after 200 recycles and only at that point, the measurement is executed.

EXAMPLE 2
#RES("R22",10000,5,39,63),GUA=40, TIM=2,NRC=20,**MRC=J**;
The iteration stops when the test gives a positive result

EXAMPLE 3
LAB="V MAX TS31",LI1=86,LI2=11,LI4=23,LOW=9,MES=9.5
,HIG=10,TIM=0.1,**NRC=100**,**MRC=H**;

The iteration stops after 100 recycles and the highest read value is stored

EXAMPLE 4
LAB="Min. Voltage",LI1=86,LI2=11,LI4=2,LOW=1,MES=1.1,HIG=1.2,TIM=0.1,**NRC=100**,**MRC=L**;
The iteration stops after 100 recycles and the lowest read value is stored

EXAMPLE 5
LAB="Average Voltage",LI1=86,LI2=11,LI4=2,LOW=3,MES=4,HIG=5,TIM=0.1,**NRC=100**,**MRC=M**;
The iteration stops after 100 recycles and the average of the read values is displayed

EXAMPLE 6
LAB="P1",**MRC=P**,LN4=1,MMD=V4,MES=20, LOW=19,HIG=21;
Peak-to-peak measurement

EXAMPLE 7
#74(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14);
@C (5,0,3.5,1.3,1,1,P,C);
NER=Y,**MRC=Z**,LAB=COMM+" 1";
NER=Y,MRC=M,LAB=COMM+" 2";
NER=N;
@L (1.H.....);
@L (0.L.....);
NER=Y,**MRC=R**,NRC=10,JER=COMM+" 2",LAB=COMM+" 3";
* A recycle on the two @L macros is executed until they both give a positive result or until the max. recycle number is reached (10). If the two @L macros pass, no recycle will be executed.
MRC=Z and MRC=R actually define the beginning and the end of the loop, although the loop is carried out only if the execution of one of the two @L macros has set a global error flag. The two commands MRC=Z and MRC=R are used to define the block of the commands delimited as single test. The error flag for the definition of PRPGRAM PASS or PROGRAM FAIL is significant only when exiting from the block, if the exit occurs due to the lack of a maximum number of recycles.*

NCK

File.SRG [X] File.LIB [X]

FUNCTION

Programs the number of clock pulses supplied by the Clock generator.

SYNTAX

NCK=val

val: indicates the number of clock pulses. The value can be: 0 ≤ val ≤ 32767

DEFAULT VALUE

NCK=0

SEE ALSO

CKN, **LN3** (=2)

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

EXAMPLE

LAB="TRESH CLK",**NCK=10**,LN3=2,LI3=4097,LI4=4097,CKN=N,MES=0,HIG=2,LOW=-2,
MMD=N,DR2=-2;
This label generates 10 clock pulses

NER

File.SRG X File.LIB X

FUNCTION

Masks the partial error flag that may occur in the present test and disables the activation of the global error flag. The NER=Z command also prevents the current test from being executed. Refer to the NOTE section, for an application of this command.

SYNTAX

NER=Y/N/Z

Y: masks the error in the current test.
N: command disabled.
Z: the test in which this command is written will not be executed.

DEFAULT VALUE

NER=N

NOTE

One application of the NER=Z command is when we have to test two diodes connected in anti-parallel. With the DIO macro, we can't execute the inverse voltage test on the diode junction because one diode is always in conduction. The problem is solved writing the following lines:

```
#DIO("D1",23,26),LAB="INV",NER=Z;
#DIO("D2",23,26),LAB="INV",NER=Z;
```

In this case, the inverse voltage tests on the diode junctions aren't executed.

EXAMPLE

```
LAB="M_1",NER=Y,SCR="contact and short test...";
*It masks the error flag for the current test because in that only a message is displayed*

LAB="POWER_ON",PW2=15,NER=Y,WAI=500;
* It masks the error flag for the current test
```

NRC

File.SRG [X] File.LIB [X]

FUNCTION

Programs the number of recycles for the current test.

SYNTAX

NRC=val

val: Number of recycles for the current test. It can be: 0 ≤ val ≤ 65535

DEFAULT VALUE

NRC=0

SEE ALSO

MRC

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

The NRC command must be used with the MRC command, which defines the recycle mode of test.

EXAMPLE

```
#RES("R10",560,5,173,140),TIM=20,NRC=200,MRC=J;
*It executes 200 recycles on the current test*
```

ONE

File.SRG [X] File.LIB [ ]

FUNCTION

Enables a digital guarding at a high threshold, on the specified channels.

SYNTAX

ONE=channel

channel: channel/s on which the digital guarding is applied. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

SEE ALSO

ZER

NOTE

Driver 2 supplies the high threshold of the digital guarding and the level of programming is the one indicated in the @C macro written in the digital library of the component.

EXAMPLE

There are two buffers connected on the same bus. During the U1 test, the U2 component is disabled applying the digital guarding on channels 21 and 29. Vice-versa for the U2 test applying the digital guarding on channels 1 and 19

#74244("U1",1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20),ONE=21,29;

#74244("U2",21,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,29,20),ONE=1,19;

OPC

File.SRG [X] File.LIB [X]

FUNCTION

Drives the open collector signals (OC0-0C7) on the service bus of the LAM module.

SYNTAX

OPC=val

val: Value which indicates the state of the open collector signals: 0X00 ≤ val ≤ 0XFF

The open collector signal is at 0 when the relative bit is at 0.

The open collector signal is float when the relative bit is at 1.

The least significant bit of "val" (LSB) corresponds to the OC0 signal while the most significant bit (MSB) corresponds to the OC7 signal.

DEFAULT VALUE

OPC=0XFF

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the "0X".

EXAMPLE

LAB="TEST1",NER=Y,OPC=0X0F;
The OC4,OC5,OC6,OC7 signal are at zero while the OC0,OC1,OC2,OC3, signals are float or at a high level, if there is a pull-up resistor

POL

File.SRG [X] File.LIB [X]

FUNCTION

Displays the network information instead of the channel number, for the current test.

SYNTAX

POL=Y/N

Y: displays the network information if the current test fails.
N: command disabled.

DEFAULT VALUE

POL=N

SEE ALSO

The Show network option in the Board.CFG dialog. The dialog is opened selecting the BOARD\Configure menu.

EXAMPLE

```
#RES("R1",1000,10,23,24);
#RES("R2",4700,10,25,24),POL=Y;
```

POP

File.SRG [X] File.LIB [X]

FUNCTION

Executes arithmetical operations between the read value and the data previously stored in the temporary variable called "stack". The operation result will substitute the value read by the system. For the arithmetical operations, the data present in the stack is considered as the first operand while the read value as the second.

SYNTAX

POP = operation

operation: Indicates the operation to be executed and can be:

+ Addition
- Subtraction
* Multiplication
/ Division
! Only the whole part of the number is considered
^ Square root
N No operation

VALORI DI DEFAULT

POP=N

SEE ALSO

FRC, **PSH**

NOTE

The FRC, POP and PSH commands belong to the same family, since they allow the execution of several tests, arithmetically processing the results in order to obtain the one desired. These commands can be used independently of each other; if more than one is used in the same test, they will be executed according to the following priority:

FRC
POP
PSH

EXAMPLE

```
#DIODIR("c-1 P5-T1_SUM",1,5),POP=+, PSH=Y,NER=Y;
*It sums the measurement result to the data in the stack*
```

PRN

File.SRG X File.LIB X

FUNCTION

Prints the information displayed in the “Test reports” dialog (of the Test environment) for the current test. The information printed is the test name, the error percentage, and the measurement value.

SYNTAX

PRN=Y

Y: enables the printout of the test result.
N: command disabled

DEFAULT VALUE

PRN=N

NOTE

The format of the information displayed in the “Test reports” dialog can be modified pressing the button and subsequently the “Others” button. In the “Others Test Settings” dialog thus opened, it is possible to choose the required option in the “Report Type” section

EXAMPLE

#RESV(“R42”,0.3,4700,5,215,130),PRN=Y;
*If the “Small” option in the “Others Test Settings” dialog has been chosen, the following information would be displayed:

+100.0 % R42

(the error percentage and the test name)

PSH

File.SRG [X] File.LIB [X]

FUNCTION

Stores the value read by the system during the measurement in the temporary variable called “stack.”. The stack has got room to store up to 20 positions.

SYNTAX

PSH=Y/N

Y: stores the value in the stack
N: command disabled

DEFAULT VALUE

PSH=N

SEE ALSO

FRC, **POP**

NOTE

The FRC, POP and PSH commands belong to the same family, since they allow the execution of several tests, arithmetically processing the results in order to obtain the one desired. These commands can be used independently of each other; if more than one is used in the same test, they will be executed according to the following priority:

FRC
POP
PSH

EXAMPLE

```
#DIODIR("c-1 P5-PT3_CALCULE",1,6),
PSH=Y,DRI=6,LOW=-0.1,MES=0,HIG=0.1;
*The value read during the measurement is stored in the stack*
```

PUL

File.SRG [X] File.LIB [X]

FUNCTION

Programs the value of the pull-up or pull-down resistor on the line 4.

SYNTAX

PUL=val

val: value, expressed in KΩ of the pull-up o pull-down resistor. The value can be:

0.1KΩ	≤ val ≤ 1.5KΩ	steps of 100Ω
1KΩ	≤ val ≤ 15KΩ	steps of 1KΩ
10KΩ	≤ val ≤ 150KΩ	steps of 10KΩ
100KΩ	≤ val ≤ 1500KΩ	steps of 100KΩ

DEFAULT VALUE

PUL=1

SEE ALSO

PUM

NOTE

N.B. The unit of measurement can be omitted because it is considered by default.

The PUM command used with the current command, specifies if the resistor is an internal or external pull-up/pull-down.

if **PUM=N**	is not considered.
if **PUM=U**	indicates the value of the internal pull-up.
if **PUM=D**	indicates the value of the internal pull-down.
if **PUM=E**	indicates the value of the external pull-up.

EXAMPLE

LAB="TX US",TIM=100,DR2=5, D2P=N,LI4=189,**PUL=10**, **PUM=D**;
Pull-down resistor of 10KΩ on line 4

PUM

File.SRG [X] File.LIB [X]

FUNCTION

Defines the way of connecting the pull-up or pull-down resistor.

SYNTAX

PUM=mode

mode: letters that indicate the connection of the pull-up resistor. These can be:

N the resistor is not connected.
U pull-up resistor connected to +5V.
D pull-down resistor connected to ground.
E resistor connected to the EXPUL signal of the service bus.

DEFAULT VALUE

PUM=N

SEE ALSO

PUL

NOTE

The resistor value is defined using the PUL command.

EXAMPLE

LAB="GATES FREQUENCY ",LI4=18,**PUL=1**,**PUM=U**;
Pull-up resistor of 1KΩ on line 4

LAB=" GATES FREQUENCY 1",LI4=18,**PUL=200**,**PUM=D**;
Pull-down resistor of 200Ω on line 4

LAB="PUM=E",**PUM=E**;
External resistor connected to the EXPUL signal of the service bus

PW1, PW2, PW3, PW4, PW5, PW6

File.SRG [X] File.LIB [X]

FUNCTION

Programs the voltage value required on user power supply nr.: 1, 2, 3, 4, 5, and 6.

SYNTAX

PW1=val; **PW2**=val; **PW3**=val; **PW4**=val; **PW5**=val; **PW6**;

val: voltage value, expressed in absolute value: Vmin ≤ val ≤ Vmax

The value of Vmin, Vmax and the resolution depend on the type of power supply present in the system.
The unit of measurement (Volt) must be omitted because it is considered by default

DEFAULT VALUE

PW1=-1; **PW2**=-1; **PW3**=-1; **PW4**=-1; **PW5**=-1; **PW6**=-1;

PW1= -1, unless there is a different value in the User Power Setting. The User Power Setting can be opened by selecting the UTILITIES\Configuration menu, selecting the User Power field and pressing the ... button.

NOTE

CAREFUL!!: Once the power supply has been programmed in a test, it will remain on also during the subsequent tests until it is disabled.

EXAMPLE

```
LAB="POWER ON",PW1=5,NER=Y,WAI=200;
*Power supply nr. 1 programmed at +5V*

LAB="POWER ON",NER=Y,WAI=500,PW2=12;
*Power supply nr. 2 programmed at +12V*

LAB="POWER ON",TIM=200,WAI=500,NER=Y, PW2=15,PW3=15;
*Power supplies nr. 2 and 3 programmed at +15V*

LAB="POWER ON",PW4=1,WAI=500,NER=Y;
*Power supply nr. 4 programmed at 1V*

LAB="POWER ON",PW5=12,NER=Y,WAI=200;
*Power supply nr. 5 programmed at +12V*

LAB="POWER ON",PW6=5,NER=Y,WAI=200;
*Power supply nr. 6 programmed at +5V*
```

RET

File.SRG [X] File.LIB [X]

FUNCTION

It defines the end of a subroutine (specified with the SUB command).

SYNTAX

RET=Y/N

Y: it ends the subroutine
N: command not active

SEE ALSO

SUB, **SER**, **SOK**

NOTE

The subroutine can be written in any part of the source program or library file.

When the subroutine ends, the program execution will go on from the label after where there was the subroutine call.

This command, when used in the library environment, is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

EXAMPLE

* The subroutine SUBR1 is called, after the test of R1. At the end of the subroutine, the execution goes on, performing the test of R3*

```
#RES (“R1”,330,10,1,2);
LAB=“LL1”,SUB=“SUBR1”,NER=Y;
#RES (“R3”,4700,10,5,6);
...
...
LAB=“LL2”,JUM=“END”,NER=Y;

LAB=“SUBR1”;
...
...
LAB=“LL3”,RET=Y,NER=Y;

LAB=“END”,NER=Y;
```

SCA

File.SRG [X] File.LIB [X] Only for Flying Probe systems [X]

FUNCTION

Used for Flying Probe systems only. It defines the measure scale (high/low) of the FEMTO board.

SYNTAX

SCA=H/L

H: capacitance range: from 1fF to 1nf, resolution 6.25fF working voltage 1V/5V.
L: capacitance range: from 1fF to 20nf, resolution 125fF working voltage 50mV/250mV.

DEFAULT VALUE

SCA=L

EXAMPLE

@CON ("CAP 825",0,25,825,GND),SCA=H;

SCR

File.SRG [X] File.LIB [X]

FUNCTION

Displays a message on the screen.

SYNTAX

SCR="string"

string: string to display of maximum 30 characters.

The string can be written on maximum three lines using the "\" as a line delimiter.

Three special characters can be used in the string with the following meaning:

@ If the string begins with this character, the message is sent to the printer.

%D If this character is present in the line, it is translated into the current date on the screen/printer.

%T If this character is present in the line, it is translated into the current time on the screen/printer.

%B If this character is present in the line, it is translated into the value of the barcode on the screen/printer.

%S01...%S15 If this characters are present in the line, the string system variable indicated is displayed/printed.

%N01...%N15 If this characters are present in the line, the floating point system variable indicated is displayed/printed.

SEE ALSO

STP, **VIW**

NOTE

This command, when used in the library environment, is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).
The LNK command is not compatible with the SRC command. If both are used in the same test, only the last one will be considered.

EXAMPLE

LAB="M_1",NER=Y,SCR=" Contact and short test...";
* The M_1 test will display the following message:

Contact and short test …

LAB=" PRINT-NAME ",NER=Y,SCR="@ Board:BBU-REP %D %T";
* If we suppose that the current date is 15/05/97 and the time is 9.45, the label PRINT-NAME will display the following message:

Board: BBU-REP 15/05/97 9.45

SER

File.SRG [X] File.LIB [X]

FUNCTION

It will call the specified subroutine if an error has occurred in the current label (if the partial error flag is active).

SYNTAX

SER=“subroutine name”

“subroutine name”: name of the subroutine to call. The name can be up to 30 characters.

SEE ALSO

SUB, **RET**, **SOK**

NOTE

When the subroutine ends, the program execution will go on from the label after where there was the subroutine call.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

This command, when used in the library environment, is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

EXAMPLE

If an error has occurred during the test of R2, the subroutine SUBR1 will be called. At the end of the subroutine, the execution goes on, performing the test of R3

```
#RES (“R1”,330,10,1,2);
#RES (“R2”,22,10,3,4),SER=“SUBR1”;
#RES (“R3”,4700,10,5,6);
...
...
LAB=“LL1”,JUM=“END”,NER=Y;

LAB=“SUBR1”;
...
...
LAB=“LL2”,RET=Y,NER=Y;

LAB=“END”,NER=Y;
```

SOK

File.SRG [X] File.LIB [X]

FUNCTION

It will call the specified subroutine, if the current test hasn't failed (the partial error flag is not active).

SYNTAX

SOK="subroutine name"

"subroutine name": name of the subroutine to call. The name can be up to 30 characters.

SEE ALSO

SUB, **RET**, **SER**

NOTE

When the subroutine ends, the program execution will go on from the label after where there was the subroutine call.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

This command, when used in the library environment, is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

EXAMPLE

If an error hasn't occurred during the test of R2, the subroutine SUBR1 will be called. At the end of the subroutine, the execution goes on, performing the test of R3

```
#RES ("R1",330,10,1,2);
#RES ("R2",22,10,3,4),SOK="SUBR1";
#RES ("R3",4700,10,5,6);
...
...
LAB="LL1",JUM="END",NER=Y;

LAB="SUBR1";
...
...
LAB="LL2",RET=Y,NER=Y;

LAB="END",NER=Y;
```

STP

File.SRG [X] File.LIB [X]

FUNCTION

Temporarily interrupts the program being executed. The program execution resumes when the user presses the START button on the fixture or the space bar on the keyboard.

SYNTAX

STP=Y/N

Y: temporarily interrupts the program
N: command disabled

DEFAULT VALUE

STP=Y

SEE ALSO

SCR, **VIW**

EXAMPLE

LAB="PIASTRA-3 ATTENDE",STP=Y,SCR="\\PREMI LA BARRA PER CONTINUARE",NER=Y;
* It displays the message and waits for the operator to press the space bar (or to press the START button on the fixture)*

SUB

File.SRG [X] File.LIB [X]

FUNCTION

It calls the specified subroutine.

SYNTAX

SUB="subroutine name"

"subroutine name": name of the subroutine to call. The name can be up to 30 characters.

SEE ALSO

RET, **SER**, **SOK**

NOTE

The subroutine can be written in any part of the source file or library file.

When the subroutine ends, the program execution will go on from the label after where there was the subroutine call.

The name used as the command argument must always be in capital letters even if the destination label uses small letters.

This command, when used in the library environment, is not maintained in the subsequent tests within the same macro (it is not considered by the COPY mode).

EXAMPLE

After the test of R2, the subroutine SUBR1 is called. At the end of the subroutine, the execution goes on, performing the test of R3

```
#RES ("R1",330,10,1,2);
LAB="LL1",SUB="SUBR1",NER=Y;
#RES ("R3",4700,10,5,6);
...
LAB="LL2",JUM="END",NER=Y;

LAB="SUBR1";
...
LAB="LL3",RET=Y,NER=Y;

LAB="END",NER=Y;
```

SYN

File.SRG [X] File.LIB [X]

FUNCTION

Allows the synchronization of the voltage used to perform the electric isolation test, with the system power supply voltage (50Hz or 60Hz)

SYNTAX

SYN=Y/N

Y: it synchronizes the test voltage with the system power supply voltage.
N: command disabled.

DEFAULT VALUE

SYN=N

SEE ALSO

@ISOHV, **@RESHV**

EXAMPLE

```
@ISOHV("SHORT 253-252",100,1000000,253,252),
SYN=Y;
```

TIM

File.SRG [X] File.LIB [X]

FUNCTION

Programs the test duration or the period of the clock signal supplied by the Clock generator.

SYNTAX

TIM=val

val: value, expressed in ms, which indicates the test time or the clock period: 0.005 ≤ val ≤ 60.000

if **NCK=0**
then "val" indicates the time that lapses between the start of the test and the instant of measurement (test time).

if **NCK>0**
then "val" indicates the clock period. In this case the test time is: T=NCK*TIM+1/4TIM

DEFAULT VALUE

TIM=1

SEE ALSO

NCK

NOTE

N.B. The unit of measurement must be omitted because it is considered by default.

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the "0X".

EXAMPLE

LAB="BOARD-1 CLOSE RL1",DRM=V,DRV=9, DRI=200, **TIM=10**,LI1=5,LI2=32,NER=Y;
Test time programmed at 10ms

LAB="NCK=4000",NCK=4000,LN3=2,LI3=4097,LI4=4097,CKN=N,MES=4000,HIG=4000,
LOW=4000,MMD=N,**TIM=.01**,DR2=5;
Clock period programmed at 0.01ms

UPG

File.SRG [X] File.LIB [X] Only for Flying Probe systems [X]

FUNCTION

It permits the programmer to define the way to connect the system ground to the channel connected on LINE 2 in the current test.

SYNTAX

UPG=Y/N

Y: It connects the channel present on LINE 2 in the current test, to the system ground through the ACTIVE PROBE board mounted on the test heads. In this configuration is suitable to measure resistors with value greater the 100KΩ since the system noise is reduced to the minimum.

N: It connects the channel present on LINE 2 in the current test, to the system ground through the channel board (SCAFP) and the instrument module (LAM).

DEFAULT VALUE

UPG=N

EXAMPLE

#RES(“R1”,1000000,10,41,5),UPG=Y;

VCC

File.SRG [X] File.LIB [X]

FUNCTION

Specifies the channel/s connected to power in the board to be tested. This definition is subsequently used in the macro calls and in the automatic debug procedures.

SYNTAX

VCC= channel

channel: channel or channels connected to power. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

SEE ALSO

GND

NOTE

The definition of the channels connected to ground and power must be specified with the @DEF macro before using the @OPICN or @OPICP macros. Refer to the example illustrated below.

EXAMPLE

@DEF (GND=7,VCC=14);
@OPICN("IC1",1,2,3,4,5,6,GND,8,9,10,11,12,13,VCC);
* After the execution of the @DEF macro, the 7 and 14 channels are considered as connected to ground and power respectively.*

VIM

File.SRG [X] File.LIB [X]

FUNCTION

Permits to choose the unit of measurement of the value read by the system.

SYNTAX

VIM=mode

mode: Letter that identifies the unit of measurement. It can be:

A Ampere. Refer to the NOTE section.
F Farad
L Henry
O Ohm
T Second
V Volt
H Hertz

DEFAULT VALUE

VIM=V

SEE ALSO

MMD, **VIW**

NOTE

If the VIM=A and PUM=E command are used in the same test, the read value will be considered in Ampere.
If only command VIM=A is used, the current gain is displayed, which is calculated with the following formula:

(Vdr2 * Val/Rpul)/Idr1

Where Vdr2 = Driver 2 voltage, Val = reading value, Idr1 = Driver 1 current, Rpul = pull-up resistance value, equivalent to 0.1KΩ

The DRI=0 and VIM=O are incompatible if used together in the same test because it is not possible to measure a resistor value with no current.
It is important to remember that the resistance measurement (VIM=O) is carried out with DRI current generator that supplies a continuous current. However, this value must differ from zero. The resistance is a linear function of the voltage drop at the terminations when current is constant.

EXAMPLE

#RES("R1",2700,10,34,35),VIW=Y,VIM=V;
The read value is expressed in Volts. The value will be displayed in the "Result" section of the Test environment

VIW

File.SRG [X] File.LIB [X]

FUNCTION

Displays the current test parameters in the “Result” section of the Test environment. The test parameters are the measurement value and range, the read value and the error percentage.

SYNTAX

VIW=Y/N

Y: displays the test parameters
N: command disabled

DEFAULT VALUE

VIW=N

SEE ALSO

VIM

EXAMPLE

LAB="POW1",LI4=1,TIM=1000,LOW=4.9, MES=5.0,HIG=5.1,VIW=Y;
The POW1 test parameters will be displayed in the “Result” section of the test environment

WAI

File.SRG [X] File.LIB [X]

FUNCTION

This command has got three different meanings:

1. It programs the wait time between the instant of measurement and the beginning of the successive.
2. Used with the MMD=U command, it executes a read operation from the optocoupled inputs or the data bus of the USB board. Refer to EXAMPLE 2.
3. Used with the WRT=... command in the @DEF macro, it defines the parameters of the @OPICN and @OPICP macros which verify the correct soldering of the component on the PCB. The WRT command defines the maximum number of pin pairs that the @OPICN and @OPICP macros can form for each pin to check. The WAI command defines the maximum distance between the two pins chosen to form the pair. Refer to EXAMPLE 3

SYNTAX

WAI=val

val (meaning 1): value of the wait time considered in ms: $0 \leq val \leq 65535$

val (meaning 2): a value in a range from 1 to 8 specifies both the USB board number, and the acquirement source as the following table:

With MMD=U and	It will do...
WAI=1	USB nr. 1 read of the optocoupled inputs
WAI=2	USB nr. 1 read of the data bus
WAI=3	USB nr. 2 read of the optocoupled inputs
WAI=4	USB nr. 2 read of the data bus
WAI=5	USB nr. 3 read of the optocoupled inputs
WAI=6	USB nr. 3 read of the data bus
WAI=7	USB nr. 4 read of the optocoupled inputs
WAI=8	USB nr. 4 read of the data bus

val (meaning 3): Used with the WRT=... command in the @DEF macro, it defines the parameters of the @OPICN and @OPICP macros which verify the correct soldering of the component on the PCB. The WRT command defines the maximum number of pin pairs that the @OPICN and @OPICP macros can form for each pin to check. The WAI command defines the maximum distance between the two pins chosen to form the pair.

DEFAULT VALUE

WAI=0

SEE ALSO

MMD (=U)

NOTE

N.B. The unit of measurement must be omitted because it is considered by default.

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the “0X”.

EXAMPLE

EXAMPLE 1
WAI command usage with the first meaning that is: between the first and the second test there is a wait time equal to 500ms
LAB="ALI_ON",NER=Y,**WAI=500**,PW2=12;
LAB="WRONG POWER ON",MES=12, LOW=11,HIG=13,LI4=129,BER="M_7";

EXAMPLE 2
WAI command usage with the second meaning that is: a read operation is executed on the data bus of the USB nr. 1
LAB="M1",**MMD=U**,**WAI=2**;

EXAMPLE 3
WAI command usage with the third meaning that is: It defines the maximum distance between the two pins chosen to form the couple(WAI=13) for the IC1 component checked using the @OPICN macro.
@DEF (GND=7,VCC=14,WRT=22,**WAI=13**);
@OPINC("IC1",1,2,3,4,5,6,7,8,9,10,11,12,13,14);

WRT

File.SRG [X] File.LIB [X]

FUNCTION

This command has got two different meanings:

1. It executes a write operation on the external buses EB1, EB2.
2. Used with the WRT=... command in the @DEF macro, it defines the parameters of the @OPICN and @OPICP macros which verify the correct soldering of the component on the PCB. The WRT command defines the maximum number of pin pairs that the @OPICN and @OPICP macros can form for each pin to check. The WAI command defines the maximum distance between the two pins chosen to form the pair. Refer to EXAMPLE 2.

SYNTAX

WRT=val

val (meaning 1): Value which indicates the data to write on the bus: 0 ≤ val ≤ 65535. The least significant bit of "val" (LSB), corresponds to line EB1-0 or EB2-0 while the most significant bit of "val" (MSB) corresponds to line EB1-15 or EB2-15.

val (meaning 2): the maximum number of pin pairs that the @OPICN and @OPICP macros can form: 0 ≤ val ≤ 65535

DEFAULT VALUE

WRT=0X0000

SEE ALSO

EB1, **EB2**, **LN4** (=4), **MMD** (=R)

NOTE

The value of the present command can be expressed as a whole decimal or as a hexadecimal. In the first case no prefix is used while in the second, the value must be preceded by the "0X".

EXAMPLE

EXAMPLE 1
WRT usage example, according to the first meaning of the command that is: a write operation of data 0X5 on the external bus EB1 is executed

LAB="WRITE",**EB1=0XFFF0**,**WRT=0X5**;

EXAMPLE 2
WRT usage example, according to the second meaning of the command that is: the definition of the maximum number of pairs (WRT=40) to form for IC1 component checked using the @OPICN macro

@DEF (GND=7,VCC=14,**WRT=40**,**WAI=13**);
@OPINC("IC1",1,2,3,4,5,6,7,8,9,10,11,12,13,14);

ZER

File.SRG [X] File.LIB [ ]

FUNCTION

Permits to execute a digital guarding at a low threshold, on the specified channels.

SYNTAX

ZER=channel

channel: channel/s on which the digital guarding is applied. The "/" character is used to indicate a group of contiguous channels and the "," or the "-" character is used to indicate non-contiguous channels.

SEE ALSO

ONE

NOTE

Driver 1 supplies the low threshold of the digital guarding and the level of programming is the one indicated in the @C macro, written in the digital library of the component.

EXAMPLE

#4049("CD4049",109,110,109,110,110,111,,117,114,117,115,,112,113,),ZER=71-12;

4.3 Macros

A group of instructions and commands that are repeatedly used for example to test a component or the check the fixture, can be defined as a macro. In this way it is possible to create a source program without having to define each time the different tests but simply calling a macro.
There are different types of macros in the in-circuit programming language. The section below provides a list of the available macros, classified according to the environment of use:

- System macros written in the source program (.srg File)

  @AURES
  @AUTIC
  @CASE
  @CLUSTER
  @CON
  @CONT
  @CTO
  @FIX
  @ISO
  @ISOV
  @OPICN/P
  @PAR
  @PINAD
  @RESV
  @SHORT
  @SHOW
  @VIS
  @Z
  @ZENHV

- System macro in the library file:

  @C
  @IEEE
  @L
  @PAT

- System macros written in both files (.srg File and .lib File):

  @APPEND
  @CALC
  @COMPARE
  @CONV
  @COPY
  @DEF
  @INPUT
  @OUTPUT
  @SUBCOMP

- User macros:

  #
  &
  %

4.3.1 System macro in the File.srg:

@AURES
@AUTIC
@CASE
@CLUSTER
@CON
@CONT
@CTO
@FIX
@ISO
@ISOV
@OPICN/P
@PAR
@PINAD
@RESV
@SHORT
@SHOW
@VIS
@Z
@ZENHV

@AURES

File.SRG [X] File.LIB [ ]

FUNCTION

This macro auto-learns the impedance values between two points:

- Auto-learnt values are used as reference values during the test.
- The two points represent the possible pairs that can be established between the channels of group 1, which are all grounded, and each channel of group 2

This macro is used to auto-learn the values from a sample board so that they can be used for the boards under test. After entering the macro in the source file, it is necessary to run command TOOLS\HARDWARE\Autolearn/Debug, using the sample board, to acquire the related values. The source file (file .srg) is automatically changed at the end of this command, so that the execution of the macro, on the boards under test, corresponds to a measurement comparable with the self-learnt value of the sample board and the value measured on the current board.

SYNTAX

@AURES (“comment”,tol, fm[all], fn)

comment: current parameter that indicates the test name (max. 30 characters).
tol: tolerance value used for the test
fm: the channels of this group are all grounded during the execution of the test, so that the resistance values between the channels of groups 2 and 1 can be self-learnt. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels.
all optional parameter that enables the comparison between the “fm” channel family and all the other system channels.
fn: The program auto-learns the resistance between each single channel of this group and the channels of group 1 connected to the ground. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels.

NOTE

The execution of this macro is useful during the autodebug process.

EXAMPLE

@AURES ("RESIST",10,142,143/176);

Macro @AURES auto-learns the resistance values between channels 142 (grounded during the test) and channels 143, 144, ….176). Auto-learnt values are later used for tests with a 10% tolerance.

For more detailed information, see the explanation provided for the example of macro @AUTIC.

@AUTIC

File.SRG [X] File.LIB [ ]

FUNCTION

This macro auto-learns the impedance values between the pins of a digital component:

- Auto-learnt values are used as reference values during the test.
- The two points represent the possible pairs that can be established between the channels of group 1, which are all grounded, and each channel of group 2

This macro is used to auto-learn the values from a sample board so that they can be used for the boards under test. After entering the macro in the source file, it is necessary to run command TOOLS\HARDWARE\Autolearn/Debug, using the sample board, to acquire the related values. The source file (file .srg) is automatically changed at the end of this command, so that the execution of the macro, on the boards under test, corresponds to a measurement comparable with the self-learnt value of the sample board and the value measured on the current board.
During the auto-debug process, this macro enables to automatically auto-learn the measured values of digital components. During the auto-adjustment process, it enables to change the limits of the measuring range to calibrate tests according to specific sets of sample boards.

SYNTAX

@AUTIC (“comment”,tol, fm, fn)

comment: current parameter that indicates the test name (max. 30 characters).
tol: tolerance value used for the test.
fm: family of channels connected to the component ground. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels.
fn: family of channels connected to the component pins to be analyzed. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels. If a channel of group 2 is associated to a pin connected to one of the listed pins, it is not possible to enter a channel number that has already been inserted. It is however possible to enter value 0 that identifies a "non auto-learnt and measured pin".

EXAMPLE

By entering the following macro in the source file:

@AUTIC ("B-1 U7230",30,335-331,421-335-354-0-98-341-327-328-0-0-0-6);
After the execution of the menu: TOOLS\HARDWARE\Autolearn\Debug, required to acquire the values, the source file will be changed as follows:

```
@AUTIC ("IC U7230",30,335-331,421-335-354-0-98-341-327-328-0-0-0-6),
LAB="-1",HIG=1.6484,LOW=0.8876,DRI=80,
LAB="-2",HIG=1.3,LOW=0.7,DRI=2,
LAB="-3",HIG=1.6146,LOW=0.8694,DRI=0.4,
LAB="-5",HIG=1.3325,LOW=0.7175,DRI=16,
LAB="-6",HIG=1.6003,LOW=0.8617,DRI=0.16,
LAB="-7",HIG=1.3702,LOW=0.7378,DRI=0.1,
LAB="-8",HIG=1.6653,LOW=0.8967,DRI=0.16,
LAB="-12",HIG=1.4625,LOW=0.7875,DRI=80;
```

LAB instructions indicate the measurements that will be performed during the testing:

- In fn, the channels corresponding to pins that don't have to be measured will be marked with value 0.
- In fm, there is only one reference point because the channels of this group are all grounded.
- The comment of instruction LAB (-1, -2, etc…) displays the pairs that can be formed between the two channel groups.
- It is useful to notice that the LAB instructions related to –4, -9, -10 and –11 are not displayed, because the channel number associated to this pin is equal to 0.

@CASE

File.SRG [X] File.LIB [ ]

FUNCTION

This macro carries out a test with a capacitive sensor to check which pins are open or incorrectly soldered on a component. The capacitive method consists in measuring the amplitude of an AC signal applied to the net connected to a pin of the component, using the capacity between a metal plate situated on the housing of the integrated circuit and the interior of the printed circuit. If the pin of the component is incorrectly soldered (faulty soldering) to the net on which the AC signal is applied, the amplitude of the signal measured will differed from the value measured on a correctly connected board.

SYNTAX

Two syntax are possible for this macro:

1st syntax: **@CASE** (“comment”,1÷16,fn);

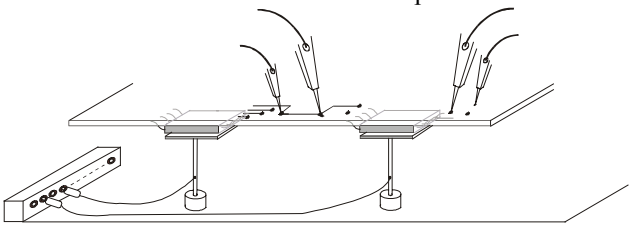
2nd syntax: **@CASE**(“comment”,0,ch1-ch11, ch 2- ch 22,...);

The first syntax is used for fixed sense while the second for movable sense.

1st syntax:

comment: comment (max. 30 characters) that indicates the component denomination.

1÷16: it indicates the number of the fixed capacitive sense used.

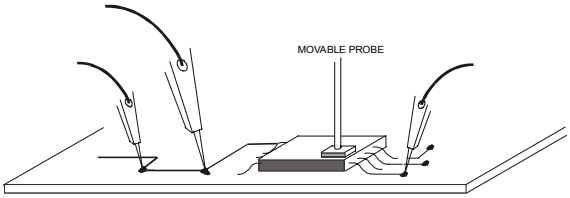


fn: family of channel connected to the component pins. The channels are separated by commas.

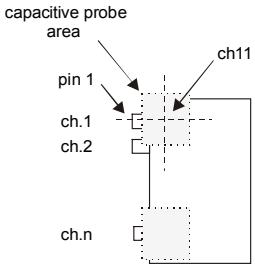
2nd syntax:

comment: comment (max. 30 characters) that indicates the component denomination.

0: it indicates that the movable capacitive sense is used.



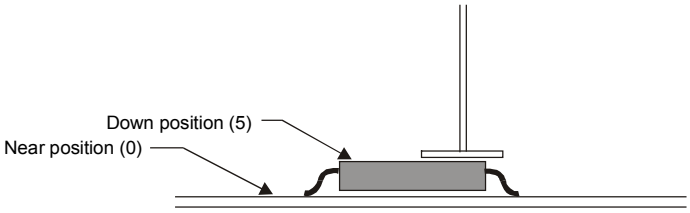
ch1-ch11,ch2-ch22,...: ch1 indicates the channel associated to the component pin. Ch11 indicates the coordinates of the movable sense center. Refer to the figure below.



NOTE

N.B. When using the movable sense, before the @CASE macro line, there must be a @Z macro that defines the parameters on the Z axis for the component under test. In the @Z macro, the DOWN parameter is used to define the height of the component.

@Z (“U1DEF”,20,0,**5**,10,12-14-15-19-99-94-23-44);
@CASE (“U1”,12,14,15,19,99,94,23,44);



To switch from a mobile capacitive sensor to a fixed one, it is possible to align the software, after appropriately changing the hardware, by entering the number of the fixed channel to be used instead of value 0 in the second statement of macro @CASE. This enables to replace the second syntax of macro @CASE (mobile sensor) with the first one (fixed sensors). In this case, **-channel11, -channel22,...** will not be taken into account for testing purposes.

The @CASE macro generates as many tests as the number of component pins present plus a final test that checks the average of all results. If the final test differs from the autolearned one, the displacement found will be applied to each pin test. This technique is used to counterbalance the errors due to a shifted component position during the soldering process.

It is possible to examine the result of each single test using the internal oscilloscope from the test environment.

EXAMPLE

EXAMPLE 1
@Z (“U1DEF”,20,0,5,10,1000-1001-1002-1003-1004-1005-1006-1007);
@CASE (“U1”,0,12-1000,14-1001,15-1002,19-1003,99-1004,94-1005,23-1006,44-1007);
In this example, the movable capacitive sense is used to check U1. The @Z macro defines the up position at 20mm, the near position at 0, the down position at 5mm, the speed of the Z axis at 10, it follows the list of the channels associated. The @CASE macro performs the pin open check for the component U1, moving the capacitive sense at the coordinates of channel 1000 for pin1, 1001 for pin 2 and so on

P.S. Macro @CASE is automatically generated through menu TOOLS\Software [IC vectorless test generator-Open Fix] in format:

@Z (“U1DEF”,,,5,10,1000-1001-1002-1003-1004-1005-1006-1007);
@CASE (“U1”,0,12-1000,14-1001,15-1002,19-1003,99-1004,94-1005,23-1006,44-1007);

Values of Up and Near positions defined in macro @Z will acquire the values set in the board configuration.

EXAMPLE 2
@CASE (“U2”,2,32,34,65,199,19,6,77,56);
In this example, the fixed capacitive sense nr. 2 is used to check the component U2

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEM
CFP module	CMP and/or CFP module

@CLUSTER

File.SRG [X] File.LIB [ ]

FUNCTION

It calls a functional cluster (static or dynamic).

SYNTAX

@CLUSTER (“cluster name”,diagnostic,channel)

cluster name: alphanumeric string long up to 9 characters that specifies the name of the functional cluster
diagnostic: this field can be one or two letters long, with the following meaning:

the 1st letter can be:

- **A** if there is an error on the output channels, all the signals involved in the diagnostic chain will be displayed. In this mode a faulty area that surely contains the error, is determined.
- **I** the diagnostic process will use automatically the in-circuit channels (eventually present in a bed of nails fixture) for the acquisition of the various signals during the diagnostic chain.
- **N** it indicates that is not performed any diagnostic process for the cluster in examination. If any error is discovered error, the program finishes pointing out the faulty channel and the step where the error happened.
- **P** it indicates that the diagnostic process will use the probe for the acquisition of the various signals.
- **S** it indicates that the cluster in examination is of static type. This letter must be present if the cluster in examination is a static cluster.

the 2nd letter can be:

- **B** If this option is selected, it specifies that the diagnostic process will use the bus guide routine for the fault recognition.
- **S** if this option is selected, the diagnostic message will not be displayed in full way, but will be listed only the names of the various testing included in the program without the indication relating to the various signals.
- **Q** if this option is selected, it indicates that the diagnostic process will examine all the points for every net.

channel: specifies the functional channels used for the static cluster execution. The character “/” indicates a group of adjacent channels and the character “-” is used to indicate not adjacent channels.

NOTE

The first letter that identifies the type of diagnostic must be always present, instead the second one can be omitted. The type of diagnostic can be indicated in this macro or can be defined in the “Others Test Settings” dialog. This dialog is automatically opened in the test environment, by clicking the button and then the “Others” button. If the type of diagnostic is defined in this dialog, it will remain valid for all the functional cluster included in the .srg file.

EXAMPLE

@CLUSTER (“CTE1”,P,1/192);
*The functional cluster CTE1 which use the channel from 1 to 192 is performed The diagnostic process will acquire the channels with the probe.
@CLUSTER (“CL1”,IB,1/224);

The functional cluster CL1 which use the channel from 1 to 224 is performed The diagnostic process will automatically use the in-circuit channels and the bus guide routine.

@CLUSTER (“CL2”,N,1-5-7/32);
The functional cluster CL2 is performed without the diagnostic process. It provides only the information regarding the channel and the test pattern where the fault has been found.

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@CON

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

FUNCTION

This macro measures a small value of capacitance (from 1fF to 20nF) between two points but it can also be used to check the short between tracks on a PCB by measuring the capacitance between the specified track and the PCB ground layer. The capacitance values for the PCB tracks in @CON macros must be autolearned the first time from a good board. This macro is used to auto-learn the values from a sample board so that they can be used for the boards under test. After entering the macro in the source file, it is necessary to run command TOOLS\HARDWARE\Autolearn\Debug, using the sample board, to acquire the related values. The source file (file .srg) is automatically changed at the end of this command, so that the execution of the macro, on the boards under test, corresponds to a measurement comparable with the self-learnt value of the sample board and the value measured on the current board.

SYNTAX

@CON (“comment”, cap. value, tol, ch 1, ch 2)

comment: alphanumeric string long up to 30 characters that identify the macro.
cap. value: capacitance value expressed in pF.
tol: measure tolerance
ch 1: channel that corresponds to the first point to be measured.
ch 2: channel that corresponds to the second point to be measured.

NOTE

While measuring the capacitance values, two scales of measure can be chosen according to the track capacitance of the PCB to be checked:

- from 1fF to 1nF (5V) used for bare boards
- from 100fF to 20nF (0.25V) used for populated boards

To choose the first scale, the SCA=H command must be added at the end of the @CON macro like this:
@CON (“CAP 789”,0,25,789,45),**SCA=H**; or defined in the @DEF macro like this: @DEF(SCA=H)

To choose the second scale, the SCA=L command must be added at the end of the @CON macro like this:
@CON (“CAP 789”,0,25,789,45),**SCA=L**; or defined in the @DEF macro like this: @DEF(SCA=L)

If the SCA command is not specified, SCA=H will be taken as a default.

Macro @CON can be automatically generated if option Capacitive in section Short Test is selected during the execution of command CAD\Generate. This option enables to measure the short-circuits between the tracks and the capacity value between the track and the grounding net of the board under test. This option is recommended for printed circuits without fitted components.

When the @CON macro is generated the first time, it will contain “0” in the capacitance field because the value must be autolearned from a good board (to do it, select the TOOLS\HARDWARE\Autolearn/Debug menu and @CON option).

When the @CON macro is generated, it will contain the word “GND” in the channel 2 field because at that moment, the software doesn’t know which ground channel will be used. To specify it, a @DEF(GND=channel number) line must be added at the beginning of the File.srg as illustrated in the example below.

EXAMPLE

EXAMPLE 1
Use of the @CON macro to measure the capacitance between two points
@CON (“C1”,400,10,23,364);

EXAMPLE 2
Use of macro @CON to auto-learn the capacity values between a signal and the grounding net of a sample board. These values will then be used to measure the same values on the boards under test.

File.srg before autolearning:

```
@DEF (GND=56);
@CON ("CAP 789",200,25,789,GND);
@CON ("CAP 825",200,25,825,GND);
```

File.srg after autolearning:

@DEF (GND=56);
@CON ("CAP 789",**340**,25,789,GND);
@CON ("CAP 825",**23**,25,825,GND);

After the auto-learning has been carried out on a sample board, the source file will contain the actual values that have to be measured on the boards under test. Value 200, entered prior to auto-learning, is an arbitrary value that is inserted to allow the compilation of a code line containing macro @CON.

@CONT

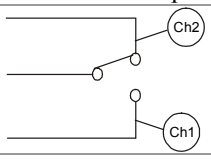
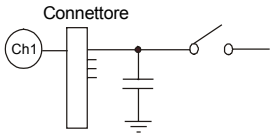
File.SRG [X] File.LIB [ ] Only for bad of nails systems [X]

FUNCTION

This macro checks the proper contact between the test channels of the fixture and the test points on the board to be tested. The @CONT macro performs the following operations:

1. All test channels except one are connected to ground.
2. A very little current is injected on the channel (the one not connected to ground) and a voltage measurement is performed If there is a good contact between the test channel and the test point, the injected current will close to ground because of the channel impedance In this case, the voltmeter will read a voltage value that is lower than the voltage clamp. On the contrary, (bad contact between channel and test point) the voltage clamp will be read.

The operation described on point 2- is then repeated for all test channels.
There are some particular configurations on the board to be tested where the test channel appears to be always disconnected from the test point. A good rule is to examine all these cases and eventually to exclude these “special” channels from the check. The most frequent particular configurations are:

A test channel connected on a relay pin that remains open when it is in its OFF position.	A test channel connected to a capacitor with a big value (greater than 1µF).
	

SYNTAX

@CONT (“comment”,checked channels,excluded channels)

comment: alphanumeric string up to 30 characters long that identifies the macro.
checked channels: family of channel to be checked. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels.
excluded channels: The channel or channels indicated here are excluded from the check because of particular configurations on the board. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels.

NOTE

The @CONT macro generates as many test labels(LAB) as the number of channels to be checked.
The @CONT macro is normally not used to check interrupted tracks on the board but in certain cases it may discover these problems.

EXAMPLE

@CONT ("BOARD-1 CANALE ",1/44,5-6-14-43);
The @CONT macro checks the good contact between channels from 1 to 44 and the correspondent test points on the board, excluding channels 5,6,14 and 43 from the check.

@CONT ("TP ",1/117,50/54-92-99-103);
The @CONT macro checks the good contact between channels from 1 to 117 and the correspondent test points on the board, excluding channels 50, 51, 52, 53, 54, 92, 99 and 103

@CTO

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

FUNCTION

This macro is used to check the electric continuity between two points of the board under test. The macro checks the resistance between the two points. If the measured value is below the reference value, the two point are regarded as connected to one another and the macro returns a positive result. If the measured value is above the reference threshold, the two points have a continuity and the macro sets an error flag.
The resistance value taken as reference corresponds to the one set in field **Impedance** of section *Continuity Test* in dialogue *CAD\Generate* (the default value is 25Ω).

SYNTAX

@CTO ("comment", ch 1, ch 2)

comment: alphanumerical string with a maximum length of 30 characters that identifies the component.
ch 1: first measured channel
ch 2: second measured channel

NOTE

Macro @CTO is automatically entered in the source file when the user selects options that enable continuity tests between two points (continuity tests, probes test, etc…) during the generation of the source file.

EXAMPLE

@CTO ("L5105",221,6);
Macro @CTO is used to check the continuity between the two points connected to channels 221 and 6).

@CTO("PROBE TEST 1-2",1000,1001),FLP=1002-1003;
@CTO("PROBE TEST 3-4",1002,1003);
Macro @CTO, entered automatically at the end of the source file, is used to check whether the four mobile probes of the system are in contact with the board under test. The automatic entry is carried out by selecting field **Add Probes Test** in dialogue *Cad Generate*.

@FIX

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

FUNCTION

Allows to assign a physical SCAFP channel to a test channel.
It is used for the assignment of the 8 fixed test channels connected to the magnetic fixed probes to a test channel

SYNTAX

syntax 1: **@FIX** (ch_f=ch_t,…);
syntax 2: **@FIX** (FXn=ch_t,…);

ch_f: physical SCAFP channel. (4099, 4100, 4103, 4104, 4107, 4108, 4111, 4112).
ch_t: test channel
FXn: physical SCAFP channel

Fix channel name	Fix channel
FX1	4099
FX2	4100
FX3	4103
FX4	4104
FX5	4107
FX6	4108
FX7	4111
FX8	4112

NOTE

The physical test channel of the SCAFP card can be expressed also as FX1,…, FX8.

For the correspondence between the fixed test channel and the SCAFP card channel, see table in the paragraph.3.1.6 Work Area.

If an .srg file includes 2 @Fix macros, the second one clears out the statement of the first one.

EXAMPLE

EXAMPLE 1
@FIX (4099=10, 4107=15, …….., 4111=17)

EXAMPLE 2
@FIX (FX1=10, FX5=15, …….., FX7=17)

@ISO

File.SRG [X] File.LIB [ ]

FUNCTION

This macro is used to verify the electric insulation between the tracks of printed circuits.
The resistance value taken as reference is 5Ω. If the read value is below 5Ω, the 2 tracks are regarded as short-circuited.
This macro is used to auto-learn the values from a sample board so that they can be used for the boards under test. After entering the macro in the source file, it is necessary to run command TOOLS\HARDWARE\Autolearn/Debug, using the sample board, to acquire the related values. The source file (file .srg) is automatically changed at the end of this command, so that the execution of the macro, on the boards under test, corresponds to a measurement comparable with the self-learnt value of the sample board and the value measured on the current board.

SYNTAX

@ISO (“comment”, ch 1, ch 2)

comment: alphanumerical string with a maximum length of 30 characters that identifies the component.
ch 1: first measure channel
ch 2: second measure channel

SEE ALSO

Macro: **@ISOV**, **@RESV**

NOTE

Macro @ISO can be generated automatically.
It is not possible to use the Guarding generator with macro @ISO.

EXAMPLE

@ISO (“SHORT 253-252”,253,252);

@ISOHV

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

FUNCTION

This macro is used to check the isolation between two nets. It is possible to set the voltage and resistor value that will be used during measurement. If the measured resistor value is higher that the declared one, the test will pass, otherwise it will fail.

SYNTAX

@ISOHV(“comment”,volt, res, channel 1, channel 2)

comment: name of the test. It is a string up to 30 characters
volt: voltage value used for measurement, expressed in Volt. The unit of measure must be omitted. The value can be from 1 to 500.
res: resistor value to measure. It is expressed in Ohm. The value can be from 100Ω to 1GΩ. The unit of measure must be omitted.
tol: tolerance of the measurement.
channel 1: first channel
channel 2: second channel

SEE ALSO

@ISOV, **@RESV, @RESHV** macros

NOTE

It is not possible to use the guarding generator if the **@ISOHV** macro is used.

EXAMPLE

@ISOHV(“SHORT 253-252”,100,1000000,253,252);

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEM
Module HVI	Module HVI

@ISOV

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

FUNCTION

This macro is used to check the electric isolation between PCB tracks on populated boards using as Flying Probe system. In the macro it is possible to specify the voltage used to perform the measure and the resistance that must be checked. If the measured resistance value is lesser that the declared one, the macro will fail.

SYNTAX

@ISOV (“comment”,voltage value,resistance value, ch 1, ch 2)

comment:	comment (max. 30 characters) that indicates the component denomination.
voltage value:	voltage expressed in Volt used to perform the measure. The voltage range is from 0 to 50V. The unit of measure must be omitted.
resistance value:	Resistance value expressed in Ohm used to check the electrical isolation between the two channels. The unit of measure must be omitted. The resistance range is from 0.01 to 1GΩ.
ch 1:	first measure channel
ch 2:	second measure channel

SEE ALSO

Macro: **@RESV**

NOTE

It is not possible to use the guarding generator while using the @ISOV macro.

EXAMPLE

@ISOV (“SHORT 25-272”,50,100000,25,272);

@OPICN/P

File.SRG [X] File.LIB [ ] Not available for BBT systems [X]

FUNCTION

This macro is used to find open pins or pins that have not correctly been soldered on a specific component.
The Open Pin technique searches the disconnected pins of an integrated circuit by analyzing and measuring the parasitic transistor, which is present between two pins of an integrated circuit and its sub-layer. A pin of the component acts as emitter while the other one represents the receiver; the sub-layer acts as basis.
This macro is used to auto-learn the values from a sample board so that they can be used for the boards under test. After entering the macro in the source file, it is necessary to run command TOOLS\HARDWARE\Autolearn\Debug, using the sample board, to acquire the related values. The source file (file .srg) is automatically changed at the end of this command, so that the execution of the macro, on the boards under test, corresponds to a measurement comparable with the self-learnt value of the sample board and the value measured on the current board.

SYNTAX

@OPICN(“comment”,ch)

comment: alphanumerical string with a maximum length of 30 characters that identifies the component
ch: group of channels connected to the pins of the component. The channel numbers are separated by commas.

NOTE

Macros OPICN or OPICP will be used, depending on the type of sub-layer of the component (type n or p).

For integrated circuits with N pins, the maximum number of available parasitic transistors is (N-1)*(N-2). To optimize testing, it is advisable to restrict the number of parasitic transistors under test. The number of parasitic transistors that will be examined is represented by the value that will assigned to variable MAX in macro @DEF (…) that precedes macro @OPIC(…). Variable MAX contains the maximum value of pairs that will be examined for each pin of the component under test. Another parameter used to restrict the number of parasitic transistors under test is represented by the value of variable FAR, which can also be analyzed in macro @DEF(…) that precedes macro @OPICN/P

N.B. before entering the program line with OPIC, it is necessary to enter a macro @DEF, as shown here below.

```
@DEF (VCC=26,MAX=10,FAR=5);
@OPICN(“U1”, 12,14,15,19,99,94,23,44);
```

where command VCC=26 defines the channel used to feed the component. If macro OPICN is used, it is also necessary to specify the channel connected to the power supply with command VCC=.…
If macro OPICP is used, it is necessary to specify the grounding channel with command GND=...
Command MAX=… defines the maximum number of pin pairs that can be formed for each pin under test. During the auto-learning of the board values, this parameter is usually set to 10. After the end of auto-learning it is instead set to 2.
Command FAR=….defines the maximum distance between the 2 pins select to form the pair. This parameter is generally set to 5.

After the auto-learning on the sample board has been completed, the source file is modified to reflect the pairs for which no test will be carried out. These represent the pairs for which no parasitic transistor was detected.

EXAMPLE

If the source file contains the following lines:

```
@DEF (VCC=6,MAX=10,FAR=5);
@OPICN("U1",30,335,331,421,335,354,341,327,328);
```

Macro OPICN carries out the test on the open pins of component U1, which has an "n" type of sub-layer. The macro ties to form 10 pairs of pins for each component pin using a distance of 5 pins

After auto-learning, the source file is modified to reflect which pairs will not be measured, as shown here below.

```
@DEF (VCC=6,MAX=10,FAR=5);
@OPICN("B-1 U7230",30,335,331,421,335,354,341,327,328),
LAB=" 1-2",NER=Z,
LAB=" 1-7",NER=Z,
LAB=" 3-2",NER=Z,
LAB=" 4-2",NER=Z,
LAB=" 2-6",NER=Z,
LAB=" 6-2",NER=Z,
LAB=" 2-7",NER=Z,
LAB=" 7-2",NER=Z,
LAB=" 5-6",NER=Z,
LAB=" 6-5",NER=Z,
LAB=" 5-7",NER=Z,
LAB=" 7-5",NER=Z,
LAB=" 8-7",NER=Z,
LAB=" 9-7",NER=Z,
LAB=" 6-7",NER=Z,
LAB=" 7-6",NER=Z;
```

@PAR

File.SRG [X] File.LIB [ ] Not available for BBT systems [X]

FUNCTION

This macro is used to perform parametric measurements on the signals of the board under test. Macro @PAR should be used when the component parameters (type, value, tolerance, etc.…) are not available, as it can be regarded as an alternative to in-circuit tests.
The macro will perform, for each signal, four types of measurements as if a condenser, a polarized diode and an inversely polarized diode were present between the measured point and the grounding net.

SYNTAX

@PAR ("comment", ch 1, GND, val 1, val 2,val 3, val 4, val 5, val 6, val 7, val 8);

comment: alphanumerical string with a maximum length of 30 characters that identifies the component.
ch 1: first measure channel
GND: indicates the channel connected to the ground net. The actual value is defined through macro @DEF(GND=0, GND= XXX); where XXX represents the channel number connected to the ground net. Macro @DEF(…) is entered automatically by the system.
val 1: reference value for the capacitive test in pF. The reference value that is automatically entered during the generation phase is 1000pF.
val 2: testing tolerance for the value measured with the capacitive test. The reference value that is automatically entered during the generation phase is 100%.
val 3: minimum reference value for the resistance test in Ω. The reference value that is automatically entered during the generation phase is 200Ω.
val 4: maximum reference value for the resistance test in Ω. The reference value that is automatically entered during the generation phase is 50.000Ω.
val 5: voltage drop value referred to the terminations of a diode on which current is directly injected in the direction of direct conduction. This value is measured in Volt. The reference value that is automatically entered during the generation phase is 0.6Volt.
val 6: voltage drop value referred to the terminations of a diode on which current is directly injected in the direction of inverse conduction. This value is measured in Volt. The reference value that is automatically entered during the generation phase is 1.9Volt. The two parameters that follow have the same meaning of the previous one but are referred to an inversely polarized diode.
val 7: voltage drop value referred to the terminations of a diode on which current is directly injected in the direction of inverse conduction. This value is measured in Volt. The reference value that is automatically entered during the generation phase is -1.9Volt.
val 8: voltage drop value referred to the terminations of a diode on which current is directly injected in the direction of direct conduction. This value is measured in Volt. The reference value that is automatically entered during the generation phase is -0.6Volt.

NOTE

Macro @PAR is automatically inserted in the source file when Parametric Test is chosen as test type during the generation of the source file (CAD\Generate). Macro @PAR.1 is entered in the source file for each signal listed in file .nod. The parameters entered in the statements of the macro are not those executed for the board testing, but the reference ones used to carry out the auto-learning of the actual values of sample boards, which are regarded as valid. For further information on auto-learning, see menu **RUN\Test** dialogue *Parametric Test*. One of the tests carried out with macro @PAR is the capacitive test. To be able to carry out auto-learning for this kind of test, it is necessary to select *Adjacency* and option *Capacitive* in section *Short Test* during the generation of the source file.
For further information on the steps to use with macro @PAR, see the example shown in the paragraph on menu CAD\Generate.

EXAMPLE

The paragraph here below shows a short example of source file containing macro @PAR.

```
…
…
$section CAPACITIVE SHORTS TEST;
@DEF(GND=0,GND=182);
…
@PAR("PAR 375",375,GND, 1000,100,200,50000,0.6, 1.9, -1.9, -0.6 );
@PAR("PAR 11",11,GND, 1000,100,200,50000,0.6, 1.9, -1.9, -0.6 );
…
@DEF(GND=0);
$section SHORTS TEST ON ADJACENCY;
…
```

During the generation of the source file and the analysis of file .nod, the system has found the signal associated to channel 182 and classified it as the signal with the most extended polygonal, thus selecting it as reference GND for the four measurements to be performed.
In the test environment, the use of dialogue *Parametric Test* with sample boards, enables to auto-learn the parameter values that will be used during the board testing.
To carry out auto-learning, select *First Board Autolearn* in dialogue *Customize Test* and *Capacitive Test* in section *Certification Method*.

@PINAD

File.SRG [X] File.LIB [ ] Only for no BBT systems Flying Probe [X]

FUNCTION

Performs the short circuit test between the specified adjacent pins.

SYNTAX

@PINAD (“comment”,ch n)

comment: alphanumeric string up to 30 characters that identifies the macro.
ch n: channels connected to adjacent pins on which perform the short circuit test. The channels numbers are separated by dashes.

EXAMPLE

@PINAD (“KB1”,297-172-114-279-314-302-301-281-280-278-272-315)

@RESHV

File.SRG [X] File.LIB [ ]

FUNCTION

This macro checks high value resistors and it gives the possibility to set the voltage value used during measurement.

SYNTAX

@RESHV(“comment”,volt, res, tol, channel 1, channel 2)

comment: name of the test. It is a string up to 30 characters
volt: voltage value used for measurement, expressed in Volt. The unit of measure must be omitted. The value can be from 1 to 100 for bed of nail systems and from 1 to 500 for flying probe systems.
res: resistor value to measure. It is expressed in Ohm. The value can be from 100Ω to 1GΩ. The unit of measure must be omitted.
tol: tolerance of the measurement.
channel 1: first channel
channel 2: second channel

SEE ALSO

@ZENHV, **@ISOV**, **@ISOHV** macros

NOTE

It is not possible to use the guarding generator if the **@RESHV** macro is used.

EXAMPLE

@RESHV(“R1”,200,1000000,23,22);

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
HVI Module	HVI Module

@RESV

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

FUNCTION

This macro is used to measure the resistance. In the macro it is possible to specify the voltage used to perform the measure.

SYNTAX

@RESV (“comment”,voltage value,resistance value,tol, ch 1, ch 2)

comment:	comment (max. 30 characters) that indicates the component denomination.
voltage value:	voltage expressed in Volt used to perform the measure. The voltage range is from 0 to 50V. The unit of measure must be omitted.
resistance value:	Resistance value expressed in Ohm used to check the electrical isolation between the two channels. The resistance range is from 0.01 to 1GΩ. The unit of measure must be omitted.
tol:	measure tolerance
ch 1:	first measure channel
ch 2:	second measure channel

NOTE

It is not possible to use the guarding generator while using the @RESV macro.

EXAMPLE

@RESV (“R2”,0.7,10000,10,23,22);

@SHORT

File.SRG [X] File.LIB [X] Only for bad of nails systems [X]

FUNCTION

This macro is used only for systems with beds of nails. The macro checks that the channels specified in the macro are short-circuited, using the procedure described here below:

1. All channels, except the first one, are grounded.
2. A small current is injected into the first channel so that it is possible to measure the impedance. If the impedance measured is above the defined value, the process continues on the second channel until the last channel. If the impedance measured is below the defined value, the process continues but the related channel is stored internally.
3. The test is again performed between each stored channel (with an impedance value below the defined value) and on all the other channels. In other words, all the stored channels of the group, less one, are connected to the ground, then a small amount of current is injected so that the impedance can be measured.... This operation is necessary to determine the short-circuited channels.

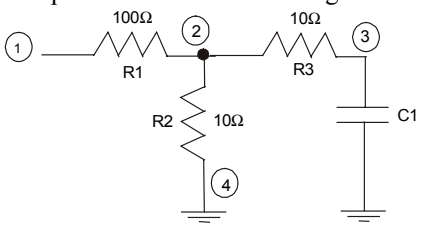
SYNTAX

@SHORT (“comment”,imped.,ch range,fam 1,fam 2,...)

comment: alphanumerical string with a maximum length of 30 characters that identifies a macro.
imped.: impedance value expressed in Ohm, below which two values are regarded as short-circuited. The value must be indicated without unit of measurement.
ch range: range of channels on which the short-circuit test has to be carried out. The range of channels must be expressed by indicating the first and last channels, separated by character "/".
fam 1: family of channels to exclude due to reasons connected with the circuits. Channels should be separated by character "-".
fam 2: family of channels to exclude due to reasons connected with the circuits. Channels should be separated by character "-".

NOTE

Unusual conditions may occur. The most common one is the LOW IMPEDANCE condition that leads to detection of a short-circuited channel during the control referred to in point 1, but does not allow to find short-circuits with other channels of the groups stored as described in point 3. This condition is generated by a circuit of this kind:



Let's suppose that the following macro has been written: @SHORT (“CORTI”,5,1/64);

In this case, during the control referred to in point 1, the application finds an impedance smaller or equal to 5Ω on channel 2, because R2 and R3 are parallel after all the channels have been grounded. The control on channel 2, referred to in point 3, does not enable to find a short-circuit on the channel.

EXAMPLE

@SHORT ("NODE",5,107/200,2-3,5-12,20/23,46-78);
Macro SHORT examines the channels range from 107 to 200, verifying that the impedance is below 5Ω. Ch 2 and 3, ch 5 and 12, ch 20,21,22,23 and 46 and 78 are regarded as short-circuited)

@SHOW

File.SRG [X] File.LIB [ ] Only for Flying Probe and Stream Line systems [X]

FUNCTION

This macro performs the visual inspection test on a component. It opens the SHOW dialog where the camera is focusing the component to test so that the operator can check visually if the component is correctly mounted or not.

SYNTAX

@SHOW ("comment",ch);

comment: alphanumeric string long up to 30 characters that identifies the macro.
ch: indicates the channel associated to the component pin.

NOTE

Temporarily interrupts the test program execution and displays the SHOW dialog The test result (PASS or FAIL depends on the answer of the operator.

Generally, the channels used for the measurements identify the pin of a component. To perform a visual inspection with macro @SHOW (…) it may be necessary to define new nodes that physically correspond to the center of the component. By doing so, the camera can focus more accurately on the nodes when macro @SHOW(…) is executed.

EXAMPLE

@SHOW ("Diode orientation verification",15);

HARDWARE REQUIREMENTS

Stream Line	FLYING PROBE SYSTEMS
Camera	None

@VIS

File.SRG [X] File.LIB [ ] Only for Flying Probe and Stream Line systems [X]

FUNCTION

Performs the visual inspection on a component.

SYNTAX

@VIS (“comment”) or **@VIS** (“comment”,tollerance)

comment: comment (max. 30 characters) that indicates the component denomination.
tolerance: tolerance of the image. The tolerance expressed in the macro has got the same meaning as the one defined in the Optical Inspection/Set parameter dialog. The tolerance indicated here has got the priority on the one specified in the Optical Inspection/Set Parameters dialog. In this way, the programmer can change the tolerance of one component in the source file without going into the Optical Inspection dialog. (***Example:*** if in a batch of board, one board is different from the others but accepted as well).

NOTE

From S20WIN software release 2.10 on, the tolerance field in the macro syntax is optional.

EXAMPLE

@VIS (“U1”);
This macro performs the visual inspection for the component U1

HARDWARE REQUIREMENTS

Stream Line	FLYING PROBE SYSTEMS
Camera	None

@Z

File.SRG [X] File.LIB [ ] Only for Flying Probe systems [X]

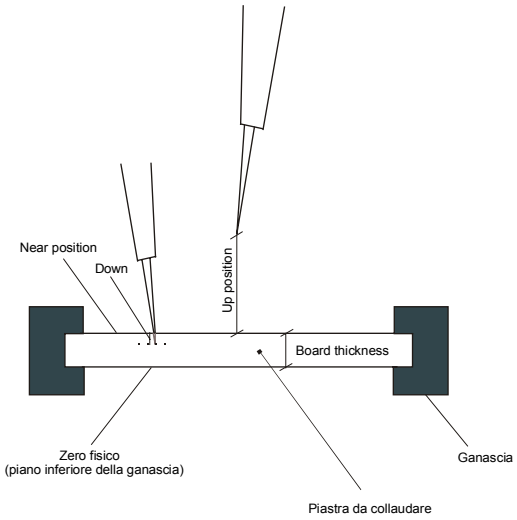
FUNCTION

Defines the parameters of axis Z (up, near, down position; board thickness and speed of axis Z) that have to be associated to a family of channels. This macro is used only to associate a family of channels and for Flying Probe equipment.

SYNTAX

@Z (“comment”,up,near,down,speed,ch)

comment: alphanumeric string long up to 30 characters that identify the macro.
up: height reached by the probes during the movements. The height is expressed in mm.
near: height of the top side of the board to be tested. The top side of the board is the one towards the probes. The height is expressed in mm.
down: Height of the probe, below the top side of the board (near).



speed: Z axis speed. The value 1 indicates the minimum speed while 10 indicates the maximum speed.
ch: family of channels associated to the parameters defined above. The “/” character is used to indicate a group of contiguous channels and the “,” or the “-” character is used to indicate non-contiguous channels.

SEE ALSO

The Z Axis parameters can be defined for all channels of the board, choosing the BOARD\Configure menu and pressing the [Z Axis Settings ...] button.

NOTE

The NEAR parameter is the reference point of the up and down parameters.

EXAMPLE

Let’s assume that the Z axis parameters are defined like this in the BOARD\Configure menu: up=20mm, near=0, down=-1.5mm, board thickness =2mm e speed=10

```
#RES("R1",100,10,23,45);
#RES("R2",330,10,24,52);
#RES("R3",4700,10,67,55);
@Z ("ALT1",15,-5,-1.5,10,100/130);
#RES("R4",330,10,100,101);
#RES("R5",4700,10,102,103);
```

*The parameters defined in the @Z macro are used for R4 and R5. These parameters are applied to channels from 100 to 130. For R1, R2 and R3 are used the parameters defined in the BOARD\Configure menu *

@ZENHV

File.SRG [X] File.LIB [ ]

FUNCTION

This macro is used to check zener diodes up to 100V for bed of nail systems and zener diode up to 250V for flying probe systems.

SYNTAX

@ZENHV(“comment”,volt, tol, channel 1, channel 2)

comment: name of the test. It is a string up to 30 characters
volt: zener diode voltage value, expressed in Volt. The unit of measure must be omitted.
tol: tolerance of the measurement.
channel 1: channel connected to cathode
channel 2: channel connected to anode

SEE ALSO

@RESHV, **@ISOV**, **@ISOHV** macro

NOTE

It is not possible to use the guarding generator if the **@ZENHV** macro is used.

EXAMPLE

@ZENHV(“R1”,80,10,23,22);

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
TIZ module	TIZ module

4.3.2 System macro in the library file:

@C
@IEEE
@L
@PAT

@C

File.SRG ☐ File.LIB X

FUNCTION

This macro defines the values of the parameters used to perform logic in-circuit tests.

SYNTAX

@C (1 logic,0 logic,high threshold,low threshold,test time,clock number,clock front,validation table);

1 logic: indicates the voltage value of the logic 1 piloted by the driver channels used for the test. The value is expressed in Volts and no unit of measurement is required.

0 logic: indicates the voltage value of the logic 0 piloted by the driver channels used for the test. The value is expressed in Volts and no unit of measurement is required.

high threshold: indicates the high threshold value of the sensor channels used for the test. The value is expressed in Volts and no unit of measurement is required.

low threshold: indicates the low threshold value of the sensor channels used for the test. The value is expressed in Volts and no unit of measurement is required.

test time: indicates the test time or the clock time output by the clock generator. If the next field (clock number) is 0, this field indicates the length of the test in ms. Time must be entered without unit of measurement. If the next field differs from 0, the value of this field indicates the period of the clock signal output to generator CKGEN.

clock number: this field indicates the number of clock pulses for the test.

clock front: this field indicates the front of the first clock impulse and can display two values:
P rise front
N fall front

validation table: Indicates the type of validation table used to test the component and can display two values:
C combined validation table. The test is carried out with commands D2P=Y and HLD=N (Driver 2 is programmed as impulsive voltage generator and the channel relay programming is changed for each test).
S sequential validation table. The test is carried out with commands D2P=N and HLD=Y (Driver 2 is programmed as continuous voltage generator and the channel relay programming is maintained for the previous and current test).

EXAMPLE

@C (4.7,0.2,2.4,0.8,0.1,3,P,C);
Macro @C sets the high/low threshold for driver channels respectively to 4.7V and 0.2V; the high/low threshold of the sensor channels respectively to 2.4V and 0.8V; the length of the clock signal to 0.1ms and a train of impulses constituted by 3 clock strikes. The first front is the rise one and the validation table used is the combined one

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@IEEE

File.SRG [ ] File.LIB [X]

FUNCTION

Permits the user to define a C language section inside the library to program the external instrumentation via the IEEE 488.

SYNTAX

```
@IEEE
...
...  C language section
...
@IEEE
```

The two @IEEE-@IEEE macros define a C language section inside the library

EXAMPLE

```
#VSET(COMM,CHANNEL,VVAL)
HIG = 2;
LOW = -2;
    @IEEE
        #include<stdio.h>
        #include<ieee.h>

        main()
        {
        int chn;
        double volt;
        char comando[50];

        chn = CHANNEL;
        volt = VVAL;
        sprintf(comando,"VSET %d,%f\r\n",chn,volt);

        init_ieee("PWSPPL");
        write_ieee("PWSPPL",comando);
        pass_par(0.0);
        exit(0);
        }
    @IEEE
LAB=COMM;
```

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@L

File.SRG [ ] File.LIB [X]

FUNCTION

Allows the forcing of determinate logic levels (H,L and T) on the input pins of the digital component under test and, on the basis of these, the verification of the logic levels (0,1 and Z) on the output pins. The values of the forcing and verification levels are written in the @C system macro.

SYNTAX

@L (S1,S2,.....Sn)

S1,S2,.....Sn: Represent the state of the component pins:

- **H** Forced high level
- **L** Forced low level.
- **1** Check an high level
- **0** Check a low level
- **h** Forced high level of the channel. This condition is not indispensable to obtain valid outputs.
- **l** Forced high level of the channel. This condition is not indispensable to obtain valid outputs
- **T** Forced clock
- **.** Not connected
- **Z** Three-state level

SEE ALSO

@C

EXAMPLE

```
#7400(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,
P12,P13,P14);
@C (4.7,0,2.4,.8,.1,1,P,C);
@L (HH0...........);
@L(Lh1...........);
@L(hL1...........);
@L(...HH0........);
@L(...Lh1........);
@L(...hL1........);
@L(.......0HH....);
@L(.......1Lh....);
@L(.......1hL....);
@L(..........0HH.);
@L(..........1Lh.);
@L(..........1hL.);
```

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@PAT

File.SRG [ ] File.LIB [X]

FUNCTION

Permits the user to define a functional section inside the library.

SYNTAX

```
@PAT
...
...          functional section
@PAT
```

The two @PAT-@PAT macros define a functional section inside the library.

EXAMPLE

```
#7473IC39(COMM,P1,P2,P3,,P5,P6,P7,P8,P9,P10,,P12,P13,P14);
!D,2,14;
*PI(III.IIIOOI.OOI);
*TY(CCCVCCC..CG..C);
!6,4,2,0,0,P1,P2,P3,P14,P12,P13;
!6,4,2,0,0,P5,P6,P7,P10,P8,P9;
@C (4.7,0,2.4,.8,.1,1,P,S);
@PAT
START ;
IL 3-24;
IL 369;
IH 442;
OM 370-466;
/ TIMING DEFAULT;
IL 442;
/ TIMING DEFAULT;
OL 370;
OH 466;
/;
* SEZ 1
IL 24;
IH 3;
OM 369;
/ TIMING DEFAULT;
IL 3;
/ TIMING DEFAULT;
IH 369;
/;
IH 24;
/;
IH 3;
OM 369;
/;
IL 3;
/;
IL 369;
@PAT
LAB=COMM;
```

HARDWARE REQUIREMENTS

BAD of NAILS	FLYING PROBE SYSTEMS
Protection key enable	Protection key enable
DHF module	DHF module

4.3.3 System macros written in both files .srg File and .lib File:

@APPEND
@CALC
@COMPARE
@CONV
@COPY
@DEF
@INPUT
@OUTPUT
@SUBCOMP

@APPEND

File.SRG [X] File.LIB [ ]

FUNCTION

Appends string 1 to string 2 and stores the result is in string 2.

SYNTAX

@APPEND (“comment”,string 1,string 2)

comment: alphanumeric string long up to 30 characters that identifies the macro.

string 1: string 1 can be:

- up to 30 characters enclosed in quotation marks (“”).
- string system variable STR[1 ÷ 15]
- one of the other alphanumeric system variables listed in the table at the end of this section.

string 2: string 2 can be:

- string system variable STR[1 ÷ 15].

SEE ALSO

Macro: **@INPUT**, **@OUTPUT**, **@COMPARE**, **@SUBCOMP**

NOTE

The @APPEND macro appends the two strings but it doesn’t show any result. To do that, the @OUTPUT macro can be used.

EXAMPLE

EXAMPLE 1
@APPEND ("LB1","The software release is: ",STR[1]);
If the STR[1] variable contains “1.2A”, after the execution of the LB1 label, the value stored in STR[1] will be “The software release is 1.2A”

EXAMPLE 2
@APPEND ("LB1","The software release is:",STR[1]);
@OUTPUT ("LB2","-- Append two strings -- ",STR[1]);
* After the execution of LB1 and LB2, it will be displayed:

-- Append two strings -- The software release is: 1.2A

EXAMPLE 3
@APPEND ("LB04",CODPRO,STR[2]);
@OUTPUT ("LB05","Product Code:",STR[2]);
*If variable STR[2] contains "ALL" and the name of the currently enabled Product Code is "Program10", the following message will be displayed after the execution of tests LB04 and LB05:

Product code: ALL Program10

@CALC

File.SRG [X] File.LIB [X]

FUNCTION

It executes an arithmetic expression between two floating point system variables. The result is stored in variable 2.

SYNTAX

@CALC (“comment”,variable 1 , ”operator”, variable 2)

comment: alphanumeric string long up to 30 characters that identifies the macro.

variable 1: variable 1 may be:

- floating point system variable (NUM[1 ÷ 15]) that contains the first operand of the expression.
- one of the other numeric system variables listed in the table at the end of this section.

operator: operator may be:

+ addition.
- subtraction.
* multiplication.
/ division.
! square root of variable 1 (the result is stored in variable 2).
^ it considers the integer part of variable 1 (the result is stored in variable 2).
& logic AND.
| logic OR.
< shift variable 1 to the left (variable 2 indicates how many times the shift must be executed).
> shift variable 1 to the right (variable 2 indicates how many times the shift must be executed).
~ logic XOR.
% it executes the division between variable 1 and variable 2 and it stores the remainder in variable 2.
$ considers the absolute value of variable 1.

variable 2:

- Floating-point variable (NUM[1 ÷ 15]) that contains the second operand of the expression.
- one of the other alphanumeric system variables listed in the table at the end of this section.

SEE ALSO

Macro: **@CONV**

EXAMPLE

@CALC (“calc”,NUM[1],”+”,NUM[2]);
It sums NUM[1] and NUM[2] and stores the result in NUM[2]

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@COMPARE

File.SRG [X] File.LIB [ ] Not available for BBT, RPS and PST systems [X]

FUNCTION

Compares two strings. If the strings are different, the partial error flag is set (controlled by the BER and BOK commands), if the strings are identical, the partial error set will not be set.

SYNTAX

@COMPARE (“comment”,variable 1,variable 2)

comment: alphanumeric string long up to 30 characters that identify the macro.

variable 1:

- up to 30 characters enclosed in quotation marks (“”).
- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed.
- in the table at the end of this section.

variable 2:

- string system variable STR[1 ÷ 15]).
- one of the other alphanumeric system variables listed in the table at the end of this section.

SEE ALSO

Macro: **@INPUT**, **@OUTPUT**, **@APPEND**, **@SUBCOMP**

EXAMPLE

@COMPARE (“CONTR”,STR[1],STR[2]);
If the two variable values are identical, the partial error flag is set, otherwise the flag is not set

@COMPARE (“CONTR1”,”board1”,STR[12]);
If the STR[12] variable doesn’t contain the “board1”string, the partial error flag will be set

@CONV

File.SRG [X] File.LIB [X]

FUNCTION

This macro converts the content of a string into a number and vice versa. The result of the conversion is placed in variable 2, which maintains the type (number/string) unaltered.
A string --> number conversion is carried out if variable 1 is a string and variable 2 a number.
A number --> string conversion is carried out if variable 1 is a number and variable 2 a string.

SYNTAX

@CONV (“comment”,variable 1, variable 2)

comment: alphanumeric string up to 30 characters that identify the macro.
variable 1:

- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed in the table at the end of this section.
- float system variable NUM[1 ÷ 15].
- one of the other numeric system variables listed in the table at the end of this section..
- up to 30 characters enclosed in quotation marks (“”).

variable 2:

- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed in the table at the end of this section.

SEE ALSO

Macro: **@CALC**

NOTE

If variable 1 and variable are both string variables or if they are both float variables, the conversion will not be executed.

EXAMPLE

```
@CONV (“STRNUM”,”AAA”,NUM[1]);
@CONV(“NUMSTR”,NUM[1],STR[2]);
```

REQUISITI HARDWARE

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@COPY

File.SRG [X] File.LIB [X]

FUNCTION

It copies the content of variable 1 into variable 2. If the two variables are strings, group of characters can be moved from one variable to the other.

SYNTAX

@COPY (“comment”,variable 1, variable 2, [num 1, num 2])

comment: alphanumeric string up to 30 characters that identify the macro.

variable 1:

- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed in the table at the end of this section.
- float system variable NUM[1 ÷ 15].
- one of the other numeric system variables listed in the table at the end of this section..
- up to 30 characters enclosed in quotation marks (“”).

variable 2:

- string system variable STR[1 ÷ 15].
- float system variable NUM[1 ÷ 15].

num1: optional field present only if the variables are two strings. It indicates the position of the first character to be moved.

num2: optional field present only if the variables are two strings. It indicates the position of the last character to be moved.

EXAMPLE

```
@COPY (“CP1”,NUM[2],NUM[4]);
@COPY (“CP2”,STR[1],STR[4]);

@COPY (“CP3”,STR[1],STR[4],1,4);
```

In this case, the first four characters will be moved from variable STR[1] to variable STR[4]

HARDWARE REQUIREMENTS

BAD of NAILS	FLYING PROBES
Protection KEY enable	Protection KEY enable

@DEF

File.SRG [X] File.LIB [X]

FUNCTION

Allows the modification of the default values of the commands. The new default values remain valid until other ones will be specified.

SYNTAX

@DEF (commands)

commands: identifies the commands to be used as default values.

NOTE

If the LI1, LI2, LI3, LI4, VCC, GND, commands are used in the @DEF macro, the channels specified will be added to the ones defined in a previous @DEF (if there is one). Refer to the EXAMPLE 2. To clear a channel from the @DEF macro, the LI1(LI2, LI3, LI4) =0, or GND=0,or VCC=0 commands must be used. Refer to EXAMPLE 3. Set always to zero a macro before set the macro with new channels.

EXAMPLE

EXAMPLE 1
@DEF (LI1=1,TIM=10);
From this point on, the default values will be: LI1=1 and TIM=10

EXAMPLE 2
@DEF (LI1=10)
...
...
@DEF (LI1=15,VCC=3,GND=6);
From this point on, both channels 10 and 15 are connected to Line 1. (Channel 15 is added to channel 10 defined previously)

EXAMPLE 3
@DEF (VCC=10)
...
...
...
@DEF(VCC=0,VCC=15);
This @DEF macro clears all channels connected previously to VCC (VCC=0 command) and it connects channel 15 to VCC, from this point on. (VCC=15)

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@INPUT

File.SRG [X] File.LIB [X]

FUNCTION

It permits the operator to insert a value with the keyboard and to store it into a system variable.

SYNTAX

@INPUT (“comment”,”string”,variable 1);

comment: alphanumeric string up to 30 characters that identify the macro.

string: up to 30 characters enclosed in quotation marks (“”). This text is displayed as user prompt in the dialogue where the input is entered.

variable 1:

- string system variable STR[1 ÷ 15].
- float system variable NUM[1 ÷ 15].

SEE ALSO

@OUTPUT

EXAMPLE

@INPUT (“IN1”,”Type a value:”,STR[1]);

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@OUTPUT

File.SRG [X] File.LIB [X]

FUNCTION

It displays the content of a variable on the screen.

SYNTAX

@OUTPUT (“comment”,”string”,variable 1);

comment: alphanumeric string up to 30 characters that identify the macro.
string: up to 30 characters enclosed in quotation marks (“”).
variable 1:

- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed in the table at the end of this section.
- float system variable NUM[1 ÷ 15].
- one of the other numeric system variables listed in the table at the end of this section..

EXAMPLE

@OUTPUT (“OUT1”, ”The variable value is:”,STR[1]);

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

@SUBCOMP

File.SRG X File.LIB X

FUNCTION

It searches a string inside another one. Variable 1 specifies the string that must be searched while variable 2 specifies where the string must be searched.

SYNTAX

@SUBCOMP (“comment”,variable 1, variable 2)

comment: alphanumeric string up to 30 characters that identify the macro.

variable 1:

- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed in the table at the end of this section.
- float system variable NUM[1 ÷ 15].
- one of the other numeric system variables listed in the table at the end of this section.
- up to 30 characters enclosed in quotation marks (“”).

variable 2:

- string system variable STR[1 ÷ 15].
- one of the other alphanumeric system variables listed in the table at the end of this section.
- float system variable NUM[1 ÷ 15].
- one of the other numeric system variables listed in the table at the end of this section.

EXAMPLE

@SUBCOMP (“SEARCH”,”low”,STR[1]);
It searches the string “low” inside the content of variable STR[1]

HARDWARE REQUIREMENTS

BAD of NAIL	FLYING PROBE SYSTEMS
Protection KEY enable	Protection KEY enable

4.3.4 User Macro:

#
&
%

User macro

File.SRG [X] File.LIB [ ]

FUNCTION

Allows the integration in the test program, of all the instructions included in the called macro.

SYNTAX

#macro name (“comment”,P1,P2,...Pn)

macro name: name of the macro to look for in the library file included using the instruction described previously.
comment: name of the test (max. 30 characters) used to univocally identify the test whether the macro generates more than one. In this case the comment will include an additional ending inside the macro.
P1,P2,...Pn: parameters defined by the user according to the specific macro they refer to.

NOTE

It is possible to specify some commands after the macro using this syntax:

#macro name (“comment”,P1,P2,....Pn),command1,command2,.... command n;

In this case, the command after the macro takes precedence over those that may be present in the called macro.

It is also possible to specify some tests after the macro using this syntax:

#macro name (“comment”,P1,P2,....Pn),LAB=“comment”,command1,command2,.... command n;

In this case all parameters related to the tests with the same name inside the macro will be changed.

EXAMPLE

#TRN(“T1”,50,1,2,3);
Call to the TRN macro to test the transistor T1 (NPN)

#TRN(“T1”,50,1,2,3),TIM=1.5,MMD=V1,MES=0.6;
Call to the TRN macro that will be executed with TM=1.5ms and MES=0.6V

#TRN(“T1”,50,1,2,3),LAB=“BE DIR”,TIM=1.5;
The test T1-BE DIR inside the macro will be executed with TIM=1.5ms

User macro &

File.SRG [X] File.LIB [X]

FUNCTION

It calls a user macro that performs a digital static functional test.

SYNTAX

&macro name (“comment”, macro parameter) [,commands]

macro name: alphanumeric string up to 20 characters long that identifies the macro.
comment: alphanumeric string up to 30 characters long that identifies the name of the test.
macro parameter: parameters of the macro
commands: Functional and in-circuit commands that can be specified at the end of the macro. The commands written here have got the priority on the ones defined inside the macro with the exception on the LI5 command that is added to those already present.
The syntax of the commands can be:
&macro name(“comment”,macro parameters),commands
Example: &7408(“IC1”,1,2,3,4,5,6,7,8,9,10,11,12,13,14),TIM=1.5;

EXAMPLE

```
$include S2074.LIB;
&7400("IC12",10,11,12,13,14,15,16,17,18,19,20,21,22,23);
*The IC12 component is tested using the pattern written in the S2074.LIB library*
```

```
$include S2074.LIB;
&7400("IC12",10,11,12,13,14,15,16,17,18,19,20,21,22,23),TIM=2,DVH=4;
*The IC12 component is tested with a test time of 2ms (TIM=2) and an high level equals to 4V on the test channels*
```

User macro %

File.SRG [X] File.LIB [X]

FUNCTION

It calls a user macro that performs a digital dynamic functional test. The real time controller (DMASTER card) must be mounted in the system.

SYNTAX

%macro name (“comment”, macro parameter) [,commands]

macro name: alphanumeric string up to 20 characters long that identifies the macro.
comment: alphanumeric string up to 30 characters long that identifies the name of the test.
macro parameter: parameters of the macro.
commands: Functional and in-circuit commands that can be specified at the end of the macro. The commands written here have got the priority on the ones defined inside the macro with the exception on the LI5 command that is added to those already present. The syntax of the commands can be:
%macro name(“comment”,macro parameters),commands
Example: %7408(“IC1”,1,2,3,4,5,6,7,8,9,10,11,12,13,14),TIM=1.5;

EXAMPLE

```
$include S2074.LIB;
%7400("IC12",10,11,12,13,14,15,16,17,18,19,20,21,22,23);
*The IC12 component is tested using the pattern written in the S2074.LIB library*

$include S2074.LIB;
%7400("IC12",10,11,12,13,14,15,16,17,18,19,20,21,22,23),TIM=2,DVH=4;
*The IC12 component is tested with a test time of 2ms (TIM=2) and an high level equals to 4V on the test channels*
```

4.4 Numeric system variables

Numeric system variables	Description
NUM[1….15]	Numeric system variables. They can be defined in TEST environment pressing the Test setting button.
ERRORNR	System variable that indicates the number of errors occurred in the test. It can be used in the File.srg to check if a group of tests has failed. See the example at the end of this table.
RICNUM	Number of recycles
PREVVAL	Last measured value
NRTEST	Number of test
LAMSCALE	Lam board measurement scale
ENATEST	For BBTsystems 0 = both tests (continuity+shorts) 1 = continuity tests only 2 = short tests only
SHORT	For BBT systems 0 = adjacency 1 = unknown board 2 = good board
EXTRA	Extra down value on Z axes for through hole points

4.5 System alphanumeric variables

System alphanumeric variables	Description
STR[1….15]	System string variables that can be defined in the TEST environment, pressing Test setting button
CODPRO	Board description
OPERAT	Last operator name
BARCODE	Last barcode number readed
BATDATE	Batch date
BATCODE	Batch code
LAM	LAM board presence (y/n)
LX	System variable that indicates the type of system. (Y= GR Pilot LX system/ N= other systems) This variable can be used to execute different block of tests according to the system type. Example: if in the File.srg it is written: @COMPARE("TYPE","Y",LX),BOK="TEST1". The program will jump to TEST1 if the system is a GR Pilot LX.
POWER	Servizi board presence (y/n)
POWER1	Set power 1 (VxKO,ScaleAdjust,Reset,Volt)
POWER2	Set power 2 (VxKO,ScaleAdjust,Reset,Volt)
POWER3	Set power 3 (VxKO,ScaleAdjust,Reset,Volt)
POWER4	Set power 4 (VxKO,ScaleAdjust,Reset,Volt)
POWER5	Set power 5 (VxKO,ScaleAdjust,Reset,Volt)
POWER6	Set power 6 (VxKO,ScaleAdjust,Reset,Volt)
CHANNELS	Channels type list
USERBOARD	User board presence (y/n)
USERGEN	User Gen board presence (y/n)
SCAFP	SCAFP board presence (y/n)
HV	High voltage module presence (y/n)
250V	System variable that indicates if the HVI module (module that supply the 250V used for isolation tests) is present in the system (Y=module present /N= module not present).
AUXBOX	AUXBOX box presence (y/n)
FEMTO	FEMTO board presence (y/n)
SCAMPO	SCAMPO board presence (y/n)
DMASTER	DMASTER board presence (y/n)
CMP	PROBAZ board presence (y/n)
HANDLER	Automatic board handler presence (y/n)
MULTIPLE	One character for each board. “E” to enable, “D” to disable the test
USERBOARD2	User board 2 presence (y/n)
USERGEN2	User Gen 2 board presence (y/n)

Example of use of the **ERRORNR** system variable:

In the following File.srg there are two groups of tests: the RESISTOR test group and the CAPACITOR test group. The program jumps to the label “AFTER” if all tests in the CAPACITOR test group have failed.

```
…
…                                   RESISTOR test group
…
…
…                                   CAPACITOR test group (composed of 4 tests)
…
@COPY(“E3”,ERRORNR,NUM[1]);  Storage of ERRORNR value in NUM[1]
…
…
…
@COMPARE(“E4”,4,NUM[1]),BER=”AFTER”,NER=Y;
```

Comparison of the value or ERRORNR previously stored with 4 that is the number of tests of the CAPACITOR GROUP. If all 4 tests have failed the program jumps to AFTER.

```
…
…
…
LAB=”AFTER”,NER=Y;
…
…
```

Section 5

5. MACRO DEVELOPMENT

The in-circuit library file in addition to the analog macros, contains also the information related to the input, output and bi-directional pin of the board components. Such information is used for automatic software procedures like the diagnostic file generation, the digital and analog guarding generation, the multiple boards creation, etc.
The library file contains the description of the various components where each component respects the syntax illustrated below. In the library file, each line is terminated by the ";" character while the "*" character indicates a comment line. It is possible to give a carriage return (CR) after the "," or ";" characters.

SYNTAX

#component name (comment, pin1,pin2,...);

#component name:	alphanumeric string of 20 characters that identifies the component. This name must be the same as the one written in the board model File.nod.
comment:	parameter that indicates a comment
pin1,pin2,...:	Component pin list in which the VCC and GND pins are included. Usually pins from the list are labeled as P1,P2,P3,... but they can also be specified with other names.

!comp. type,nr. of component section,number of component pin;

!comp. type.:	component type that can be: A: analog component D: digital component X: indeterminate component
nr. of component section:	it indicates the number of internal sections of the component.
number of component pin:	total number of component pins.

!nr. of pin for section, nr. of input pin, nr. of output pin, nr. of bi-directional pin, nr. of enable pin, input pin list, output pin list, bi-directional pin list, enable pin list, section enable formula, section disable formula;
…

! nr. of pin for section:	umber of pin for each internal section of the component
nr. of input pin:	number of input pins for each section of the component
nr. of output pin:	number of output pins for each section of the component
nr. of bi-directional pin:	number of bi-directional pins for each section of the component. If there is no bi-directional pin, write zero (0) as pin number.
nr. of enable pin:	number of enable pins for each section of the component. If there is no enable pin, write zero (0) as pin number.
input pin list:	input pin list, pins indicated here must have the same name as the ones used in the component pin list (P1,P2,...).
output pin list:	output pin list, pins indicated here must have the same name as the ones used in the component pin list (P1,P2,...).
bi-directional pin list:	bi-directional pin list, pins indicated here must have the same name as the ones used in the component pin list (P1,P2,...). If there is no bi-directional pin, this field must be ignored.
enable pin list:	enable pin list, pins indicated here must have the same name as the ones used in the component pin list (P1,P2,...). If there is no enable pin, this field must be ignored and the enable/disable formulas are not described.
section enable formula:	This indicates the section enable formula and has the following syntax: Pin = L o l (indicates a low logic pin level) H o h (indicates a high logic pin level) The various pin enable conditions can be combined together using the following operators: & = AND, | = OR **Example:**(P1=l&P2=H) or (P4=L|P9=L)
section disable formula:	This indicates the section disable formula. The same rules described for the enable formula are valid.

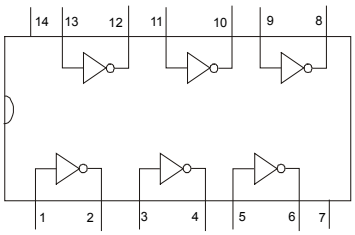
!nr. of pin for section *i*, nr. of input pin, nr. of output pin, nr. of bi-directional pin, nr. of enable pin, input pin list, output pin list, bi-directional pin list, enable pin list, section enable formula, section disable formula;

NOTE

In the library file, it can happen that the internal section description of the component doesn't reflect the real component architecture but the logic one. As an example we consider the 74245 component that is described ad if it were made up of 16 blocks with one input pin, one bi-directional pin and two enable pins each. This description is apparently different from the real one in which there are 16 bi-directional buffers and 2 AND gates.

EXAMPLES

Library description for the 7404 component.



There are no bi-directional and enable pins in the component, hence the related fields are filled in with a zero (0). Obviously the enable/disable formulas are not described.

#7400(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14);
!D,6,14; show that is a digital component, with 6 sections and total number pins is 14.
!2,1,1,0,0,P1,P2; show section 1, 1 INPUT pin, 1 OUTPUT pin, 0 bi-directional pin, 0 enable pin, P1 is an INPUT pin, P2 is an OUTPUT pin.
!2,1,1,0,0,P3,P4; show section 2, etc…
!2,1,1,0,0,P5,P6; show section 3, etc…
!2,1,1,0,0,P9,P8; show section 4, etc…
!2,1,1,0,0,P11,P10; show section 5, etc…
!2,1,1,0,0,P13,P12; show section 6, etc…

7474 component description:

#7474(COMM,P1,P2,P3,P4,P5,P6,,P8,P9,P10,P11,P12,P13,);
!D,2,14;
!6,4,2,0,0,P1,P2,P3,P4,P5,P6;
!6,4,2,0,0,P10,P11,P12,P13,P8,P9;

74125 component description:

#74125(COMM,P1,P2,P3,P4,P5,P6,,P8,P9,P10,P11,P12,P13,);
!D,4,14;
!3,1,1,0,1,P2,P3,P1,(P1=L),(P1=H);
!3,1,1,0,1,P5,P6,P4,(P4=L),(P4=H);
!3,1,1,0,1,P9,P8,P10,(P10=L),(P10=H);
!3,1,1,0,1,P12,P11,P13,(P13=L),(P13=H);

74214 component description:

#74241(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,,P11,P12,P13,P14,P15,P16,P17,P18,P19,);
!D,8,20;
!3,1,1,0,1,P2,P18,P1,(P1=L),(P1=H); Section 1 is enabled when P1 is at a logic low level and disabled when it is high
!3,1,1,0,1,P4,P16,P1,(P1=L),(P1=H);
!3,1,1,0,1,P6,P14,P1,(P1=L),(P1=H);
!3,1,1,0,1,P8,P12,P1,(P1=L),(P1=H);
!3,1,1,0,1,P11,P9,P19,(P19=H),(P19=L);
!3,1,1,0,1,P13,P7,P19,(P19=H),(P19=L);
!3,1,1,0,1,P15,P5,P19,(P19=H),(P19=L);
!3,1,1,0,1,P17,P3,P19,(P19=H),(P19=L);

74245 component description:

#74245(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,,P11,P12,P13,P14,P15,P16,P17,P18,P19,P20);
!D,16,20;
!4,1,0,1,2,P18,P2,P1,P19,(P1=L & P19=L),(P19=H | P1=H & P19=L); Section 1 is enabled when P1 is at a low logic level. The section is disabled when P1 is at a high logic level.
!4,1,0,1,2,P17,P3,P1,P19,(P1=L & P19=L),(P19=H | P1=H & P19=L);

```
!4,1,0,1,2,P16,P4,P1,P19,(P1=L & P19=L),(P19=H  | P1=H & P19=L);
!4,1,0,1,2,P15,P5,P1,P19,(P1=L & P19=L),(P19=H  | P1=H & P19=L);
!4,1,0,1,2,P14,P6,P1,P19,(P1=L & P19=L),(P19=H  | P1=H & P19=L);
!4,1,0,1,2,P13,P7,P1,P19,(P1=L & P19=L),(P19=H  | P1=H & P19=L);
!4,1,0,1,2,P12,P8,P1,P19,(P1=L & P19=L),(P19=H  | P1=H & P19=L);
!4,1,0,1,2,P11,P9,P1,P19,(P1=L & P19=L),(P19=H  | P1=H & P19=L);
!4,1,0,1,2,P9,P11,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P8,P12,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P7,P13,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P6,P14,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P5,P15,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P4,P16,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P3,P17,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
!4,1,0,1,2,P2,P18,P1,P19,(P1=H & P19=L),(P19=H  | P1=L & P19=L);
```

Section 6

6. TEST PROGRAM DEVELOPMENT

The program development is divided into two different procedures because it depends if CAD data are available or not. Both procedures will be described below.

The following paragraphs shows the basic steps needed to develop a test program so that the user can see the whole procedure. It is important to note that:

- In the two procedures, there are a lot of references to specific software menus whose detailed description can be found in the S20WIN Environment manual.
- Some steps can be automatically obtained with the "wizard" procedure that can be enabled from the *BOARD\Select*: *menu.* The whole listing of all steps is given here to help the user to understand the full architecture of the procedure that normally is hidden or implicit inside the wizard.
- Sometimes there can be more that one way to obtain a result. Here is given one possibility.

6.1 Test program development without cad data

The following operations are those suggested for the development of a test program on the test system when the values of the coordinates of the board test points are not available. To ensure a simple, linear development we recommend that the user follow the work method described below; however, it is not a limitation or restriction, in that the flexibility of the test system allows the user complete freedom to carry out test development in whatever way best suits his needs.

Necessary:

- Bare board circuit of board under test.
- Three or more golden board to use as reference and for debug the program..
- Electrical diagram and board layout.
- Part list.
- Values, tolerances required by the test program, data sheets and functions of any special components.
- Transparent tracing paper.
- Fine-point, indelible marker.
- Fine-point white-out pen.

Preparation

1. Examine the board and decide which side use for the test, considering that the chosen board side gives the face to the bad of nail fixture or the mobile probes, depending by the system used.
2. Examine the schematics and assign one or more test channel to each network, giving it a progressive number.
   - Make sure that the assigned test points are always accessible by the side chosen for the test. Once this operation has been completed, one or more channels should correspond to each network.
3. Cover the probe side of the printed circuit board with the tracing paper, and fix it into place to keep it from shifting. Using the indelible pen, mark the positions of the channels on the paper, writing the number next to each point. When assigning the channels, it is necessary to take the following rules into consideration:
   - select points that are easily accessible, that is, without any particular mechanical restrictions.
   - if possible, select points on the network that allows good contact by the mobile test probe.

4. Using the white-out pen, mark the same channel points on the printed circuit board as the ones marked on the tracing paper.

Board creation

1. Select menu BOARD\Select
2. In the “New Board” section, type the new board name
3. Press the CREATE button and answer YES to the message “Create a New Program?” . Type in the board thickness expressed in mm.

6.1.1 FLYING PROBE SYSTEMS

6.1.1.1 Board configuration

1. Select BOARD\Configure and assign the “Flying” type of fixture.
2. Press the button to configure the Z Axis. Set the Up, down, near position and board thickness according to the board and the room of its components.
3. Set the other configuration parameters. Save and exit form the Configuration menu.

6.1.1.2 Autolearn the coordinates

1. Place the board in the work area.

2. Select FLYING PROBES\Autolearn.
3. Select: Insert mode: Channel - Image Autolearn: Manual – Unit of measure: Mm.
4. Using the joystick/direction buttons, move the camera to the test point previously marked and autolearn its coordinates.
   - Use the transparent tracing paper or the marks on the board to identify the test points.
   - It is advisable NOT to remove the board from the work area during the autolearn process to have more precision. If for any reason the board is removed, it is necessary to execute an automatic translation of the board coordinate placed in the new position before going on with the autolearn process. This is important because in a practical way, it is impossible to place the board in the same position twice.
   - After the autolearn process is finished, the file .par, the file .nod and the XY are generated. The fist two can be modified if necessary (refer to chapter 8 for their structure) while the file.XY is an object file used to execute the test.

6.1.1.3 Modification of the file.par (that contains the component parameters)

1. With the BOARD\Open menu, select the file .par and insert the component list together with their value and characteristics.

6.1.1.4 Generation of the database (file.SHO) for the short test

1. Select the CAD\Short Data Generation menu
2. Choose: Overwrite previous data, Adjacent Pads and write a value expressed in mm in the Clearance filed. This value indicates the area around a test point that must be considered for short test. Example: if the majority of points on a board are 100mils distant, the suggested value for the clearance is 101 mils so that all adjacent points will be included. This procedure generates the file .SHO that contains all data for short test.

6.1.1.5 Creation of the source file (File.srg)

Select the CAD\Generate menu, he suggested options are:
- Use Channels
- In Circuit test
- Short test - Impedance 5 Ohm - Voltage 0,5V - Adjacency Low Voltage

At the end of this step, the file (.srg) is generated. This file can be modified if necessary, using the BORD\Open menu.

6.1.1.6 Optimization of the test program

1. Choose the TOOLS\Software menu
2. Choose the desired options, the suggested ones are:
   - **Board Library Generation** (to create a board library extracting data from the system libraries).
   - **Parallel finder** (to automatically calculate the value of components connected in parallel).
   - **Capacitor discharge** (To automatically write the capacitor discharge macros at the beginning of the test program).
   - **Analog guarding** (to automatically insert the guarding points).

6.1.1.7 Compilation of a test program

1. Compile the File.srg by choosing the RUN\Compile menu.

NOTE: for an example of file .nod, .par and .srg, refer to the following chapter where the description of the board and system file can be found.

6.1.1.8 Execution of a test program

1. Choose the RUN\Test menu to enter the test environment.
2. Press RUN to execute the test program.

6.1.1.9 Debug of a test program

1. Choose the RUN\Debug menu.
2. Choose the test to be examined.
3. Press RUN to execute the test.

Any modification of the test parameters in this environment can be saved in the File.srg, when leaving the debug session.

6.1.2 BED of NAIL SYSTEMS

6.1.2.1 Configuration of the board

1. Select BOARD\Configure and assign the “Manual” or “Automatic” or “Pneumatic” type of fixture according to the one you are using.

6.1.2.2 Creation of the source file (File.srg)

1. Select the BOARD\New menu. Choose the Source Programs (*.srg) option. An empty File.srg will be opened.
2. Write the File.srg containing the measurements to be executed on the board under test. Refer to the relative section for the different testing methods and for the structure of the File.srg

6.1.2.3 Optimization of the test program

1. Choose the TOOLS\Software menu
2. Choose the desired options, the suggested ones are:
   - **Board Library Generation** (to create a board library extracting data from the system libraries).
   - **Parallel finder** (to automatically calculate the value of components connected in parallel).
   - **Capacitor discharge** (To automatically write the capacitor discharge macros at the beginning of the test program).
   - **Analog guarding** (to automatically insert the guarding points).

6.1.2.4 Compilation of a test program

1. Compile the File.srg by choosing the RUN\Compile menu

6.1.2.5 Execution of a test program

1. Choose the RUN\Test menu to enter the test environment.
2. Press RUN to execute the test program

6.1.2.6 Debug of a test program

1. Choose the RUN\Debug menu
2. Choose the test to be examined. Press OK
3. Press RUN to execute the test

Any modification of the test parameters in this environment can be saved in the File.srg, when leaving the debug session.

6.2 Test program development with cad data

When CAD data are available, it is possible to generate the source file (File.srg) automatically. The following operations describe the development of a test program using data from CAD.

You need:

- Three or more known good boards to be used as reference and to debug the program correctly
- Bare printed circuit board of the board under test
- Electrical diagrams and layout of the board under test
- List of the board components
- Values, tolerances required by the test program, data sheets and functions of any special components
- Netlist file of the board.

Examine the board under test and decide which side of the board will be the test side.

6.2.1 FLYING PROBE SYSTEMS

6.2.1.1 Creation of the board

1. In the software environment, select the BOARD\Select menu.
2. In the “New Board” section, type the name of the new board

Press the CREATE button and answer YES to the message “Create a New Program?”. Type in the board thickness expressed in mm.

6.2.1.2 Configuration of the board

1. Select BOARD\Configure and assign the “Flying” type of fixture.
2. Press the button to configure the Z Axis. Set the Up, down, near position and board thickness according to the board and the room of its components.
3. Set the other configuration parameters. Save and exit form the Configuration menu.

6.2.1.3 Import data from CAD environment

1. Select the CAD\Import menu
2. Select the cad converter
3. Select the files used to import the requested information. The file.par and file .nod are automatically generated. If there are no coordinates in the file.nod, check the CAD files.

6.2.1.4 Generation of the database (file.SHO) for the short test

1. Select the CAD\Short Data Generation menu
2. Choose: Overwrite previous data, Adjacent Pads and write a value expressed in mm in the Clearance filed. This value indicates the area around a test point that must be considered for short test. Example: if the majority of points on a board are 100 mils distant, the suggested value for the clearance is 101 mils so that all adjacent points will be included. This procedure generates the file .SHO that contains all data for short test.

6.2.1.5 Generation of the source file (File.srg)

1. Select the CAD\Generate menu, he suggested options are:
   - Use Channels
   - In Circuit test
   - Short test - Impedance 5 Ohm - Voltage 0,5V - Adjacency Low Voltage

   At the end of this step, the file (.srg) is generated. This file can be modified if necessary, using the BORD\Open menu.

6.2.1.6 Optimization of the test program

1. Choose the TOOLS\Software menu
2. Choose the desired options, the suggested ones are:
   - **Board Library Generation** (to create a board library extracting data from the system libraries).
   - **Parallel finder** (to automatically calculate the value of components connected in parallel).
   - **Capacitor discharge** (To automatically write the capacitor discharge macros at the beginning of the test program).
   - **Analog guarding** (to automatically insert the guarding points).

6.2.1.7 Compilation of a test program

1. Compile the File.srg by choosing the RUN\Compile menu

NOTE: for an example of file .nod, .par and .srg, refer to the following chapter where the description of the board and system file can be found.

6.2.1.8 Board orientation check

1. Select the FLYING PROBES\View board Layout menu. Check that the orientation of the points displayed corresponds to the orientation of the board in test.
2. Select the FLYING PROBES\TRANSLATION\Manual menu if a flip or rotation of the board is necessary.

6.2.1.9 Translation of the coordinates

1. Place the board in the work area with the correct orientation.
2. Select the FLYING PROBES\View Board Layout menu.
3. Choose two reference points on the board layout that will be used for the board translation (TP1 e TP2). If possible, choose two points on the board diagonals.
4. Select the FLYING PROBES\TRASLATION\Automatic menu. Move the camera with the joystick to the first reference point chosen for the translation.
5. Press the "Channel" button. Select the name of the first reference point and press "Save Ch. 1" to save its coordinates.
6. Move the camera with the joystick to the second reference point chosen for the translation.

7. Press the “Channel”.. button. Select the name of the second reference point and press the “Save Ch. 1” button to save its coordinates.
8. Press EXIT.
9. Press OK.
10. After the translation has been done, it is better to check some coordinates just to ensure that the translation is OK.. To do that, select the FLYING PROBES\Autolearn menu. Select a test point and press GO. Check that the crosshair is exactly in the test point center. Repeat this operation for 5-10 points.

6.2.1.10 Execution of a test program

1. Choose the RUN\Test menu to enter the test environment.
2. Press RUN to execute the test program

6.2.1.11 Debug of a test program

1. Choose the RUN\Debug menu
2. Choose the test to be examined. Press OK
3. Press RUN to execute the test

Any modification of the test parameters in this environment can be saved in the File.srg, when leaving the debug session. Refer to the following paragraph for debug suggestions.

6.2.2 BED OF NAILS SYSTEMS

6.2.2.1 Creation of the board

1. Select BOARD\Select and create the board under test.

6.2.2.2 Configuration of the board

1. Select BOARD\Configure and assign the “Manual” or “Automatic” or “Pneumatic” type of fixture according to the one you are using.

6.2.2.3 Import data from CAD environment

1. In the S20 Windows environment, select CAD\IMPORT, select the format of the requested CAD and import the file/s.
2. The S20 Windows automatically generates the File.par and File.nod.

6.2.2.4 Generation of the source file (File.srg)

1. Select CAD\Generate, choose the desired options and press OK

6.2.2.5 Optimization of the test program

1. Choose the TOOLS\Software menu
2. Choose the desired options, the suggested ones are:
   - Board Library Generation (to create a board library extracting data from the system libraries
   - Parallel finder (to automatically calculate the value of components connected in parallel
   - Capacitor discharge (To automatically write the capacitor discharge macros at the beginning of the test program)

- Analog guarding (to automatically insert the guarding points)

6.2.2.6 Compilation of a test program

1. Compile the File.srg by choosing the RUN\Compile menu

6.2.2.7 Board orientation check

1. Select the FLYING PROBES\View board Layout menu. Check that the orientation of the points displayed corresponds to the orientation of the board in test.
2. Select the FLYING PROBES\TRANSLATION\Manual menu if a flip or rotation of the board is necessary.

6.2.2.8 Execution of a test program

1. Choose the RUN\Test menu to enter the test environment.
2. Press RUN to execute the test program

6.2.2.9 Debug of a test program

1. Choose the RUN\Debug menu
2. Choose the test to be examined. Press OK
3. Press RUN to execute the test

Any modification of the test parameters in this environment can be saved in the File.srg, when leaving the debug session.

6.3 Test program debug

Once the test program has been created, the programmer must execute it on a known good board and he needs to debug it (to insert some guarding points, to increase the test time etc). There are two possible ways to follow:

Autodebug: (to leave automatically debug it by the test system). This method is basically used when there isn't a lot of time to debug a program. We have to keep in mind that normally, after the debug phase, the programmer needs always to check the result of the procedure and maybe to adjust manually some tests. To execute the autodebug, refer to Chapter 4, paragraph TOOLS\HARDWARE\Autolearn/Debug.

Manual: the programmer examines each single test and decides what to do. Here it follows some suggestions that can be useful during the debug phase. Of course they are not unique because each single component must be evaluated in its context in the board but these instructions are a general guide to execute the debug steps on one board.

6.3.1 General suggestion for the manual debug

As general suggestion in the debug phase it is always better to:

- Check for the component value
- Check if the component is mounted on the board

In the measurement:

- The low measurement limit must not be < 0.15V
- The high measurement limit must not be > 2V

- The expected value must not be greater that the voltage levels on the drivers (DR1, DR2)

The measurement tolerance must be greater that the component tolerance

6.3.1.1 To debug a RESISTOR

If the value read is **lower** than the expected one:

- Check for parallel components
- Place a guarding point/s
- Increase the test time (if a resistor is in parallel to a capacitor that needs to be charged)

If the value is greater than the expected one:

- Check if the channels are the correct ones
- Check if channels make a good contact on the resistor pins
- Try to invert the two channels (in case the test system is injecting current on a GND/VCC network that has a lot of dispersion points)
- Increase the test time (in case of overshoots)

To increase accuracy:

- Stabilize the read value writing the NCR,MRC commands
- Use the @RESV macro to test the resistor at a different voltage value than 0.3V
- Guard the VCC or GND if the resistor is connected directly to an IC.
- Use 4-wire resistor measurement for resistors below 100□.

6.3.1.2 To debug a CAPACITOR (using the #CON, #CAP macro)

As a general rule:

- Check if the channels are the correct ones
- Check for parallel components
- When we need to modify the current or time values for a measurement, we have to keep in mind that they must vary proportionally so that its product is always constant (that is: if we multiply the current by 2, we have to divide the time by 2) otherwise the value of capacitor measured will be different from the one written in the macro.
- If it is not possible to have a correct capacitor value, use the file .con. Refer to Chapter 8 for its description.

If the value read is **lower** than the expected one:

- Check if channels make a good contact on the resistor pins
- If the value of the capacitor is lower that the expected one but you see it charging, increase the current and decrease the time.
- Try to invert the two channels (in case the test system is injecting current on a GND/VCC network that has a lot of dispersion points)

If the value read is **greater** than the expected one.

- Place a guarding point/s
- If the value of the capacitor is greater that the expected one but you see it charging, decrease the current and increase the time.
- If you can't get the capacitor value, use the File.con

6.3.1.3 To debug a CAPACITOR (using the #CON_AC)

- Check if the channels are the correct ones
- Add or erase some guarding points
- Modify the expected voltage (using the #CON_ACV macro)
- Modify the expected frequency or Rg (using the #CON_ACF macro)
- Check for parallel components

6.3.1.4 To debug a CAPACITOR (using the @CON macro)

- Check if the channels are the correct ones
- Insert guarding points

- Use the autodebug procedure

6.3.1.5 To debug a INDUCTOR (using the #IND macro)

- Check if the channels are the correct ones
- Examine the voltage curve of the inductor with the software integrated scope and if necessary, vary voltage and/or time to be sure the measurement is in the linear zone of the curve.

6.3.1.6 To debug a INDUCTOR (#IND_AC macro)

- Add or erase some guarding points
- Modify the expected voltage (using the #CON_ACV macro)
- Modify the expected frequency or Rg (using the #CON_ACF macro)
- Check for parallel components

6.3.1.7 To debug a DIODE

- Check if the channels are the correct ones
- Check the channel order (A/K) and if necessary, invert them
- Vary the test voltage if the diode requires a voltage different than 0.6V
- Increase the test time (i.e. diode in parallel to a capacitor)
- Change the expected frequency value or the resistor in series to the generator (using the #IND_ACF).
- Check for parallel components

6.3.1.8 To debug a TRANSISTOR

- Check if the channels are the correct ones
- Check the channel order and make sure it is E/B/C

6.3.1.9 To debug a MOS

- Check for missing component
- Check if the channels are the correct ones
- Check the channel order and make sure it is D/G/S

6.3.1.10 To debug a RELAY

- Check the channel order in the macro

Section 7

7. UNDER BOARD TEST and SYSTEM FILES

All files related to a test program are located under the board directory. Some of this files are object files not accessible to the programmer but others can be modified according to the needs. In this section is given the format of the most common files that are:

File.adj
File.aut
File.con
File.lib
File.nod
File.par
File.pol
File.shi
File.sho
File.srg
File.tab

The following files are not listed in the board directory. They are under the \S20WIN\Board directory because they are system files.

S20.adj
S20.aut
S20.cad
S20ANA.lib
S20UGEN.LIB
S20USER.LIB
S2074.LIB
S20LOG.LIB
S20FUN.LIB

7.1 The .ADJ file

This file is used during the autoadjustment procedure (TOOLS\HARDWARE\Autoadjustment menu) that permits to adjust the range of the different measurements running the test program and auto learning the values on a batch of good boards. The file contains the rules to be used during the autoadjustment process.
The example listed below illustrates the rules used in the autoadjustment process for the #RES macro (macro used to test a resistor).
The instructions LAB, MAXGUA, SAVE, WARN, GUA are used to insert a guarding point, to execute a test and store the result, to define a test label.

AUT	Y/N	Y= autodebug enable N= autodebug disable
FUL	Y/N	Y= FULL mode enable N= PARTIAL mode disable
LRN	Y/N	Y= autolearn enable N= autolearn disable
MTT	[ms]	Maximum Test Time Value
DEB	Y/N	Y= autolearn/debug run from DEBUG environment N= autolearn/debug run command not run from DEBUG environment

The example below shows the rules used during the “autoadjustment” process for the #RES macro (macro used to test a resistor)
The LAB, MAXGUA, SAVE, WARN, GUA instructions are used to insert a guarding point, to execute a test and to save the result, to define a label for the test.

```
#RES(COMM,VAL,PERC,P1,P2)
LOCAL P,VA,C,G,OK,D;
P=PERC/2;
OK=0;
LAB=COMM;
HIG=MES*(1+(P/100));
LOW=MES*(1-(P/100));
MAXGUA(G);
GUA=0;
if(G>0)
 for (C=1; C<=G+1; C=C+1;)
  EXEC(VA);
  if(VA<LOW)
   GUA=+(C);
   OK=-1;
  else
   if(VA>HIG)
    if(C=1)
    WARN=COMM+" HIGH VALUE";
     OK=1;
     C=G+2;
    else
     D=C-1;
     GUA=-(D);
     if (C<G+1)
      GUA=+(C);
     endif
    endif
   else
    C=G+2;
    OK=0;
   endif
```

```
  endif
 endfor
endif
if (OK=-1)
 for (C=1; C<=10; C=C+1;)
  D=VA;
  TIM=TIM*5;
  EXEC(VA);
  D=VA-D;
  if (D<.01)
  WARN=COMM+" LOW VALUE";
   C=11;
  else
   if (VA>LOW)
    OK=0;
    C=11;
   endif
  endif
 endfor
 if (OK=0)
  HIG=MES+(MES*PERC)/100;
  LOW=MES-(MES*PERC)/100;
  SAVE();
 endif
endif
HIG=MES+(MES*PERC)/100;
LOW=MES-(MES*PERC)/100;
SAVE();
```

7.2 The .AUT file

This file is used during the “autodebug” procedure (Menu TOOLS\HARDWARE\Autolern/Debug) where the system tries to modify the test parameters automatically when a component test has failed. In the file.aut is possible to use some variables containing values defined in the *Autolearn* window.

AUT	Y/N	Y= autodebug has been selected N= auto debug has not been selected
FUL	Y/N	Y= FULL mode has been selected N= PARTIAL mode has been selected
LRN	Y/N	Y= autolearn has been selected N= autolearn has not been selected
MTT	[ms]	Test Time maximum value
DEB	Y/N	Y= autolearn/debug has been executed from the DEBUG environment N= autolearn/debug has not been executed from the DEBUG environment

The example here below shows the rules used during the “autodebug” phase for the #RES macro (used to test a resistor).
The LAB, MAXGUA, SAVE, WARN, GUA instructions are used to insert a guarding point, to execute a test and store the result, to define a label for the test

Example:

#RES(COMM,VAL,PERC,P1,P2)

```
LOCAL P,VA,HI,LO,W,TEMP,I,DIF,C,CC,CCC,DIFF,DIFTEM,G;

* TEST WITH ½ PERCENTAGE
LAB=COMM;
P=PERC/2;
W=0;
TEMP=0;
HI=MES*(1+(P/100));
LO=MES*(1-(P/100));
MAXGUA(G);
GUA=0;
* LOOP MAX. 20 TIMES
for ( I=1; I<20; I=I+1; )
  EXEC(VA);
  if (VA > HI)
    if (W =0)
      W=1;
      G=0;
    endif
    if (DRI>.00005)
* IF THE CURRENT IS NOT MIN., DIVIDE BY 10
      DRI=DRI/10;
      if (DRI<.00005)
            DRI=.00005;
      endif
* INCREASE THE TIME WHILE DECREASING THE CURRENT
      TIM=5/(DRI*1000);
      if (TIM>60000)
            TIM=60000;
      else
            if (TIM<.1)
              TIM=.1;
            endif
      endif
    else
** OPEN
      MES=.5;
      LOW=.4;
      HIG=1.5;
      I=100;
      W=2;
    endif
  else
    if (VA < LO)
      if(G>0)
***************************  GUARDING ****************************
****** FIRST GUARDING
            if (TIM<1)
              TIM=1;
            endif
            DIFF=MES-VA-.01;
            for (C=1; C<=G; C=C+1;)
              GUA=+(C);
              EXEC(VA);
              if(VA<LO)
                DIFTEM=MES-VA;
                if (DIFTEM<DIFF)
                  DIFF=DIFTEM;
                else
                  GUA=-(C);
```

```
            endif
          else
            if(VA<HI)
              C=G+1;
              I=100;
            else
              GUA=-(C);
            endif
          endif
        endfor
        G=0;
************************ END GUARDING ROUTINE
        else
        DIF=VA-TEMP;
        TEMP=VA;
        if (DIF>.01)
* IF V> 10mV, ICREASE THE TIME
          if (TIM>12000)
            TIM=60000;
          else
            TIM=TIM*5;
          endif
        else
* STORE LOW VALUE BECAUSE THE CURRENT CHANGES
          if (W =0)
            TIM=TIM/5;
            W=3;
          endif
          if (VA<=0)
            DRI=101;
          else
            DRI=(MES*DRI)/VA;
          endif
          if (DRI>100)
            MES=(MES*100)/DRI;
            DRI=100;
            if (MES < .5)
* SHORT CIRCUIT
              HIG=.6;
              LOW=-1;
              TIM=1;
              I=100;
              W=4;
            endif
* LOW RESISTANCE - CRITICAL CONDITION
            HI=MES*(1+(P/100));
            LO=MES*(1-(P/100));
          endif
        endif
    endif
   else
    I=100;
   endif
  endif
endfor

switch (W)
  case ( 1 )
   WARN=COMM+" HIGH VALUE";
  case ( 2 )
```

```
    WARN=COMM+" OPEN";
  case ( 3 )
    WARN=COMM+" LOW VALUE";
  case ( 4 )
    WARN=COMM+" SHORT";
endswitch
if (TIM>1000)
  WARN=COMM+" TEST TIME LONGER THAT 1 SECOND";
endif

SAVE ();
```

7.3 The .CON file

This file is generated during the wiring capacity autolearn to learn the capacitive value of the fixture cables (menu TOOLS\FIXTURE\Wiring Capacity Autolearn). These autolearned values will be subtracted to the measured value for those capacitors with value lesser than 40nF to increase the measurement precision. Values are stored in the file.con with this syntax:

“Channel”=”Autolearned capacitive value”

Example:

“C4”=5
“C9”=700
“C15”=5
“C19”=700
“C20”=100
“C24”=60
...
...
where: values are expressed in pF.

7.4 The .LIB file

The file.lib contains all macros that can be used in the file.srg. There are some system libraries (S20ANA.lib, S2074.lib,...) under the S20WIN\BOARD directory. The user anyway can develop its own libraries containing the requested macros. In this case the user library is under the board directory. The library file/files are then included into the file.srg using the $include LIBRARY NAME.LIB instruction.
Here below is a library listing. To crate a new user library, refer to section 6.

Example:

```
GLOBAL CONTOL,CAPTOLP,CAPTOLM,ZENTOL,CABIMP=0;

#DISCAP(COMM,VAL,P1,P2)
BEE=N,VIW=N,STP=N;
ALM=0X00,ELM=0XFF,PUM=N;
MMD=V4,VIM=V;
NRC=0,MRC=M;
DR2=0,D2P=N;
LN4=3;
LI1=0,LI2=P2,LI3=0,LI4=P1;
TIM=10,NRC=4000,MRC=J,MES=.3,HIG=.5,LOW=-.5;
```

```
LAB=COMM+" SC1";
MES=.01,HIG=.05,LOW=-.05;
LN4=1;
LAB=COMM+" SC2";
NRC=0,MRC=M,NER=Y;
TIM=10;
LI2=P1-P2,LI4=0;
LAB=COMM+" SC3";

#BOARD(COMM,P1,P2)
NER=N,BEE=N,VIW=N,STP=N;
ALM=0X00,ELM=0XFF,PUM=N;
MMD=V4,VIM=V;
DRM=I,DRV=.5,DRI=10;
TIM=1;
LOW=-.5,MES=.1,HIG=.25;
LN1=0,LN2=0,LN3=0,LN4=0;
LI1=P1,LI2=P2,LI3=0,LI4=P1;
LAB=COMM;

#RESV(COMM,VOLT,VALO,PERC,P1,P2)
!A,1,2;
!2,0,0,2,0,P1=1,P2=2;
LOCAL VAL;
VAL=VALO+CABIMP;
NER=N,BEE=N,VIW=N,STP=N;
ALM=0X00,ELM=0XFF,PUM=N;
VIM=O;
DRM=I,DRV=1.1,DRI=(VOLT/VAL)*1000;
if (DRI>100)
  DRI=100;
  MES=VAL*.1;
else
  MES=(DRI*VAL)/1000;
endif
TIM=VAL*.000025;
if (TIM<.1)
  TIM=.1;
endif
if (VAL<100)
  MMD=D;
else
  MMD=V4;
  if (VAL>10000)
    MES=(DRI*VAL)/1000;
  endif
endif
LOW=MES-(MES*PERC)/100;
HIG=MES+(MES*PERC)/100;
LN1=0,LN2=0,LN3=0,LN4=0;
LI1=P1,LI2=P2,LI3=0,LI4=P1;
LAB=COMM;
```

7.5 The .PAR file

It contains the list of components mounted on the UUT.

Example of a File.par

*COMPONENT	DEVICE	VALUE	UNIT	TOL+	TOL-	NR.PIN	MACRO
C46	0.33UF	0.330000	UF	80.00	20.00	2	CAP
R5	1.2	1.200000	K	20.00	0.00	2	RES
R6	1.2	1.200000	K	5.00	0.00	2	
R91	100K	100.000000	K	10.00	0.00	2	
R98	RES0912	100	O	5.00	5.00	2	$RESISTOR
C46	0.33UF	0.330000	UF	80.00	20.00	2	
C5	100NF	100.000000	NF	80.00	20.00	2	$CAPACITOR
IC12	7400	0.000000	V	0.00	0.00	14	7400
IC19	7400	0.000000	V	0.00	0.00	14	7400

Data in this file are organized into fields separated by one or more characters that can be blanks or TAB.
Comment lines begin with an (*) and end with a (CR+LF).
It is advised to fill all fields except the MACRO one that can be empty because the correspondence between the component and the macro to use during test is written in the S20.CAD file. If the MACRO field is filled with a value, the value will have the priority on the one in the S20.cad file.

NOTE: The SPACE and TAB characters mustn't be present as values because they are separator characters.

COMPONENT: (max 30 characters) it contains the commercial component name.

DEVICE:(max 20 characters) it contains the commercial name of the component (Example: RESISTOR, CAPACITOR

VALUE: (floating point number) it contains the component value.

UNIT: (max. two characters) indicated the unit of measure of the component:

Voltage [Volt]	Resistors [Ohm]	Capacitors [Farad]	Inductances [Henry]
V [V]	O [Ω]	F [F]	H [H]
	M [MΩ]	m [mF]	
	MO [MΩ]	u [µF]	
	K [KΩ]	n [nF]	
	KO [KΩ]	p [pF]	
		mF [mF]	
		uF [µF]	
		nF [nF]	
		pF [pF]	

TOL+: (floating point number) indicates the positive tolerance of the component

TOL-: (floating point number) indicates the negative tolerance of the component

NRPIN: (integer number) number of component pin

MACRO:(max 20 character) this field contains the name of the macro that will be used to test the component (the macro will be listed in the file.srg). This field can be:

1) Empty, in this case the value taken into account will be the one listed in the S20.cad file (file that contains the correspondence between the component and the test macro).

Example: R45 –#RES macro. The component value is also important to choose the correct macro inside the S20.cad.
Example: R37 1Ohm – macro #RES4.

2) the name of a section present in the S20.CAD. Example: $RESISTOR. In this case the test macro will be chosen between the macros listed inside that section only.
Example: MN45 1Kohm $resistor – macro #res MN34 1Ohm $resistor – macro #res4.
Using the name of the section is useful when components have got not standard names
Example: a resistor called D45 instead of R45

The S20.cad file is under the S20WIN directory. If the S20.CAD is copied also under the board directory with the same name, (S20.cad) this one will be used during the automatic generation of the program instead of the system one. In this way it is possible to customize the S20.cad file for different demands.

7.6 The .NOD file

It contains the list of the component coordinates

Example of file.nod

SIGNAL	COMPONENT	PIN	X	Y	PAD	POS	TECN	TEST	CHANNEL USER
N8	R5	1	10.0	10.0	100	T	T	Y	12
N8	R6	1	13.0	6.0	50	T	T	Y	0
N22	R91	2	34.0	94.0	30	T	S	Y	45
N18	R98	1	56.0	1.0	X100	B	T	Y	0
VCC	C5	1	78.0	57.0	X50Y30	T	T	Y	8
30V	C46	1	49.0	38.0	R30H10	B	S	N	0
/RFSH_7	IC12	3	10.0	4.0	30	B	T	Y	56
/CSRM_7	IC19	8	4.0	67.9	50	T	T	Y	0
Mechanical reference			43.0	28.0	0	O	M	N	

Data in this file are organized into fields separated by one or more characters that can be blanks or TAB.
Comment lines begin with an (*) and end with a (CR+LF).
It is advised to fill all fields except the USER one.

NOTE: The SPACE and TAB characters mustn't be present as values because they are separator characters.

SIGNAL: (max 20 characters) it contains the signal name

COMPONENT: (max 30 characters) it contains the component name relating to the signal name

Campo PIN: (integer number) it indicates the component pin

Campo X: (integer number) it is X coordinate of point (indicated in the signal field) expressed in mm

Campo Y: (integer number) it is Y coordinate of point (indicated in the signal field) expressed in mm

PAD: (integer number) indicates the pad dimensions expressed in mils.
Example: **X190Y254** indicates a rectangular pad with dimensions: 190x254 mils.
R30H10 indicates a round pad with 30 mils diameter an a 10 mils hole
X100 indicates a square pad 100x100 mils,
100 indicates a square pad of 100x100.

POS: (max 1 character) indicates the position of the point on the UUT that can be:
T for Top
B for Bottom
O for Top and Bottom (example a via hole)

TECN: (max 1 character) indicates the component technology. Possible value:

T for Through hole
S for SMD
M for mechanical reference

TEST: (max 1 character) indicates the pin testability. Possible values:
Y for testable pin
N for NOT testable pin
F for forced pin.
It is possible to define more that one testable point in a network. If there is an "F" value on a point, this one will be the one chosen for the test.
T for testable pin and network terminal (used mainly for Bare Board Tester)

CHANNEL: (integer number) indicates the channel number of the point .If this field is (0), the software assign a channel to the point automatically.

USER: (max 20 characters) optional field used to indicate the total number of probes that can't be used to test that point (because there are mechanical limitations, for example). It is not mandatory to fill this field.

7.7 The .POL file

In the File.pol is written the correspondence between test channels and component pins (it is obtained from CAD data normally). The information contained in the file will be used during diagnostic messages, to indicate also the component-pin instead of channel number only. This file is automatically generated choosing the Tool\Software\Network file generation but can be written by the programmer as well. The syntax is:

CHANNEL=;COMPONENT-PIN

Example:

262=;R134-1
264=;R129-2;R132-2;R136-2;R143-2;R149-2;R152-2;R155-2;R158-2;R161-2;R164-2;
268=;R136-1
270=;R137-1

7.8 The .SHI file

This file lists all adjacent points (indicated as signal names). The list is created during the Short generation process (CAD\Short Data Generation menu).

The first signal (N12 in the example) is adjacent to N13,N19,N3,N27,N25,N43,N28. For this reason a SHORT test will be generated in the File.srg.

Example of File.shi

* SHORT TEST

short N12 ==>,N13,N19,N3,N27,N25,N43,N28
short N20 ==>,N17,N50,N14,N16,N15,N4,N46
short N14 ==>,N15,N17,N51,N57,N28,N52

7.9 The .SHO file

This file lists all adjacent points (indicated as channel number). The list is created during the Short generation process (CAD\Short Data Generation menu).

The first number (76 in the example) indicates the test channel that will be adjacent with the others listed.

The second number (20 in the example) indicates the number of adjacencies that the first channel can have.

Example:

* SHORT TEST
76,20,251,252,9,10,24,25,61,255,261,127,248,125,1,96,18,23,58,126,6,3;
1,5,42,60,125,264,215;
307,1,26;

Explanation: In the first line, channel 76 has got 20 adjacent points and they are:
251,252,9,10,24,25,61,255,261,127,248,125,1,96,18,23,58,126,6,3;

7.10 The .SRG file

The file.srg lists all tests that will be executed on the UUT. Here below is given an example of file.srg. Normally the file.srg is composed of four blocks and it is advisable to follow the same structure

---

Capacitor discharge
Contact test (for bed of nails only) **Block 1**
Short circuit test

Not powered test of resistors, capacitors, …
Block 2

Powered test of Ics
Block 3

Final block for comments
Block 4

In the example below, a dashed line separates test blocks.

7.10.1 Example of File.srg

```
$include SAMPLE.LIB;

$section CAPACITORS DISCHARGE;

#DISCAP("DISC2105//C2106//C2107//C2",283.26,98,221);
#DISCAP("DISC2113//C2118",1.022,645,98);
#DISCAP("DISC2498",10,444,633);
#DISCAP("DISC2501",10,193,127);

@COMPARE("SHORTS2",1,ENATEST),MOV=N,BOK="ADJACENT_END";
LAB="ADJACENT_BEGIN",NER=Y,MOV=N,SCR="\SHORTS TEST ON ADJACENCY...";
$section SHORTS TEST ON ADJACENCY;

@ISO("SHORT 1-107",1,107);
@ISO("SHORT 1-647",1,647);
*@ISO("SHORT 195-196",195,196);  IN PARALLEL WITH "X4500"
@ISO("SHORT 665-670",665,670);
@ISO("SHORT 668-669",668,669);
LAB="ADJACENT_END",NER=Y,MOV=N;
LAB="ERRORE",JER="MESERR",NER=Y;
```

If there has been an error (short found) in the previous tests, the program jumps to the "MESERR" label and stops because other components may fail because of the short

```
*------------------------------------------------------------------------------------------------------------------------ fine blocco 1

* ANALOG TESTS

$section CONNECTORS;
LAB="CONNECTORS_BEGIN",NER=Y,MOV=N,SCR="\CONNECTORS TEST...";
#CNN("J1101",35,98,623,,6,36,13,12,98,89,
629,650,120,107,1,647,627,92,91,626,
645,644,,);
#CNN("J1102",98,86,98,87,648,617,118,73,67,69,
98,84,83,6,6,116,93,3,94,11,
98,98,,);
#CNN("J1530",100,100,178,150,149,148,100);
#CNN("J1531",130,129,101,101,170);
LAB="CONNECTORS_END",NER=Y,MOV=N;

$section FUSES;

LAB="FUSES_BEGIN",NER=Y,MOV=N,SCR="\FUSES TEST...";
#CTO("L5105",221,6);
#CTO("L5130",271,6);
*CTO("R3906",91,);
*CTO("X4324",,22);
LAB="FUSES_END",NER=Y,MOV=N;

$section RESISTORS;

LAB="RESISTORS_BEGIN",NER=Y,MOV=N,SCR="\RESISTORS TEST...";
#RES("R3108",4700,10,6,116);
#RES("R3109",47000,10,116,273);
```

```
#RES("R3111",180000,10,274,628);
#RESCAP("R3112//C2119",470000,10,0.082,50,20,98,628);
LAB="RESISTORS_END",NER=Y,MOV=N;

$section CAPACITORS;

LAB="CAPACITORS_BEGIN",NER=Y,MOV=N,SCR="\CAPACITORS TEST...";
#CAP("C2104",0.082,50,20,91,98);
#CAP("C2105//C2106//C2107//C2",283.26,50,20,98,221);
*#CAP("C2105",0.022,50,20,98,221); - P A R A L L E L -
*#CAP("C2106",0.1,50,20,98,479); - P A R A L L E L -
*#CAP("C2107",47,50,20,479,98); - P A R A L L E L -
@CON("C2512",33,25,100,178);
@CON("C2513",33,25,100,150);
LAB="CAPACITORS_END",NER=Y,MOV=N;

$section DIODES;

LAB="DIODES_BEGIN",NER=Y,MOV=N,SCR="\DIODES TEST...";
#DIO("D6120",259,256);
#DIO("D6130",116,636);
#DIO("D6155",94,431);
#DIO("D6300",293,292);
LAB="DIODES_END",NER=Y,MOV=N;

$section TRANSISTORS;

LAB="TRANSISTORS_BEGIN",NER=Y,MOV=N,SCR="\TRANSISTORS TEST...";
#TRN("U7121",10,6,261,256);
#TRN("U7182",10,137,136,139);
*TRN("U7503",10,100,,154);
*TRN("U7504",10,100,,178);
LAB="TRANSISTORS_END",NER=Y,MOV=N;

$section INDUCTANCES;

LAB="INDUCTANCES_BEGIN",NER=Y,MOV=N,SCR="\INDUCTANCES TEST...";
#IND("L5104",100,0.3,40,6,479);
#IND("L5480",0,0.3,40,99,451);
#IND("L5481",0,0.3,40,99,447);
LAB="INDUCTANCES_END",NER=Y,MOV=N;

$section ICS;

LAB="ICS_BEGIN",NER=Y,MOV=N,SCR="\ICS TEST...";
*#IC_SM_74HC4053D("U7205",398,306,398,308,306,98,98,98,485,486,122,411,85,306,384,6);
@DEF(VCC=0,GND=0,VCC=6,GND=98);
@Z("U7205_Z",,,3.0,,5174);
@CASE("U7205",0,398-5174,306-5174,398-5174,308-5174,306-5174,98-5174,98-5174,98-
5174,485-5174,486-5174,122-5174,411-5174,85-5174,306-5174,384-5174,6-5174);
*#IC_SM_74HC4053D("U7280",407,408,407,366,408,98,98,98,104,106,
*105,408,407,364,365,6);
@DEF(VCC=0,GND=0,VCC=6,GND=98);
@AUTIC("U7280.1",30,VCC-GND,407);
@AUTIC("U7280.2",30,VCC-GND,408);
```

```
@AUTIC("U7280.4",30,VCC-GND,366);
@AUTIC("U7280.9",30,VCC-GND,104);
@AUTIC("U7280.10",30,VCC-GND,106);
@AUTIC("U7280.11",30,VCC-GND,105);
@AUTIC("U7280.14",30,VCC-GND,364);
@AUTIC("U7280.15",30,VCC-GND,365);
LAB="ICS_END",NER=Y,MOV=N;
LAB="ERRORE1",JER="MESERR1",NER=Y;
```

If there has been an error in the previous tests, the program jumps to the "MESERR1" label and stops because it may be dangerous for the UUT to switch on the power supplies and continue the test

```
*------------------------------------------------------------------------------------------------------------------------ end block 2

$section POWER SUPPLIES SETTING;

LAB="ALION",NER=Y,WAI=500,PW1=5;
LAB="TESTALI",MES=5,LOW=4.5,HIG=5.5,LI4=VCC,BER="END";

$section POWERED TESTS;

LAB="POWER_BEGIN",NER=Y,MOV=N,SCR="POWERED TEST...";
#IC_SM_74HC4066D("U7230",335,331,421,335,354,354,98,341,327,328,
341,354,354,6);
#IC_SM_74HC4066D("U7240",98,354,417,98,485,485,98,48,,,
49,417,485,6);
LAB="POWER_END",NER=Y,MOV=N,JUM="END";

*GESTIONE ERRORI
LAB= "MESERR",SCR="ERRORE NEL TEST DEI CORTO CIRCUITI",NER=Y,JUM="END";
LAB= "MESERR1",SCR="ERRORE NEL TEST DEI COMPONENTI NON ALIMENTATI",NER=Y;

* PROGRAM END
LAB="END",NER=Y;

*------------------------------------------------------------------------------------------------------------------------ end block 3
*#RES("R3112",470000,10,628,98); - P A R A L L E L -
*RES("R3922",100,10,,420);   NON COLLAUDATO PERCHE' NON ACCESSIBILE
*RES4("R3504",4.7,10,,7,,7); NON COLLAUDATO PERCHE' NON ACCESSIBILE

*------------------------------------------------------------------------------------------------------------------------ end block 4
```

7.11 The .TAB file

In the file.tab there is the correspondence between a UUT network name and the test channel. This file is automatically generated when choosing CAD\Generate but can be written by the programmer as well. The syntax is:

"network name"=channel

Example:

```
PARAMETER=CHANNEL
"&N942"=14
"&N949"=2
```

"DCL_IO2"=3
"VCC"=4

7.12 The S20.CAD file

This file contains the correspondence between the component and the macro to use during test. It is divided into sections, one for each type of component. It is possible to activate different macros to test a specific component and it is also possible to define different macro parameters like the component value and the test tolerance. Moreover, there are some sections that describe the software procedure parameters (menu TOOLS\SOFTWARE in the S20WIN) and the whole list of library that can be added to the file.srg.
The s20.cad file is on the C:\S20WIN\BOARD directory and it is used during the automatic generation of the file.srg. The file.srg will be crated with the same sections of the s20.cad file and with the same order.

The s20.cad file syntax is:

Component: this field defines the component type (C for capacitors, R for resistors, J or JP for jumpers,...). It is also possible to assign a different name (defined by customer) to a specific component.

Macro description: this field indicates the macro to use during the test of the component (#RES macro to test a resistor, #DIO macro to test a diode, #RESCAP macro for a resistor and capacitor in parallel, ...). If a line begins with @X it means that it is used for the software procedure (TOOLS\SOFTWARE).

NOTE: The order of the test in the file.srg will be the same of the one in the s20.cad file.

S20.cad listing with the explanation and use (written in " ")

```
* WARNING: THE MACRO SECTIONS ORDER WILL BE THE SOURCE MACROS ORDER
* COMPONENT     MACRO DESCRIPTION

$SECTION CONNECTORS
* CONNECTORS
DIN*            #CNN(COMM,PINLIST);
SC*             #CNN(COMM,PINLIST);
SP*             #CNN(COMM,PINLIST);
TP*             #CNN(COMM,PINLIST);
J*              #CNN(COMM,PINLIST);
X*              #CNN(COMM,PINLIST);
```

"This section means that all components in the UUT that begins with DIN* (i.e. DIN12, DIN5,…), SC*,SP*,TP*,J*,X* in the CAD data will be tests with the CNN macro. For this reason, after the automatic generation of the test program it is possible to see lines as #CNN("DIN12",1,34,58,9,10,3); in the file.srg.
PINLIST means that all channels connected to the component will be listed in the macro"

```
$SECTION JUMPERS
* JUMPERS
I*              @ISO(COMM,P1,P2);
JP*             @ISO(COMM,P1,P2);
@X              #OPE(COMM,P1,P2);
@X              @ISOV(COMM,VALUE[V],VALUE[O],P1,P2);
@X              @ISOHV(COMM,VALUE[V],VALUE[O],P1,P2);
```

"see explanation above. The line @X #OPE(COMM,P1,P2); can be used to customize the test. For example: in one UUT, the connectors are called NM* instead of J*. It is possible to change the line into:
MN #OPE(COMM,P1,P2);. After the automatic generation, all MN* components will be tested with the OPE macro"

```
$SECTION FUSES
* FUSES
F           #CTO(COMM,P1,P2);
SW*         #CTO(COMM,P1,P2);
@X          @CTO(COMM,P1,P2);
```

"see explanation above"

```
$SECTION RESISTORS
* RESISTORS

R*          #RES4(COMM,VALUE[O]<10,TOL+[10],P1,P2,P1,P2);
```

"the RES4 macro will be used if the resistor value is lesser that 10OHM otherwise the RES macro will be used.
A tolerance of 10 will be used to test it. If different tolerance is requested, it is possible to modify the specific field"

```
R*          #RES(COMM,VALUE[O],TOL+[10],P1,P2);
@X          #RESV(COMM,VALUE[V][0.5],VALUE[O]>1000000,TOL+[10],P1,P2);
TRIM*  #TRIM(COMM,VALUE[O],TOL+[10],P1,P2,P3);
@X          #TRI(COMM,VALUE[O],TOL+[10],P1,P2,P3);
@X          #RESCON(COMM,VALUE[O],TOL+[10],,,P1,P2);
@X          #RESCONV(COMM,VALUE[V][0.5],VALUE[O],TOL+[10],,,P1,P2);
@X          #RESCAP(COMM,VALUE[O],TOL+[10],,,,P1,P2);
@X          #RESCAPV(COMM,VALUE[V][0.5],VALUE[O],TOL+[10],,,,P1,P2);
@X          @RESV(COMM,VALUE[V],VALUE[O],TOL+[10],P1,P2);
@X          @RESHV(COMM,VALUE[V],VALUE[O],TOL+[10],P1,P2);
@X          #RP108C2(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP168I1(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP84I1(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP168I2(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP84I2(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP109C(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP98C(COMM,VALUE[O],TOL+[10],PINLIST);
@X          #RP87C(COMM,VALUE[O],TOL+[10],PINLIST);

$SECTION CAPACITORS
* CAPACITORS
C*          #CAP(COMM,VALUE[uF],TOL+[50],TOL-[20],P1,P2);
C*          @CON(COMM,VALUE[pF]<1000,TOL+[25],P1,P2);
@X          #CON_AC_FP(COMM,VALUE[nF]<1000,TOL+[25],P1,P2,);
@X          #CON(COMM,VALUE[nF],TOL+[25],P1,P2);
@X          #CAPV(COMM,VALUE[V][0.5],VALUE[uF],TOL+[50],TOL-[20],P1,P2);
@X          #CONV(COMM,VALUE[V][0.5],VALUE[nF],TOL+[25],P1,P2);
@X          #CONFP(COMM,VALUE[nF],TOL+[25],P1,P2);
@X          #CON_AC(COMM,VALUE[nF],TOL+[25],P1,P2,);
@X          #CON_ACV(COMM,VALUE[V],VALUE[nF],TOL+[25],P1,P2,);
@X           #CON_ACF(COMM,0.8,VALUE[0],50,VALUE[nF],TOL+[25],P1,P2,);
@X          #RESCON(COMM,,,VALUE[nF],TOL+[25],P1,P2);
@X          #RESCONV(COMM,,,,VALUE[nF],TOL+[25],P1,P2);
@X          #RESCAP(COMM,,,VALUE[uF],TOL+[50],TOL-[20],P1,P2);
@X          #RESCAPV(COMM,,,,VALUE[uF],TOL+[50],TOL-[20],P1,P2);
```

"see explanation above"

```
$SECTION DIODES
* DIODES
CR*         #DIO(COMM,P1,P2);
```

```
D*          #DIO(COMM,P1,P2);
GR*         #DIO(COMM,P1,P2);
@X          #DIOV(COMM,VALUE[V][0.6],P1,P2);
@X          #DIO3(COMM,P1,P2,P3);
@X          #DIODE(COMM,P1,P2);
@X          #DIODE3(COMM,PINLIST);
@X          #BYV34(COMM,P1,P2,P3);
LED*        #DIOLED(COMM,P1,P2);
LD*         #DIOLED(COMM,P1,P2);
"see explanation above"

$SECTION TRANSISTORS
* TRANSISTORS
Q*          #TRN(COMM,10,P1,P2,P3);
T*          #TRP(COMM,10,P1,P2,P3);
@X          #TRA(COMM,10,P1,P2,P3);
"see explanation above"

$SECTION MOSFET
* MOSFET
M*          #MOSN(COMM,P1,P2,P3);
MF*         #MOSP(COMM,P1,P2,P3);
@X          #MOS(COMM,P1,P2,P3);
"see explanation above"

$SECTION INDUCTANCES
* INDUCTANCES
L*          #IND(COMM,VALUE[uH],0.3,TOL+[40],P1,P2);
@X          #IND_AC(COMM,VALUE[uH],TOL+[40],P1,P2,);
@X          #IND_ACV(COMM,VALUE[V],VALUE[uH],TOL+[40],P1,P2,);
@X          #IND_ACF(COMM,VALUE[V],VALUE[O],0.8,VALUE[uH],TOL+[40],P1,P2,);
"see explanation above"

$SECTION TRANSFORMERS
* TRANSFORMERS
TR*         #TRASF(COMM,VALUE[V],VALUE[V],1,TOL+[30],P1,P2,P3,P4);
@X          #TRASF_AC(COMM,VALUE[V],VALUE[V],1,TOL+[30],P1,P2,P3,P4);
"see explanation above"

$SECTION RELAIS
* RELAIS
RL*         #PICOREED(COMM,PINLIST);
K*          #RELAIS(COMM,VALUE[V],PINLIST);
"see explanation above"

$SECTION ZENERS
* ZENERS
DZ*         #ZEN(COMM,VALUE[V],TOL+[10],P1,P2);
Z*          #ZEN(COMM,VALUE[V],TOL+[10],P1,P2);
"see explanation above"

$SECTION ICS
* ICS
IC*         DEVICE(COMM,PINLIST);
U*          DEVICE(COMM,PINLIST);
REG*        #REG(COMM,VALUE[V],TOL+[5],1,P1,P2,P3);
"see explanation above"

$SECTION CLOCKS
* CLOCKS
```

```
Y*          #COUNT(COMM,VALUE[V],TOL+[10],P1);
QZ*         #COUNT(COMM,VALUE[V],TOL+[10],P1);
```
"see explanation above"

```
$SECTION PARALLEL
* PARALLEL
@X          #RESCAP(COMM,VALUE[O],TOL+[10],VALUE[uF],TOL+[50],TOL-[20],P1,P2);
@X          #RESCON(COMM,VALUE[O],TOL+[10],VALUE[nF],TOL+[25],P1,P2);
@X          #RESCAPV(COMM,,VALUE[O],TOL+[10],VALUE[uF],TOL+[50],TOL-[20],P1,P2);
@X          #RESCONV(COMM,,VALUE[O],TOL+[10],VALUE[nF],TOL+[50],P1,P2);
```
"Section related to the parallel finder (TOOLS\SOFTWARE menu, parallel finder option). Here are listed the macro to use during parallel finder calculation. Example: if a #RES and a #CAP macro are in parallel, they will be substituted with the #RESCAP macro. It is possible to change the name of the macro listed above but not their parameters. For Example if the line:

```
@X          #RESCAP(COMM,VALUE[O],TOL+[10],VALUE[uF],TOL+[50],TOL-[20],P1,P2);
```
is changed in:

```
@X          #RESCAP2(COMM,VALUE[O],TOL+[10],VALUE[uF],TOL+[50],TOL-[20],P1,P2);
```
during the parallel finder calculation a #RES macro and a #CAP2 macro will be substituted with s #RESCAP2"

modify

```
$SECTION SHORT
* SHORT
@X          $SECTION FUSES
@X          $SECTION INDUCTANCES
@X          #TRASF(COMM,VALUE[V],VALUE[V],1,TOL+[30],P1,P2,,);
@X          #TRASF(COMM,VALUE[V],VALUE[V],1,TOL+[30],,,P1,P2);
@X          #TRASF_AC(COMM,VALUE[V],VALUE[V],1,TOL+[30],P1,P2,,);
@X          #TRASF_AC(COMM,VALUE[V],VALUE[V],1,TOL+[30],,,P1,P2);
@X          #RES4(COMM,VALUE[O]<10,TOL+[10],P1,,P2,);
@X          #RES4(COMM,VALUE[O]<10,TOL+[10],,P1,,P2);
@X          $SECTION RESISTORS VALUE[O]<=5
@X          $SECTION CAPACITORS VALUE[mF]>10
```
"Section related to the short finder (TOOLS\SOFTWARE menu, short finder option)
All macro listed in the section (included those listed in the SECTION FUSES and SECTION INDUCTANCE) will be considered as Shorts and automatically inserted in the group of shorted points inside the @SHORT macro.
Example: @SHORT("SHORT TEST", 5, 1/32,1-13-22-45); will execute the short test from channel 1 to 32 considering channels 1,13, 22 and 45 shorted together (imagine there is a transformer or an inductance connected to these points)"

```
$SECTION DISCHARGE
* DISCHARGE
@X      $SECTION CAPACITORS VALUE[uF]>=1.0
```
"Section related to the capacitor discharge (TOOLS\SOFTWARE menu, capacitor discharge option). It means that all electrolytic capacitors with value ≥ 1 uF will be discharged at the beginning of the test program so there will be a #DISCAP macro into the file.srg.
It is possible to modify the value if requested"

```
$SECTION GUARDING
* GUARDING
@X          $SECTION RESISTORS
@X          #CAP(COMM,VALUE[uF],TOL+[50],TOL-[20],P1,P2);
@X          #CON(COMM,VALUE[nF],TOL+[25],P1,P2);
@X          #CAPV(COMM,VALUE[V][0.5],VALUE[uF],TOL+[50],TOL-[20],P1,P2);
@X          #CONV(COMM,VALUE[V][0.5],VALUE[nF],TOL+[25],P1,P2);
@X           #CONFP(COMM,VALUE[nF],TOL+[25],P1,P2);
```

"Section related to the analog guarding generation (TOOLS\SOFTWARE menu, analog guarding option). It means that all macro(components) included into the RESISTOR sections plus #CAP,#CON,#CAPV, #CONV,#CONFP will be taken into account during the automatic guarding procedure so there can be a GUA command in the file.srg.

It is possible to add or deled lines if a customization is requested
NOTE: By default the analog guarding option works on those components listed in the UTILITIES\CONFIGURATION menu, button SOFTWARE, Guarding macros field (#RES,#CAP,#CON;). To enable the use of the section GUARDIN in the S20.cad file, the Guarding macros field must be emptied."

$include S20ANA.LIB;
$include S20UGEN.LIB;
$include S20USER.LIB;
"All libraries listed here above will be included in the file.srg at the beginning of the program"

NOTE: if one or more component pin is missing (not connected) in the UUT description, the macro will be generated in any case if the PINLIST parameter is used in the macro description.

Example:

C* #CAP(COMM,VALUE[uF],TOL+[50],TOL-[20],PINLIST);

NOTE: if one or more component pin is missing (not connected) in the UUT description, the macro will be generated beginning with an asterisk (commented, no test executed for the specified component) if the P1,P2 parameter is used in the macro description. This is done to permits the user to compile the file.srg even if some pins are not connected.

Example:

C* #CAP(COMM,VALUE[uF],TOL+[50],TOL-[20],P1,P2);

7.13 The S20ANA.LIB file

The S20ANA:LIB contains the macro library for in-circuit analogical test. These macros can be called by the file.srg. Here below is an example

```
**** ANALOG COMPONENTS LIBRARY    *** WARNING: RES & CAP MEASURED AT 0.3 V *********
*#CONTACT("COMMENT",CHN_FROM,CHN_TO);
*#SHO("COMMENT",CHN_FROM,CHN_TO);
*#IMPMES("COMMENT",CHN);
*#DISCAP("COMMENT",MICROFARAD,CHN+,CHN-);
*#BOARD("COMMENT",CHN1,CHN2);
*#OPE("COMMENT",CHN1,CHN2);
*#CTO("COMMENT",CHN1,CHN2);
*#TRI("COMMENT",OHM,%TOL,CHN1,WIPER,CHN2);
*#TRIM("COMMENT",OHM,%TOL,CHN1,WIPER,CHN2);
*#RES("COMMENT",OHM,%TOL,CHN1,CHN2);
*#RESV("COMMENT",VOLT,OHM,%TOL,CHN1,CHN2);
*#RES4("COMMENT",OHM,%TOL,CHN1,CHN2,CHN3,CHN4);
*#RESCON("COMMENT",OHM,%RESTOL,NANOFARAD,%CONTOL,CHN1,CHN2);
*#RESCONV("COMMENT",VOLT,OHM,%RESTOL,NANOFARAD,%CONTOL,CHN1,CHN2);
*#RESCAP("COMMENT",OHM,%RESTOL,MICROFARAD,%CAP+TOL,*CAP-TOL,CHN1,CHN2)
*#RESCAPV("COMMENT",VOLT,OHM,%RESTOL,MICROFARAD,%CAP+TOL,%CAP-TOL,CHN1,CHN2);
*#CON("COMMENT",NANOFARAD,%TOL,CHN1,CHN2);
*#CONV("COMMENT",VOLT,NANOFARAD,%TOL,CHN1,CHN2);
*#CAP("COMMENT",MICROFARAD,%TOL+,%TOL-,CHN+,CHN-);
*#CAPV("COMMENT",VOLT,MICROFARAD,%TOL+,%TOL-,CHN1,CHN2);
*#DIO("COMMENT",ANODECHN,CATHODECHN);
*#DIOV("COMMENT",VOLT,ANODECHN,CATHODECHN);
```

```
*#DIOLED("COMMENT",ANODECHN,CATHODECHN);
*#ZEN("COMMENT",VOLT,%TOL,KATOECHN,ANODECHN);
*#TRZ("COMMENT",VOLT,%TOL,CHN1,CHN2);
*#TRP("COMMENT",GAIN,EMITTERCHN,BASECHN,COLLECTORCHN);
*#TRN("COMMENT",GAIN,EMITTERCHN,BASECHN,COLLECTORCHN);
*#MOSP("COMMENT",DRAINCHN,GATECHN,SOURCECHN);
*#MOSN("COMMENT",DRAINCHN,GATECHN,SOURCECHN);
*#REG("COMMENT",VOLTREG,%TOL,TIM,INCHN,REFCHN,OUTCHN);
*#IND("COMMENT",MICROHENRY,VOLT,%TOL,CHN1,CHN2);
*#TRASF("COMMENT",VOLTPRI,VOLTSEC,TIM,%TOL,PRI1CHN,PRI2CHN,SEC1CHN,SEC2CHN);
*#PICOREED("COMMENT",CON1CHN,,COIL+CHN,,COIL-CHN,,CON2CHN);
*#RELAIS("COMMENT",VOLT,COIL+CHN,COIL-CHN,COMMONCHN,CLOSECHN,OPENCHN);
*#COUNT("COMMENT",IMPULSENR,%TOL,CHN);
*#CNN("COMMENT",..);
*#PINAD("COMMENT",..);
*#CONFP(COMM,VAL,CONTOL,P1,P2);
*#DIO3(COMM,P1,P2,P3);
*#BYV34(COMM,P1,P2,P3);
*#DIOBRIDGE(COMM,PHASE,NEUTRAL,POS,NEG)
*#SCR (COMM,ANODO,KATODO,GATE)
********************************************** LIBRARY WITH MACRO FOR RESISTOR ARRAY  ****
*#RP108C2(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10)
*#RP168I1(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14,P15,P16)
*#RP84I1(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8)
*#RP168I2(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14,P15,P16)
*#RP84I2(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8)
*#RP109C(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10)
*#RP98C(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8,P9)
*#RP87C(COMM,VALO,PERC,P1,P2,P3,P4,P5,P6,P7,P8)
************************************ LIBRARY WITH MACRO FOR AUTOMATIC DEBUG FOR S20 ****
*#TRA(COMM,GA,EMIT,BASE,COLL)
*#DIODE(COMM,P1,P2)
*#DIODE3(COMM,..)
*#MOS("COMMENT",DRAINCHN,GATECHN,SOURCECHN);
```

Appendix A

8. APPENDIX A

8.1 SYSTEM EDITOR

The S20 system editor is used to work on documents in the S20 Windows environment. It has got the following features:

- It is compatible with the Microsoft Windows™ edit control.
- The size of the text that can be handled is only limited by the amount of memory Windows™ can globally allocate.
- It has got a great variety of cursor movement.
- It has got the capability of line marking, column marking and block marking
- It has got the ability to jump to a specific line and to set bookmarks in the text.
- It permits the operator to define keyboard macros.
- It has got the capability to search and substitute strings in the text.

In this appendix, the main S20 editor commands are described together with the search and replace commands that make the S20 editor a very powerful tool.

8.1.1 MAIN COMMANDS

The editor commands are here described grouping them in different classes to facilitate the location.

CURSOR COMMANDS

COMMAND	FUNCTION
LEFT	Moves the cursor left of one character
RIGHT	Moves the cursor right of one character
CTRL+LEFT	Moves the cursor to the beginning of the previous word
CTRL+RIGHT	Moves the cursor to the beginning of the following word
HOME	Moves the cursor to the beginning of the line
END	Moves the cursor to the end of the line
CTRL+PGUP	Moves the cursor to the previous paragraph
CTRL+PGDN	Moves the cursor to the following paragraph
CTRL+HOME	Moves the cursor to the first character in the document
CTRL+END	Moves the cursor to the last character in the document
PGUP	Moves the cursor one page up
PGDN	Moves the cursor one page down
UP	Moves the cursor up of one line
DOWN	Moves the cursor down of one line
CTRL+UP	Moves the cursor to the first character in the window
CTRL+DOWN	Moves the cursor to the lower left-hand corner in the window
F3	Moves the cursor to a specified line in the document
INSERT	Toggles the insert/overstrike mode

BLOCK COMMANDS

COMMAND	FUNCTION
CTRL+INSERT	Copies marked text and put it in the Clipboard
F8	Cuts marked text and puts it in the Clipboard
SH+F8	Cuts marked text and appends it to the text already present in the Clipboard
CTRL+X	Cuts marked text and put it into the Clipboard. If there is no marked text, the entire line is inserted into the Clipboard
ALT+D	Cuts marked text and put it into the Clipboard. If there is no marked text, the entire line is inserted into the Clipboard
F7	Marks all the lines written between two points. **USE:** Press F7, move with the arrows, press again F7. All the lines between these two points will be marked.
SH+F7	Marks all columns present between two points. **USE:** Press SH+F7, move with the arrows, press SH+F7 again. All columns between these two points will be marked
ALT+B	Marks all columns present between two points. **USE:** Press ALT+B, move with the arrows, press ALT+B again. All columns between these two points will be marked
CTRL+F8	Clears the selection of marked text
ALT+U	Clears the selection of marked text
SH+F9	Copies marked text into the Clipboard
CTRL+C	Copies marked test into the Clipboard
F9	Pastes the content of the Clipboard
CTRL+V	Pastes the Clipboard content to the right of the cursor position
ALT+Z	Pastes the Clipboard content to the right of the cursor position
SH+DELETE	Deletes the marked text if present, otherwise it deletes the line
CTRL+BACKSPACE	Deletes the marked text if present, otherwise it deletes the line
CTRL+L	Changes marked text from lower to upper case letters
CTRL+U	Changes marked text from upper to lower case letters

COMMANDS TO FORMAT THE TEXT

COMMAND	FUNCTION
TAB	Indents text if the line/lines are marked
SH+TAB	Removes the indentation from the marked line/lines
DELETE	Deletes the current character
BACKSPACE	Deletes the previous character
CTRL+DELETE	Deletes the current word
F6	Deletes from the cursor to the end of the line
CTRL+K	Deletes from the cursor to the end of the line
SH+F6	Deletes from the cursor to the beginning of the line
CTRL+F9	Duplicates the current line
ALT+A	Duplicates the current line
F5	Deletes a line
ALT+ENTER	Inserts a line
ALT+L	Selects a line

STRING SEARCH AND REPLACE COMMANDS

COMMAND	FUNCTION
SH+F5	Search for a specified string
CTRL+F	Searches for a string
CTRL+F5	Repeats the last search
CTRL+E	Repeats the last string search or replace
CTRL+R	Replaces a string with another one

UNDO COMMANDS

COMMAND	FUNCTION
CTRL+A	Redoes the last UNDO operation
CTRL+Z	Undoes the last operation

COMMANDS TO OPEN THE MENUS

COMMAND	FUNCTION
CTRL+B	Selects the BOARD\select menu
CTRL+G	Selects the CLUSTER\SIMULATOR\Pattern Compiler. This command is active only if a File.pat is open.
CTRL+H	Selects the CLUSTER\Select menu
CTRL+J	Selects the CLUSTER\Autolearn menu
CTRL+T	Selects the RUN\Test menu
CTRL+D	Selects the RUN\Debug menu
CTRL+M	Selects the RUN\Compiler menu
CTRL+N	Selects the BOARD\New menu
CTRL+O	Selects the BOARD\Open menu
CTRL+P	Selects the BOARD\Print menu
CTRL+Q	Selects the UTILITIES\Configuration menu
CTRL+Y	Selects the UTILITIES\Diagnostic menu
CTRL+W	Selects the TOOLS\Software menu

MACRO COMMANDS

COMMAND	FUNCTION
ALT+M	Permits the operator to define keyboard macros. It is possible to define, up to ten macros for frequently performed operations. **USE**: Press ALT+M, perform the operations you want to include in the macro, press ALT+M, insert the number associated to the macro (a whole number from 0 to 9) To execute the saved macro, press CTRL+n where n is the number of the macro
ALT+P	Executes the last stored macro
CTRL+0÷9	Executes the stored macros
CTRL+ALT+C	Inserts the dummy #CON macro
CTRL+ALT+D	Inserts the dummy #DIO macro
CTRL+ALT+E	Runs the configured external editor
CTRL+ALT+I	Inserts the dummy #IND macro
CTRL+ALT+L	Inserts the following line “$include S20ANA.LIB”
CTRL+ALT+O	Inserts the dummy #OPE macro
CTRL+ALT+R	Inserts the dummy #RES macro
CTRL+ALT+S	Inserts the dummy @SHORT macro
CTRL+ALT+T	Inserts the dummy #TRN macro
CTRL+ALT+Z	Inserts the dummy #ZEN macro

COMMANDS FOR FILES

COMMAND	FUNCTION
F2	Saves the file
CTRL+S	Saves the file
F4	Close the file
SH+F2	Opens the ERR.ICT document
SH+F3	Opens the S20ANA.LIB document
SH+F4	Opens the S20LOG.LIB file
CTRL+F6	Switches from one document to another when more than one is open

HELP COMMANDS

COMMAND	FUNCTION
SH+INSERT	Actives the guided editor on the marked text. The button must be pressed to activate the guided

	editor
F1	Activates the help for the word pointed by the cursor
SH+F1	Activates the cursor help

8.1.2 STRING SEARCH

The editor allows you to search forwards or backwards through a file for a certain "pattern" of text. In addition to searching for a series of characters, you can search for text that has a certain "property". For instance , you can ask the editor to locate all words that begin with the letter 'Q', or for all occurrences of three digits in a row. When you invoke the search command, the editor will ask you the text to search for. Certain characters have a special meaning in the search pattern that you type. Here is a list of the special characters than can be used during the search process.

CHARACTER	MEANING	EXAMPLE
string	Matches the specified string	***Example:*** abc, will match an 'a' followed by a 'b' followed by a 'c'.
?	Matches any single character	***Example:*** a?c will match an 'a' followed by any character followed by a 'c' ***Example:*** beat Matches 'be', followed by any character, followed by a 't'. (***Example:*** beat, belt, bent)
*	Matches zero or more occurrences of the previous character	***Example:*** ab* will find an 'a' followed by zero or more b's ***Example:*** ab*c matches an 'a', followed by zero or more 'b's, followed by a 'c'.
+	Matches one or more occurrences of the previous character	***Example:*** ab+ will find an 'a' followed by one or more b's
[c1-c2]	Matches any character within the class. If there is a dash encountered in the class, then all characters starting from the character before the dash and ending with the character after the dash will be included in the class	***Example:*** [a-e] will match 'a','b','c','d', or 'e'. ***Example:*** [a-ex-z] will match 'a' through 'e' or 'x' through 'z'
[^c1-c2]	Matches any character NOT IN the class	***Example:*** [^a-e] will match any character that is not in lower case. ***Example:*** [A-Z][0-9] matches a capital letter followed by a digit ***Example:*** [^A-Za-z0-9] matches a character which is not alphanumeric ***Example:*** [A-Z][A-Z]*[.?,!] matches a capitalized word
\	Takes the next character literally. (Used when the special characters must be taken literally).	***Example:*** matches 'ab*c'. Note that "*" tells the pattern matcher to take the asterisk literally
^	Anchors the match at the beginning of the line	***Example:*** ^abc finds a line that begins with "abc".
$	The match succeeds if at the end of the line	***Example:*** abc$ finds a line which ends with "abc" ***Example:*** ^$ matches a line with no characters (the beginning of the line followed by the end of the line) ***Example:*** ^abc$ matches a line which contains only 'abc'
a|b	Matches pattern 'a' or pattern 'b'	***Example:*** if|else matches the word 'if' or the word 'else' ***Example:*** if|else|for matches the word 'if' or 'else' or 'for'
\c	If the match was successful, places the cursor at that position	***Example:*** ab*\cc matches an 'a' followed by zero or more 'b's, followed by a 'c' and places the cursor over the 'c' after the match is completed. The "\c" places the cursor at the position after "ab*" is matched
\t	Matches a tabulation in the text	
\(a\)	Tags pattern <a> This tag can be referred to in subsequent parts of the search pattern or in the substitution pattern	***Example:*** \(abc\)[0-9]\1 matches 'abc', any digit, then 'abc'. Note that "\1" refers to tagged pattern 1, which is 'abc'
\n	Refers to the tagged pattern <n>. N is a number from 1 to 9.	***Example:*** \([A-Z]+\)?*\1 matches a line that contains two or more non-contiguous occurrences of the same upper-case word. Note that ?* indicate zero or more occurrences of any character and "\1" refers to whatever [A-Z]+ matched
{}	Parenthesizes an expression	***Example:*** {abc}+ search for one or more occurrences of "abc"
:a	Matches all alphanumeric characters. It is the equivalent of [a-zA-Z0-9]	
:b	Matches all white space characters. It is the equivalent of [\t]+ where \t indicates a tabulation	
:c	Matches all alphabetic characters. It is the equivalent of [a-zA-Z]	
:d	Matches all numeric characters. It is the equivalent of [0-9]	
:h	Matches all hex numbers. It is the equivalent of [0-9a-fA-F]+	
:i	Matches all integer numbers. It is the equivalent of [0-9]+	
:l	Valid C variable. It is the equivalent of [a-zA-Z][a-zA-Z 0-	

	9]+	
:w	Matches a word. It is equivalent to [a-zA-Z]+	

Note: To search for a “special character”, you must prefix that character with a backslash. ***(Example:*** if you wanted to search for a question mark, the search pattern would be “\?”. To search for a backslash, the pattern is “\\”.)

8.1.3 STRING SUBSTITUTION

Like in the string search, also in the string replacement process, there are characters with a special meaning. In fact, whenever the '&' character is encountered in the replacement pattern, the entire matched pattern will be substituted at that point. ***Example:*** let's say that you specified 'abc' as the pattern to search for. A replacement pattern of '&xxx' would change 'abc' to 'abcxxx'. Likewise, a replacement pattern of '12&&34' would change 'abc' to '12abcabc 34'. To use a '&' literally in the replacement pattern, precede the '&' by a backslash ('\').

You may also include a tagged pattern in the replacement pattern. ***Example:*** let's say that the you want to replace the **dog=cat** string with the **cat=dog** string.

The search pattern will be: \(dog\)=\(cat\)

and the replacement pattern is:

\2=\1 that means: string 2 (cat) = string 1 (dog)

In the following example the line **currentline->prev=newline;** must be replaced with the line **newline->next=currentline;**

The search pattern is \([a-z]+\)->prev=\([a-z]+\);

and the substitution pattern is:

\2->next=\1;